

PRSV Analyzer

Papaya Ring Spot Virus Diagnostic Web Application

Complete Technical Documentation — Version 3.2.0

A-to-Z system reference: architecture, every module, every route,
every data flow, installation, testing, and security — built strictly from
the actual shipped source code as the single source of truth.

Project Technical Documentation

Prepared: September 2026

Scope and method of this document. Every claim in this documentation is derived directly from inspecting the real, shipped project source code inside the delivered `PRSV-ANALYZER-v3.2.zip` archive — its Python modules, HTML templates, JavaScript, configuration files, tests, and generated artifacts (trained model files, evaluation reports, changelogs). Nothing here is invented. Where a design decision, historical rationale, or runtime behavior cannot be established from the available files, this document says so explicitly rather than guessing, using the phrase *“Not verifiable from the available project files”*. Real file names, function names, class names, route paths, and variable names are used throughout instead of generic placeholders, so that any statement in this document can be checked directly against the source tree it describes.

Contents

I	Project Overview	1
1	What This Project Is	2
1.1	Problem Statement	2
1.2	Objectives	2
1.3	Scope and Non-Goals	3
1.4	Target Users	3
1.5	High-Level Feature List	4
2	Technology Stack	6
2.1	Backend Framework and Web Layer	6
2.2	Image Processing and Numerical Computing	6
2.3	Machine Learning and Explainability	7
2.4	RAG, NLP, and LLM Integration	8
2.5	Infrastructure, Caching, MLOps, and Reporting	9
2.6	Testing	9
2.7	Deliberately Excluded / Deferred Technologies	9
3	Directory Structure	11
3.1	Complete Directory Tree	11
3.2	Directory-by-Directory Explanation	13
II	Backend Architecture	15
4	Application Entry Point: app/main.py	16
4.1	What Runs at Startup	16
4.2	Application Object Construction	17
5	Configuration: app/config.py	18
5.1	The Settings Class	18
5.2	Configuration Fields	18
5.3	Derived Properties	20
6	Security Middleware: app/middleware.py	21
6.1	ApiKeyAndRateLimitMiddleware	21
7	JSON API Routes	23
7.1	app/routes/api_health.py — /api/health	23
7.2	app/routes/api_analysis.py — /api/analysis	23
7.2.1	Path-Parameter Validation: Preventing Path Traversal	25
7.3	app/routes/api_chatbot.py — /api/chatbot	25
8	Data Contracts: app/schemas.py	27

9	The Core Analysis Pipeline: <code>app/services/analysis_service.py</code>	29
9.1	Why This File Is the Heart of the System	29
9.2	Step-by-Step Trace	29
9.3	Persisted JSON Files Per Image	31
10	Run Records, Batch Orchestration, and Background Jobs	32
10.1	<code>app/services/run_manager.py</code> — <code>RunManager</code> and <code>RunContext</code>	32
10.2	<code>app/services/batch_service.py</code> — <code>BatchService</code>	32
10.3	<code>app/services/run_store.py</code> — <code>RunStore</code>	33
10.4	<code>app/services/job_queue.py</code> — <code>BatchJobQueue</code>	34
11	Supporting Services: Cache, Cleanup, Dataset, Export	35
11.1	<code>app/services/cache_service.py</code> — <code>CacheService</code>	35
11.2	<code>app/services/cleanup_service.py</code> — <code>run_cleanup</code>	35
11.3	<code>app/services/dataset_service.py</code> — <code>DatasetService</code>	36
11.4	<code>app/services/export_service.py</code> — <code>ExportService</code>	36
III	The Computer Vision and Machine Learning Pipeline	37
12	Image Preprocessing, Segmentation, and Symptom Detection	38
12.1	Shared Constants: <code>image_processing/constants.py</code>	38
12.2	<code>image_processing/preprocess.py</code>	38
12.3	<code>image_processing/segmentation.py</code>	39
12.4	<code>image_processing/symptom_enhancement.py</code>	39
12.5	<code>image_processing/feature_extraction.py</code> and <code>texture_features.py</code>	40
13	Severity Estimation	42
13.1	<code>image_processing/severity.py</code> — The Original Weighted Formula	42
13.2	<code>image_processing/severity_v2.py</code> — Lesion-Ratio Severity with Bootstrap Interval	43
14	Heatmaps and Quality Assessment	45
14.1	<code>image_processing/heatmaps.py</code>	45
14.2	<code>image_processing/quality_checks.py</code>	45
14.3	<code>image_processing/segmentation_quality.py</code>	46
14.4	<code>image_processing/visual_debug.py</code> — An Unwired Utility Module	46
15	Extended (Opt-In, Not Live-Wired) Image Processing Modules	48
15.1	<code>image_processing/texture_extended.py</code>	48
15.2	<code>image_processing/color_spaces_extended.py</code>	49
15.3	<code>image_processing/illumination_normalization.py</code>	49
15.4	<code>image_processing/segmentation_classical.py</code>	49
15.5	<code>image_processing/weather_fusion.py</code> — v3.2	50
16	Classification: Model Loading and Inference	51
16.1	<code>ml/feature_schema.py</code>	51
16.2	<code>ml/model_loader.py</code> — <code>load_model_artifacts</code>	51
16.3	<code>ml/infer_svm.py</code> — <code>predict_with_svm</code>	52
16.4	The Actual Shipped Model: What Is In <code>models/</code>	52
17	Model Training, Evaluation, and SHAP Explainability	54
17.1	<code>ml/train_svm.py</code> — <code>train_svm_from_feature_csv</code>	54
17.2	<code>ml/evaluate.py</code> and <code>ml/metrics.py</code>	54

17.3	ml/shap_explainer.py — explain_prediction	55
18	Calibration, Ensembling, and Extended Explainability	56
18.1	ml/calibration.py — calibrate_model	56
18.2	ml/ensemble.py — StackingClassifier	56
18.3	ml/train_ensemble.py — Real, Executed Results	57
18.4	xai/lime_explainer.py — explain_with_lime	57
18.5	xai/surrogate_tree.py — distill_surrogate_tree	58
19	Feature Quality, Active Learning, Federated Learning, and LoRA Groundwork	59
19.1	ml/feature_quality.py — analyze_feature_quality	59
19.2	ml/active_learning.py — Built, Not Yet Applicable	59
19.3	ml/federated_learning.py — A Real, Executed Simulation	60
19.4	rag/qa_dataset_builder.py and ml/lora_finetune.py — Fine-Tuning Groundwork	61
IV	Retrieval-Augmented Generation and the Chatbot	62
20	The Knowledge Base and Core RAG Pipeline	63
20.1	rag/kb/prsv_knowledge.json — The Knowledge Base	63
20.2	rag/knowledge_loader.py	63
20.3	rag/schemas.py	63
20.4	rag/retriever.py — LocalTfidfRetriever	63
20.5	rag/query_builder.py — Turning Pipeline Output Into a Query	64
20.6	rag/generator.py — The Deterministic Template Generator	64
20.7	rag/hybrid_retriever.py — HybridRetriever	64
20.8	app/services/rag_service.py — RagService	65
21	Advanced RAG: Dense Retrieval and Grounded LLM Generation	67
21.1	rag/embeddings_retriever.py — DenseEmbeddingRetriever	67
21.2	rag/llm_generator.py — generate_grounded_explanation	68
22	RAG Quality Layers: Reranking, Expansion, Correction, Graph, Structured Output, Routing, Evaluation	70
22.1	rag/reranker.py — Cross-Encoder Reranking	70
22.2	rag/query_expansion.py — expand_query	70
22.3	rag/corrective_rag.py — grade_and_decide	71
22.4	rag/knowledge_graph.py — A Real Graph Over the 30-Chunk KB	71
22.5	rag/structured_output.py — Schema-Constrained Generation	72
22.6	rag/agent_orchestrator.py — route	72
22.7	rag/rag_eval.py — RAGAS-Lite Evaluation	72
23	The Chatbot: Service, Translation, and NLP	74
23.1	app/services/chatbot_service.py — ChatbotService	74
23.2	app/services/translation_service.py	76
23.3	nlp/ — The Four NLP Modules, Each With an Explicit Honesty Note	76
V	Evaluation Rigor, MLOps, and Sustainability	78
24	Statistical Evaluation and Evaluation Rigor Modules	79
24.1	evaluation/statistical_tests.py	79

24.2	evaluation/holdout_eval.py — Honest-by-Design	80
24.3	evaluation/stress_testing.py	80
24.4	evaluation/fairness_audit.py	81
24.5	evaluation/trust_scores.py — Jiang et al. (2018)	81
24.6	evaluation/pdp_ice.py	81
24.7	evaluation/shap_interactions.py	81
25	MLOps: Experiment Tracking, Model Registry, Drift Detection, ONNX Export	83
25.1	mlops/experiment_tracking.py — MLflow Wrapper	83
25.2	mlops/model_registry.py — ModelRegistry	83
25.3	mlops/drift_detection.py	84
25.4	mlops/onnx_export.py — Verified, Executed	84
25.5	sustainability/energy_benchmark.py	85
VI	The Frontend	86
26	Server-Rendered Pages: app/routes/pages.py and Templates	87
26.1	No JavaScript Framework — By Architecture, Not Omission	87
26.2	app/routes/pages.py — The HTML Page Router	87
26.3	Shared Helpers in pages.py	89
26.4	Template Structure	89
26.5	Client-Side JavaScript	89
26.6	CSS	90
VII	Security, Testing, and Operations	91
27	Security Documentation	92
27.1	Implemented	92
27.2	Partially Implemented	93
27.3	Not Implemented / Absent	93
VIII	Testing, Installation, and Deployment	95
28	Testing Documentation	96
28.1	Overview and Philosophy	96
28.2	Test Framework and Configuration	96
28.3	Running the Suite Locally	97
28.4	Test File to Module Mapping	97
28.4.1	What Is Notably <i>Not</i> Covered	99
28.5	Continuous Integration	100
28.6	Interpreting a Local Test Failure	100
29	Installation, Configuration, and Deployment	101
29.1	Prerequisites	101
29.1.1	A Note on Windows and NumPy/Compiler Failures	101
29.2	Option A: One-Click Setup via <code>run.bat</code>	101
29.3	Option B: Manual Setup (Any Platform)	102
29.3.1	Optional Dependencies	102
29.4	Running the Test Suite After Setup	103

29.5	Environment Variables and Configuration Reference	103
29.6	Deployment: Render.com	104
29.6.1	What This Blueprint Actually Configures	104
29.6.2	Free-Tier Limitations (Documented Honestly)	105
29.6.3	Deploying via the Blueprint	106
30	Complete User Journeys	107
30.1	Journey 1: Single-Image Analysis (Web Form)	107
30.2	Journey 1b: Single-Image Analysis (JSON API)	108
30.3	Journey 2: Batch Analysis (Multiple Files)	108
30.4	Journey 3: ZIP Upload Analysis	108
30.5	Journey 4: Demo Dataset Analysis	109
30.6	Journey 5: Chatbot Conversation (Text)	109
30.7	Journey 5b: Chatbot Photo-in-Chat	110
30.8	Journey 6: Browsing Run History	110
30.9	Journey 7: Exporting a Report	110
30.10	Journey 8: Checking System Status	111
31	Debugging Guide	112
31.1	The App Won't Start	112
31.2	Analysis Requests Fail or Return Unexpected Results	112
31.3	The Chatbot Gives Unhelpful or Ungrounded Answers	113
31.4	Tests Fail	113
31.5	Deployment-Specific Issues	113
IX	Master Architecture and Summary	115
32	How Everything Connects	116
32.1	Top-Level System Architecture	116
32.2	The Core Analysis Pipeline, Step by Step	117
32.3	The RAG and Chatbot Flow	117
32.4	Dependency and Call-Relationship Map	117
33	Master Summary Package	119
33.1	The One-Paragraph Explanation	119
33.2	The 5-Minute Explanation	119
33.3	The 15-Minute Explanation	120
33.4	The Technical Deep-Dive	121
33.5	Complete Feature Map	121
33.6	Complete File Map	123
33.7	Complete Technology Map	123
33.8	Complete Data-Flow Map	124
33.9	Complete API Map	124
33.10	Complete Database Map	125
33.11	Closing Note	125

Part I

Project Overview

Chapter 1

What This Project Is

1.1 Problem Statement

Papaya Ring Spot Virus (PRSV) is a viral disease of papaya plants that produces visible leaf symptoms (ring-shaped chlorotic and necrotic spots, mosaic mottling, and leaf distortion). Smallholder farmers who suspect their plants are infected often have no fast, low-cost way to get a second opinion without physically bringing a sample to an agricultural extension office. The PRSV Analyzer addresses this narrow problem: given a photograph of a papaya leaf, produce an automated Healthy/Diseased prediction, a severity estimate, a visual explanation (heatmap overlays and feature-importance plots), and a plain-language, locally-sourced explanation of what the result means and what to do next — all inside a single self-hosted web application, with no dependency on a live internet connection for its core diagnostic path.

The project is explicit, in its own `README.md`, about what it is not: it is described as “*a research/demo system, not a validated diagnostic tool*”. This documentation follows that same framing throughout: it describes what the system actually does and how, without overstating clinical or scientific validity that the project itself does not claim.

1.2 Objectives

Based on the shipped code, tests, and documentation, the system’s working objectives are:

1. Accept a papaya leaf image (single file, a batch inside a ZIP, or a built-in demo dataset) and classify it as *Healthy* or *Diseased*.
2. Produce a severity estimate for diseased leaves, expressed as a percentage and a descriptive bucket (e.g. *Mild to moderate*), while never reporting a severity value for a leaf classified as *Healthy*.
3. Explain *why* the model reached its prediction, using multiple complementary explainability techniques (SHAP, LIME, a distilled surrogate decision tree, partial dependence, trust scores) rather than treating the classifier as a black box.
4. Generate a plain-language explanation of the result grounded in a curated PRSV knowledge base, using retrieval-augmented generation (RAG) rather than an unconstrained language model, so the explanation text is traceable to a specific, human-curated source document.
5. Offer a conversational chatbot interface (multilingual, with voice input/output and photo-in-chat diagnosis) so a farmer can ask follow-up questions in natural language, including in one of nine supported Indian languages in addition to English.

6. Keep a persistent, inspectable record of every analysis run (individual and batch) on disk, so results can be revisited, exported as PDF/CSV/ZIP reports, or compared over time.
7. Provide the underlying machine-learning pipeline (feature extraction, model training, calibration, ensembling, evaluation, explainability, drift detection, and experiment tracking) as inspectable, testable, independently runnable code rather than a single opaque inference call, since the stated audience explicitly includes people who want to look under the hood (researchers, students, portfolio reviewers), not only end-user farmers.
8. Run this entire system as free/local software with no mandatory paid service dependency: every feature that touches a paid API (Anthropic Claude for grounded generation, in particular) is designed to be fully optional, with the system degrading gracefully to a local, offline equivalent when no key is configured.

1.3 Scope and Non-Goals

In scope (implemented and present in the shipped code): image upload and preprocessing; leaf segmentation; handcrafted feature extraction; SVM-based classification (plus a calibrated variant and a stacking ensemble); severity estimation (two independent methods); SHAP/LIME/surrogate-tree/PDP/trust-score explainability; a hybrid BM25+dense-embedding RAG pipeline with optional LLM generation, reranking, query expansion, corrective RAG, and a lightweight knowledge graph; a multilingual, voice-capable, NLP-assisted chatbot; batch analysis over uploaded ZIP files or a bundled demo dataset; PDF/CSV/ZIP report export; a run history/dashboard UI; ONNX model export; a Flower-based federated learning simulation; a curated LoRA fine-tuning dataset and training script; weather-based spread-risk heuristics; MLflow experiment tracking; a JSON model registry with rollback; drift detection; DVC-based data/model versioning; and a pytest suite of 125 tests.

Explicitly out of scope or only partially built (documented honestly in the project's own `ROADMAP_v3.md` rather than hidden): a native or cross-platform mobile app (the ONNX export exists and is verified to match the scikit-learn model numerically, but no mobile app shell consumes it); execution of the LoRA fine-tuning script against a real GPU (the script and dataset are real and complete, but were never actually run to produce fine-tuned model weights, because the sandbox this project was built in has no GPU and cannot install the multi-gigabyte `torch/transformers/peft/bitsandbytes` stack); live, continuously-running federated learning across real distributed devices (a real Flower *simulation* was executed and its results are recorded, but this is a single-machine simulation of the federated protocol, not a live multi-device deployment); fully wired multi-turn dialogue state and coreference resolution in the chatbot API route layer (the underlying `nlp/dialogue_state.py` module exists and is tested in isolation, but the project's own changelog notes it needs a session-id concept to be connected end-to-end through the chatbot route); and subgroup/fairness auditing exercised against real data (the `evaluation/fairness_audit.py` module is implemented and tested, but the shipped dataset has no subgroup/source column, so the audit has never been run against real subgroup labels). Each of these is called out again, with more detail, in the relevant later chapter, and is never presented as fully working end-to-end.

1.4 Target Users

Three audiences can be identified directly from the project's structure and documentation:

- **Farmers / end users** — the primary web UI (`app/templates/home.html`, `analyze.html`, `result.html`) and the chatbot widget (`app/templates/partials/chatbot_widget.html`) are de-

signed for a non-technical person uploading a photo and reading a plain-language result, with multilingual support aimed at this audience specifically.

- **Agricultural researchers / extension workers** — the `methodology.html` page, the batch-analysis and dataset tooling (`app/services/dataset_service.py`, `app/services/batch_service.py`), and the evaluation/statistics modules (`evaluation/`) serve a more technical reviewer who wants to inspect methodology and aggregate results, not just one photo’s verdict.
- **Developers / ML engineers extending the system** — the `scripts/` directory (retraining, labeling, feature extraction), the MLOps modules (`mlops/`), the test suite, and the `ROADMAP_v3.md` / `CHANGELOG_v3.*.md` files are written for someone who will read and modify the code, not only run the packaged application.

1.5 High-Level Feature List

The following is the complete, real feature list of the shipped v3.2 system, grouped by area. Every item is backed by an identifiable module, route, or test in the codebase; later chapters trace each one in detail.

Image ingestion	Single-image upload, ZIP-of-images batch upload, and a bundled demo dataset (<code>Original Images/</code>), all routed through <code>app/routes/api_analysis.py</code> and <code>app/routes/pages.py</code> .
Preprocessing	Resize, CLAHE contrast enhancement, denoising (<code>image_processing/preprocess.py</code>), plus v3.1 illumination normalization (Retinex, shadow detection, gray-world white balance) in <code>image_processing/illumination_normalization.py</code> .
Segmentation	HSV-threshold leaf segmentation (<code>image_processing/segmentation.py</code>) used in the live pipeline, plus three classical alternative segmenters (GrabCut, Watershed, SLIC superpixels) in <code>image_processing/segmentation_classical.py</code> that are implemented, tested, and verified against a real dataset image but not wired into the live <code>/api</code> inference path.
Feature extraction	A 12-feature vector combining the original 7 global features with v3.1 texture/color/vegetation-index extensions (<code>image_processing/feature_extraction.py</code> , <code>texture_extended.py</code> , <code>color_spaces_extended.py</code>).
Classification	RBF-kernel SVM baseline (<code>ml/infer_svm.py</code> , <code>ml/train_svm.py</code>), a <code>CalibratedClassifierCV</code> wrapper (<code>ml/calibration.py</code>), and a <code>StackingClassifier</code> ensemble of SVM + <code>RandomForest</code> + <code>GradientBoosting</code> with a logistic-regression meta-learner (<code>ml/ensemble.py</code> , <code>ml/train_ensemble.py</code>).
Explainability	SHAP (<code>ml/shap_explainer.py</code>), LIME (<code>xai/lime_explainer.py</code>), a distilled global surrogate decision tree (<code>xai/surrogate_tree.py</code>), SHAP interaction values, PDP/ICE curves, and nearest-neighbor trust scores (all under <code>evaluation/</code>).
Severity estimation	The original weighted-formula estimator (<code>image_processing/severity.py</code>) and a v3.0 lesion-area-ratio estimator with bootstrap confidence intervals (<code>image_processing/severity_v2.py</code>).
RAG explanation generation	Hybrid BM25 + dense-embedding retrieval with Reciprocal Rank Fusion

(`rag/hybrid_retriever.py`), optional real LLM generation via the Anthropic API (`rag/llm_generator.py`), template-based fallback generation (`rag/generator.py`), cross-encoder reranking (`rag/reranker.py`), query expansion (`rag/query_expansion.py`), corrective RAG relevance grading (`rag/corrective_rag.py`), a knowledge graph over the KB (`rag/knowledge_graph.py`), structured-output generation (`rag/structured_output.py`), an agentic router (`rag/agent_orchestrator.py`) and a RAGAS-lite evaluation harness (`rag/rag_eval.py`).

Chatbot / conversational AI

Multilingual text chat with translation fallback (`app/services/translation_service.py`), intent classification, symptom/plant-part extraction, urgency detection, and dialogue-state tracking (`nlp/`), all orchestrated by `app/services/chatbot_service.py` and exposed via `app/routes/api_chatbot.py`.

Batch processing / dataset tooling

ZIP and demo-dataset batch runs (`app/services/batch_service.py`, `app/services/dataset_service.py`), a persistent run store (`app/services/run_store.py`, `run_manager.py`), and a background job queue (`app/services/job_queue.py`).

Reporting / export

PDF, CSV, and ZIP report generation (`app/services/report_service.py`, `app/services/export_service.py`).

MLOps

MLflow experiment tracking (`mlops/experiment_tracking.py`), a JSON-file model registry with rollback (`mlops/model_registry.py`), PSI/KS-test drift detection (`mlops/drift_detection.py`), and ONNX export with numerical verification (`mlops/onnx_export.py`).

Federated learning

A real, executed Flower FedAvg simulation over partitions of the real dataset (`ml/federated_learning.py`).

LLM fine-tuning groundwork

A curated 41-pair Alpaca-format Q&A dataset generator (`rag/qa_dataset_builder.py`) and a complete QLoRA fine-tuning script for Phi-3-mini (`ml/lora_finetune.py`), the latter not executed in this sandbox.

Weather / environmental fusion

Live Open-Meteo API integration and an illustrative spread-risk heuristic (`image_processing/weather_fusion.py`).

Evaluation rigor

McNemar’s test and calibration reporting (`evaluation/statistical_tests.py`), an honest-by-design holdout evaluator (`evaluation/holdout_eval.py`), adversarial stress testing under blur/brightness/JPEG/occlusion perturbations (`evaluation/stress_testing.py`), and subgroup fairness auditing infrastructure (`evaluation/fairness_audit.py`).

Data versioning

DVC-tracked training data and model artifacts, documented in `docs/DVC_SETUP.md`.

Security

Optional API-key gating and a sliding-window rate limiter on all `/api/` routes (`app/middleware.py`), strict upload validation (`app/utils/validation_utils.py`), and a debug-off-by-default configuration posture (`app/config.py`).

Sustainability

CPU-time-proxy (or CodeCarbon-measured, if installed) inference energy benchmarking (`sustainability/energy_benchmark.py`).

Chapter 2

Technology Stack

This chapter lists every real dependency declared in `prsv_project/requirements.txt` and `prsv_project/requirements_optional.txt`, what it is used for, and where. “Base” means installing `requirements.txt` alone is enough to run the app and the full test suite; “Optional” means the corresponding module degrades gracefully (documented per-module in later chapters) when the package is absent.

2.1 Backend Framework and Web Layer

Package	Tier	Role in this project
<code>fastapi</code>	Base	The web framework itself. All routing (<code>app/routes/</code>), dependency injection (<code>app/dependencies.py</code>), and the <code>app/main.py</code> application object are built on it.
<code>uvicorn[standard]</code>	Base	ASGI server that actually runs the FastAPI app (invoked as <code>uvicorn app.main:app</code> per the README’s run instructions).
<code>jinja2</code>	Base	Server-side HTML templating for every page under <code>app/templates/</code> (home, analyze, result, run detail, dashboard, status, error, methodology, batch result, image detail).
<code>python-multipart</code>	Base	Required by FastAPI/Starlette to parse <code>multipart/form-data</code> bodies, i.e. the image and ZIP file uploads handled in <code>app/routes/api_analysis.py</code> and <code>pages.py</code> .
<code>pydantic</code>	Base	Data validation/serialization. Every request and response shape in <code>app/schemas.py</code> is a Pydantic model.
<code>pydantic-settings</code>	Base	Backs <code>app/config.py</code> ’s <code>Settings(BaseSettings)</code> class, which loads configuration from <code>.env</code> with typed fields and validation.

2.2 Image Processing and Numerical Computing

Package	Tier	Role in this project
numpy	Base	The array/numeric substrate underneath essentially every module in <code>image_processing/</code> , <code>ml/</code> , and <code>evaluation/</code> .
opencv-python-headless	Base	Core CV operations: resizing, color space conversion, CLAHE, denoising, Canny/Sobel/Laplacian edge detection, morphological operations, contour finding, Grab-Cut and Watershed segmentation. The <code>-headless</code> variant is used deliberately because the app runs server-side with no GUI, avoiding an unnecessary GTK/Qt dependency.
Pillow	Base	General image I/O and format handling alongside OpenCV, used where PIL's API is more convenient (e.g. thumbnail/format conversions in utility code).
scikit-image	Base	Texture and structural analysis: GLCM (<code>graycomatrix/graycoprops</code>), local binary patterns, and SLIC superpixel segmentation (<code>image_processing/segmentation_classical.py</code>).
PyWavelets	Base (v3.0 add)	Wavelet-domain texture energy features in <code>image_processing/texture_extended.py</code> , wrapped so the feature is skipped gracefully if the package were ever absent.

2.3 Machine Learning and Explainability

Package	Tier	Role in this project
scikit-learn	Base	The core ML library: SVC (the baseline classifier), StandardScaler, LabelEncoder, GridSearchCV, StratifiedKFold, CalibratedClassifierCV, StackingClassifier, RandomForestClassifier, GradientBoostingClassifier, LogisticRegression, DecisionTreeClassifier, NearestNeighbors, SGDClassifier (used for the federated simulation because it supports <code>partial_fit</code>), PCA, RFECV, LDA, and <code>partial_dependence</code> .
scipy	Base (v3.0 add)	Statistical tests in <code>evaluation/statistical_tests.py</code> (binomial test / chi-square with continuity correction for McNemar's test) and <code>ks_2samp</code> for drift detection in <code>mlops/drift_detection.py</code> .
joblib	Base	Serialization format for every persisted model artifact under <code>models/</code> (<code>svm_model.joblib</code> , <code>scaler.joblib</code> , <code>label_encoder.joblib</code> , <code>shap_background.joblib</code>).
matplotlib	Base	Renders the static evaluation plots shipped in <code>models/evaluation/</code> (confusion matrix, ROC curve, precision-recall curve) and any chart generation in the evaluation modules.

Package	Tier	Role in this project
shap	Base	<code>KernelExplainer</code> (used with a k-means-summarized background in <code>ml/shap_explainer.py</code>) and <code>TreeExplainer</code> (used for SHAP interaction values against the tree members of the stacking ensemble in <code>evaluation/shap_interactions.py</code>).
lime	Base (v3.0 add)	<code>lime_tabular.LimeTabularExplainer</code> in <code>xai/lime_explainer.py</code> , explicitly used in tabular mode (not image/superpixel mode) because the classifier consumes the 12 engineered features, not raw pixels.
sentence-transformer	Optional	Backs the dense-embedding half of hybrid RAG retrieval (<code>rag/embeddings_retriever.py</code> , model <code>paraphrase-multilingual-MiniLM-L12-v2</code>), the cross-encoder reranker (<code>rag/reranker.py</code> , model <code>cross-encoder/ms-marco-MiniLM-L-6-v2</code>), and the embedding-based intent classifier (<code>nlp/intent_classifier.py</code>). Pulls in PyTorch transitively, hence kept optional.
faiss-cpu	Optional	Vector index backend for <code>DenseEmbeddingRetriever</code> ; the module falls back to a plain numpy matrix-multiply similarity search when FAISS is not installed.
skl2onnx	Optional (v3.2 add)	Converts the <code>StandardScaler</code> + <code>SVC</code> pipeline to ONNX format in <code>mlops/onnx_export.py</code> .
onnxruntime	Optional (v3.2 add)	Runs the exported ONNX graph to numerically verify it matches the scikit-learn pipeline’s own predictions, in the same module.
flwr[simulation]	Optional (v3.2 add)	Federated learning simulation framework in <code>ml/federated_learning.py</code> ; pulls in <code>ray</code> as the simulation backend.

2.4 RAG, NLP, and LLM Integration

Package	Tier	Role in this project
rank-bm25	Base (v3.0 add)	<code>BM25Okapi</code> , the sparse-retrieval half of <code>rag/hybrid_retriever.py</code> .
anthropic	Base (v3.0 add)	Official Anthropic Python SDK, used in <code>rag/llm_generator.py</code> (grounded explanation generation), <code>rag/structured_output.py</code> (tool-use structured diagnosis output), <code>rag/query_expansion.py</code> (LLM-based query rewriting), and <code>rag/agent_orchestrator.py</code> (LLM-based tool-use routing). All of these check <code>is_configured()</code> / the presence of <code>ANTHROPIC_API_KEY</code> first and fall back to a non-LLM path when no key is set, so the package being installed does not by itself cause any network calls or cost.

Package	Tier	Role in this project
deep-translator	Base	Machine translation for the multilingual chatbot (<code>app/services/translation_service.py</code>), with an offline phrasebook fallback when translation is unavailable.

2.5 Infrastructure, Caching, MLOps, and Reporting

Package	Tier	Role in this project
redis	Base (v3.0 add)	Optional cache backend for <code>app/services/cache_service.py</code> ; falls back to an in-process <code>_InMemoryCache</code> dict when no <code>REDIS_URL</code> is configured or the package/service is unreachable.
mlflow	Optional (v3.0 add)	Experiment tracking in <code>mlops/experiment_tracking.py</code> , via a <code>tracked_run</code> context manager that becomes a full no-op when <code>mlflow</code> is not installed or <code>MLFLOW_TRACKING_URI</code> is unset.
codecarbon	Optional (v3.1 add)	Measured (not proxy-estimated) energy tracking in <code>sustainability/energy_benchmark.py</code> ; needs OS/hardware power-draw access (e.g. Intel RAPL) that is often unavailable in sandboxed containers, so a CPU-time-based proxy estimate is the documented fallback.
reportlab	Base	PDF generation for exported analysis reports in <code>app/services/report_service.py</code> .
requests	Base (v3.2 add)	HTTP client used in <code>image_processing/weather_fusion.py</code> to call the Open-Meteo forecast API.

2.6 Testing

Package	Tier	Role in this project
pytest	Base	Test runner for all 125 tests under <code>tests/</code> .
httpx	Base	Used by FastAPI's <code>TestClient</code> (built on <code>httpx</code>) to drive HTTP-level tests against the app without a real running server.

2.7 Deliberately Excluded / Deferred Technologies

Two package groups are present in `requirements-optional.txt` only as *commented-out* guidance, never installed or executed in this project's own development sandbox:

- `torch`, `transformers`, `peft`, `bitsandbytes`, `accelerate`, `datasets` — the full stack `ml/lora_finetune.py` would need to actually run QLoRA fine-tuning of `microsoft/Phi-3-mini-4k-instruct`. The comment directly above them in `requirements-optional.txt` reads “NOT installable here / needs a GPU”, and instructs uncommenting them “on a GPU machine”.

No frontend JavaScript framework (React, Vue, etc.) is used anywhere in the project; the entire client side is server-rendered Jinja2 HTML plus hand-written vanilla JavaScript files under `app/static/js/`. This is a deliberate architectural characteristic of the project, not an omission, and is documented in full in Chapter 26.

Chapter 3

Directory Structure

3.1 Complete Directory Tree

The tree below is the real, complete layout of the delivered archive (excluding the `.git` internal object store, the DVC cache's hashed blob storage under `.dvc/cache/`, and the raw `Original Images/` demo dataset's individual image files, which are hundreds of leaf photographs and not meaningfully listable file-by-file).

```
1 PRSV-ANALYZER-v3.2/
2 |-- .github/workflows/ci.yml # GitHub Actions CI (test-base + test-full jobs)
3 |-- .dvc/ # DVC internal config + cache (data/model versioning)
4 |-- .gitignore
5 |-- .dvcignore
6 |-- CHANGELOG_v3.0.md
7 |-- CHANGELOG_v3.1.md
8 |-- CHANGELOG_v3.2.md
9 |-- README.md # Primary project documentation
10 |-- ROADMAP_v3.md # Honest "what's built / blocked / why" tracker
11 |-- render.yaml # Render.com deployment config
12 |-- run.bat # Windows convenience launcher
13 |-- docs/DVC_SETUP.md # DVC usage instructions
14 |-- Original Images/ # Raw demo dataset (papaya leaf photos), DVC-tracked
15 '-- prsv_project/ # The actual application package (Python import root)
16     |-- .env # Local environment configuration (not committed secrets)
17     |-- .env.example # Template for .env
18     |-- .gitignore
19     |-- labels_template.csv # Template for manual labeling
20     |-- pytest.ini # Registers the "slow" pytest marker
21     |-- requirements.txt # Base (always installable) dependencies
22     |-- requirements-optional.txt # Heavier / GPU-only dependencies
23     |
24     |-- app/ # FastAPI application (web layer)
25     | |-- __init__.py (implicit via package structure)
26     | |-- main.py # FastAPI app object, startup/shutdown lifecycle
27     | |-- config.py # Settings (pydantic-settings) - single source of config
28     | |-- dependencies.py # FastAPI dependency-injection helpers
29     | |-- middleware.py # API key + rate limit middleware
30     | |-- schemas.py # All Pydantic request/response models
31     | |-- routes/
32     | | |-- pages.py # HTML page routes (server-rendered UI)
33     | | |-- api_analysis.py # /api/analyze* JSON routes
34     | | |-- api_chatbot.py # /api/chat* JSON routes
35     | | '-- api_health.py # /api/health
36     |-- services/ # Business logic layer, called by routes
37     | |-- analysis_service.py # Orchestrates the full image->result pipeline
38     | |-- batch_service.py # ZIP / multi-image batch orchestration
39     | |-- dataset_service.py # Demo-dataset batch orchestration
40     | |-- run_manager.py # Coordinates a "run" (single or batch) end-to-end
41     | |-- run_store.py # Persists/reads run + image records on disk (JSON)
42     | |-- chatbot_service.py # Chat turn handling (intent/NLP/RAG orchestration)
43     |-- rag_service.py # RAG retrieval + explanation generation
```

```

44 | | -- translation_service.py # Multilingual translation + phrasebook fallback
45 | | -- cache_service.py # Redis-or-in-memory caching layer
46 | | -- report_service.py # PDF/CSV report generation
47 | | -- export_service.py # ZIP export of a run's outputs
48 | | -- cleanup_service.py # Retention-based deletion of old run folders
49 | | '-- job_queue.py # Lightweight in-process background job queue
50 | -- utils/ # Small stateless helpers used across the app
51 | | -- file_utils.py, id_utils.py, image_utils.py, json_utils.py,
52 | | logging_utils.py, path_utils.py, validation_utils.py, zip_utils.py,
53 | | '-- display_labels.py
54 | -- templates/ # Jinja2 HTML templates (server-rendered UI)
55 | | -- base.html, home.html, analyze.html, result.html, run_detail.html,
56 | | batch_result.html, image_detail.html, status.html, methodology.html,
57 | | error.html
58 | | '-- partials/chatbot_widget.html
59 | '-- static/
60 | -- css/style.css, chatbot.css
61 | -- js/app.js, analyze.js, upload.js, dashboard.js, chatbot.js
62 | '-- img/ (favicons, PWA icons, logo)
63 |
64 | -- image_processing/ # Classical computer-vision pipeline
65 | | -- preprocess.py, segmentation.py, segmentation_classical.py,
66 | | segmentation_quality.py, symptom_enhancement.py, feature_extraction.py,
67 | | texture_features.py, texture_extended.py, color_spaces_extended.py,
68 | | illumination_normalization.py, severity.py, severity_v2.py, heatmaps.py,
69 | | quality_checks.py, visual_debug.py, weather_fusion.py, constants.py
70 |
71 | -- ml/ # Model training, inference, and ML-adjacent tooling
72 | | -- feature_schema.py, model_loader.py, infer_svm.py, train_svm.py,
73 | | calibration.py, ensemble.py, train_ensemble.py, evaluate.py, metrics.py,
74 | | shap_explainer.py, feature_quality.py, active_learning.py,
75 | | federated_learning.py, lora_finetune.py
76 |
77 | -- xai/ # Explainability beyond SHAP
78 | | -- lime_explainer.py, surrogate_tree.py
79 |
80 | -- rag/ # Retrieval-augmented generation subsystem
81 | | -- kb/prsv_knowledge.json # The 30-entry curated knowledge base
82 | | -- knowledge_loader.py, retriever.py, embeddings_retriever.py,
83 | | hybrid_retriever.py, reranker.py, query_builder.py, query_expansion.py,
84 | | generator.py, llm_generator.py, structured_output.py, corrective_rag.py,
85 | | knowledge_graph.py, agent_orchestrator.py, rag_eval.py, schemas.py,
86 | | qa_dataset_builder.py
87 |
88 | -- nlp/ # Chatbot NLP building blocks
89 | | -- intent_classifier.py, symptom_ner.py, urgency_detection.py, dialogue_state.py
90 |
91 | -- evaluation/ # Model evaluation and rigor tooling
92 | | -- statistical_tests.py, holdout_eval.py, stress_testing.py,
93 | | fairness_audit.py, trust_scores.py, pdp_ice.py, shap_interactions.py
94 |
95 | -- mlops/ # Model lifecycle operations
96 | | -- experiment_tracking.py, model_registry.py, drift_detection.py, onnx_export.py
97 |
98 | -- sustainability/ # Green-AI / energy tooling
99 | | -- energy_benchmark.py
100 |
101 | -- scripts/ # Standalone CLI scripts (not imported by the app)
102 | | -- auto_label_from_filenames.py, create_label_template.py, prepare_manifest.py,
103 | | extract_features_dataset.py, retrain_model.py, cleanup_outputs.py
104 |
105 | -- models/ # Persisted trained model artifacts (checked-in binaries)
106 | | -- svm_model.joblib, scaler.joblib, label_encoder.joblib,
107 | | shap_background.joblib, metadata.json, training_summary.json
108 | | '-- evaluation/confusion_matrix.png, roc_curve.png,
109 | | precision_recall_curve.png, evaluation_metrics.json
110 |
111 | -- data/ # Runtime data directories + DVC-tracked datasets
112 | | -- training_features.csv (+ .dvc pointer)
113 | | -- lora_qa_dataset.json (+ .dvc pointer)
114 | | -- demo_manifest.csv
115 | | '-- uploads/, extracted/, processed/, outputs/, temp/, logs/ (empty at rest;
116 | | runtime-populated, tracked only via .gitkeep placeholders)

```

```

117 |
118 | -- reports/ # Generated report output directory (runtime-populated)
119 | -- tests/ # 125 pytest tests, one file per module/feature area

```

3.2 Directory-by-Directory Explanation

prsv_project/app/ — the web application

This is the only directory that talks HTTP. It is deliberately layered: `routes/` contains thin FastAPI path-operation functions that parse requests and call into `services/`; `services/` contains the actual business logic and is what `routes/` depends on; `schemas.py` defines the Pydantic contracts that cross the HTTP boundary; `utils/` holds small stateless helpers with no framework dependency, reused by both routes and services. This mirrors a standard controller/service/-model layering even though FastAPI does not enforce one.

image_processing/, ml/, xai/ — the diagnostic pipeline

These three packages together implement the actual computer-vision and machine-learning pipeline that turns a leaf photo into a prediction. `image_processing/` never imports from `ml/` or vice versa in a way that creates a cycle; `ml/infer_svm.py` imports feature-extraction output (a plain list of floats) rather than image-processing internals, keeping the boundary clean.

rag/ and nlp/ — language and knowledge

`rag/` is exclusively about retrieving and generating text grounded in `rag/kb/prsv_knowledge.json`; `nlp/` is exclusively about understanding the user's own chat message (intent, urgency, entities). The chatbot service (`app/services/chatbot_service.py`) is the only place that imports from both.

evaluation/, mlops/, sustainability/ — model lifecycle, not request-time code

None of these three packages is imported by `app/main.py` or any route. They are used by the standalone scripts in `scripts/` and by direct test invocation. This is an intentional separation: the live web service's request path does not depend on MLflow, DVC, ONNX, or Flower being installed or configured at all.

scripts/ — operator-run, not user-run

Every file here has a `if __name__ == "__main__":` entry point and is meant to be invoked directly from a shell (python `scripts/retrain_model.py`), never imported by the running web application.

models/ — checked-in binary artifacts

Unlike `data/`, this directory's core files (`svm_model.joblib`, `scaler.joblib`, `label_encoder.joblib`) are real, trained artifacts shipped inside the archive itself, not empty placeholders — this is what the README means when it says the zip “ships with a real trained model” and is not running on `inference_mode: "heuristic_fallback"` by default. The directory as a whole is also tracked as a single DVC-versioned unit (`models.dvc` at the repository root).

data/ — a mix of tracked datasets and empty runtime directories

`data/training_features.csv` and `data/lora_qa_dataset.json` are real, DVC-tracked data files. The remaining subdirectories (`uploads/`, `extracted/`, `processed/`, `outputs/`, `temp/`, `logs/`) ship

containing only a `.gitkeep` placeholder file each; they are populated at runtime by the application (see Chapter 10) and are excluded from git via `prsv_project/.gitignore` except for those placeholders, so a fresh checkout always has the directories present but empty.

tests/ — one file per feature area

There is a close, almost one-to-one mapping between test file names and the source modules described in this documentation (e.g. `test_severity_v2.py` tests `image_processing/severity_v2.py`, `test_onnx_export.py` tests `mlops/onnx_export.py`). This mapping is used throughout this document: each module’s chapter cross- references the test file that exercises it.

Part II

Backend Architecture

Chapter 4

Application Entry Point: `app/main.py`

4.1 What Runs at Startup

`app/main.py` builds and configures the single `FastAPI` instance that is the whole application. Startup and shutdown are handled by an `asynccontextmanager` called `lifespan`, registered on the `FastAPI(...)` constructor via `lifespan=lifespan` — this is the modern `FastAPI` startup pattern (replacing the older `@app.on_event("startup")` decorator style).

On startup, in order, `lifespan` does the following:

1. `ensure_dirs([...])` — creates every runtime data directory (`upload_dir`, `extracted_dir`, `processed_dir`, `output_dir`, `temp_dir`, `log_dir`, `models_dir`, `reports_dir`) if it does not already exist, using `app/utils/path_utils.py`'s `ensure_dirs` helper. This is why a fresh check-out with empty `data/` subfolders still works: the app self-heals the directory layout before serving any request.
2. `_warn_if_no_trained_model()` — calls `ml.model_loader.load_model_artifacts(settings)` and checks `artifacts.model_available`. If false, it prints and logs a large, impossible-to-miss ASCII banner warning that every prediction will use `inference_mode="heuristic_fallback"` (a hand-tuned formula, not a trained classifier), and gives the exact two commands to fix it: `python scripts/auto_label_from_filenames.py` then `python scripts/retrain_model.py`. This function exists specifically so a missing/corrupted model file is a loud, obvious startup event rather than a silent quality regression discovered later.
3. `_warm_up_shap_explainer()` — calls `ml.shap_explainer.explain_prediction` once, on a dummy 12-element feature vector of all 0.5s, with prediction label "Healthy". The function's own docstring states the reason: SHAP's first import/use in a process has an observed one-time cost of roughly 7 seconds, so triggering that cost once at startup keeps the very first real user request from paying it. The whole call is wrapped in `try/except Exception`, so if SHAP warm-up fails for any reason (e.g. the package genuinely missing), the app still starts and SHAP simply lazy-loads on first real use instead — startup failure here is explicitly treated as non-fatal.
4. `_start_scheduled_cleanup()` — starts a daemon `threading.Thread` running an infinite loop that, once every `_CLEANUP_INTERVAL_SECONDS` (a module constant hardcoded to $24 \times 60 \times 60 = 86400$ seconds, i.e. 24 hours), calls `run_cleanup(settings)` from `app/services/cleanup_service.py` if `settings.output_retention_days > 0`. This means old run folders under `data/outputs/` are deleted automatically in the background without an external cron job or scheduled task, controlled purely by the `OUTPUT_RETENTION_DAYS` environment variable (0 disables it). The thread body catches and logs any exception with `_logger.exception(...)`, with an explicit comment

that “cleanup must never crash the app”. `lifespan` keeps the returned `threading.Event` and calls `.set()` on it after the `yield`, so the thread is signaled to stop on application shutdown.

After `yield` (i.e. after the app has served for however long the process ran), the only shutdown action is stopping that cleanup thread. There is no database connection pool, message broker connection, or other resource to release — consistent with the project’s actual persistence model (flat JSON/CSV files on disk, described in Chapter 10), which needs no explicit teardown.

4.2 Application Object Construction

```

1 app = FastAPI(
2     title=settings.app_name,
3     version=settings.app_version,
4     debug=settings.debug,
5     lifespan=lifespan,
6 )
7
8 app.add_middleware(ApiKeyAndRateLimitMiddleware, settings=settings)
9
10 app.mount("/static", StaticFiles(directory=str(ROOT_DIR / "app" / "static")), name="static")
11 app.mount("/outputs", StaticFiles(directory=str(settings.output_dir)), name="outputs")
12
13 app.include_router(pages_router)
14 app.include_router(analysis_router)
15 app.include_router(health_router)
16 app.include_router(chatbot_router)

```

Listing 4.1: `app/main.py` – app construction (excerpt)

Four things happen here, in an order that matters:

- `title/version/debug` come straight from `settings`, so the FastAPI-generated OpenAPI docs (`/docs`, `/redoc`) always reflect the real configured app name and version string (`"3.2.0"`), and `debug` is off unless `DEBUG=true` is explicitly set in `.env` — the config module’s own comment calls out that Starlette/FastAPI “will happily return stack traces to any caller when this is on”, which is why the default is `False`.
- `add_middleware` registers `ApiKeyAndRateLimitMiddleware` (Chapter 6) as the single piece of cross-cutting HTTP middleware in the whole app.
- **Two static mounts:** `/static` serves `app/static/` (CSS/JS/images — fixed application assets), while `/outputs` serves `settings.output_dir` (`data/outputs/` — the *runtime-generated* per-run image results, heatmaps, and report files). Separating these into two mounts means the browser can request e.g. `/outputs/run_2026_.../images/img_.../heatmap_edge.png` directly as a plain static file, without a dedicated route or controller for serving result images.
- **Four routers** are included, each owning a distinct URL prefix (documented individually below): `pages_router` (no prefix — the HTML UI), `analysis_router` (`/api/analysis`), `health_router` (`/api/health`), and `chatbot_router` (`/api/chatbot`).

Chapter 5

Configuration: `app/config.py`

5.1 The `Settings` Class

All configuration in the project passes through exactly one class: `Settings(BaseSettings)` in `app/config.py`, instantiated once at import time as the module-level singleton `settings = Settings()`. Every other module that needs configuration does `from app.config import settings` and reads a field or property off that one object — there is no second configuration mechanism anywhere in the codebase.

`Settings` is built on `pydantic-settings`, configured with:

```
1 model_config = SettingsConfigDict(  
2     env_file=".env",  
3     env_file_encoding="utf-8",  
4     extra="ignore",  
5 )
```

This means every field can be overridden by an environment variable (or a value in `prsv_project/.env`), matched by each field's explicit `alias` (e.g. `app_version: str = Field(default="3.2.0", alias="APP_VERSION")` is set via the `APP_VERSION` environment variable). `extra="ignore"` means unrecognized keys in `.env` are silently ignored rather than raising a validation error, which is a deliberate tolerance choice — it lets the same `.env` file carry commented-out or future keys without breaking startup.

5.2 Configuration Fields

Field / env var	Default	Purpose
<code>app_name</code> / <code>APP_NAME</code>	"PRSV Research Diagnostic System"	Shown in the FastAPI title, health check, and UI.
<code>app_version</code> / <code>APP_VERSION</code>	"3.2.0"	Shown in the FastAPI version and health check; bumped by hand with every release.
<code>debug</code> / <code>DEBUG</code>	False	Controls whether FastAPI/Starlette exposes stack traces to callers.

Field / env var	Default	Purpose
<code>enable_debug_visuals</code> / <code>ENABLE_DEBUG_VISUALS</code>	<code>True</code>	Toggle for saving intermediate debug images (see <code>image_processing/visual_debug.py</code>).

5.3 Derived Properties

Beyond the declared fields, `Settings` exposes a large set of `@property` accessors that derive filesystem paths from `ROOT_DIR` rather than storing them as separate configurable fields. This is a deliberate design choice: it guarantees the directory layout is always internally consistent (e.g. `models_dir` can never accidentally point somewhere different from where `model_path` looks for the SVM file), at the cost of these paths not being independently overridable via environment variables. The full list: `image_size` (tuple derived from width/height), `data_dir`, `upload_dir`, `extracted_dir`, `processed_dir`, `output_dir`, `temp_dir`, `log_dir`, `models_dir`, `reports_dir`, `kb_path`, `model_path`, `scaler_path`, `label_encoder_path`, `model_metadata_path`, `shap_background_path`, `ensemble_model_path`, `calibrated_model_path`, `allowed_extensions` (`[".jpg", ".jpeg", ".png", ".bmp", ".webp"]`), `severity_thresholds` and `severity_weights`.

`severity_thresholds` is the single source of truth for mapping a numeric severity percentage to a human label, used by `image_processing/severity.py` and the result templates:

```

1 {
2     "Healthy": (0.0, 0.0),
3     "Very mild": (0.1, 5.0),
4     "Mild to moderate": (5.1, 25.0),
5     "Moderate": (25.1, 50.0),
6     "Moderate to severe": (50.1, 75.0),
7     "Severe": (75.1, 100.0),
8 }
```

`severity_weights` is the fixed weighting used by the original (v1/v2-era) severity formula in `image_processing/severity.py` (Chapter 13): `inverse_green_ratio` 0.30, `edge_density` 0.20, `entropy` 0.15, `abnormal_color_score` 0.20, `symptom_region_ratio` 0.15 — five weighted components that sum to 1.0.

Chapter 6

Security Middleware: `app/middleware.py`

6.1 `ApiKeyAndRateLimitMiddleware`

The project has exactly one piece of HTTP middleware, and its class docstring states its scope precisely: “Two lightweight, dependency-free protections for `/api/*` routes.” It is a subclass of Starlette’s `BaseHTTPMiddleware`, registered once in `app/main.py` via `app.add_middleware(ApiKeyAndRateLimitMiddleware, settings=settings)`.

Scope guard

The very first line of dispatch is:

```
1 if not request.url.path.startswith("/api/"):
2     return await call_next(request)
```

This means every server-rendered HTML page (`/`, `/analyze`, `/run/{run_id}`, etc.) bypasses both protections entirely and is always publicly reachable with no key and no rate limit. Only paths under `/api/` are protected.

Protection 1: optional API key

```
1 if self.settings.api_key:
2     provided_key = request.headers.get("x-api-key", "")
3     if provided_key != self.settings.api_key:
4         return JSONResponse(status_code=401, content={"detail": "Missing or invalid API key."})
```

If `settings.api_key` is the default blank string, this whole block is skipped (falsy), so by default every `/api/` endpoint is open – appropriate for local/research use, and explicitly called out in the class docstring as the intended default. Setting `API_KEY` in `.env` turns on a single shared-secret check via the `X-API-Key` request header. This is a simple constant-string comparison, not a constant-time comparison (e.g. `hmac.compare_digest`) — see Chapter 27 for the security implications of this.

Protection 2: sliding-window rate limiting

Implemented with a plain Python dict of `collections.deque`, keyed by client IP, guarded by a single `threading.Lock`:

```
1 def _check_rate_limit(self, client_ip: str) -> bool:
2     now = time.time()
3     window = self.settings.rate_limit_window_seconds
```

```

4     limit = self.settings.rate_limit_requests
5     with self._lock:
6         log = self._request_log[client_ip]
7         while log and now - log[0] > window:
8             log.popleft()
9         if len(log) >= limit:
10             return False
11         log.append(now)
12         return True

```

For each request, timestamps older than `window` seconds are popped off the left of that IP’s deque, then the remaining count is compared against `limit` (default 60 requests per 60-second window). If at capacity, the request is rejected with HTTP 429 and a message stating the configured limit and window; otherwise the current timestamp is appended and the request proceeds. The client IP is resolved by `_client_ip`, which prefers the first entry of an `X-Forwarded-For` header (for deployments behind a reverse proxy such as Render) and falls back to `request.client.host`.

This limiter is explicitly documented in its own class docstring as *not* distributed: state lives only in this one process’s memory, so it resets on every restart and does not coordinate across multiple worker processes if the app is ever run with `uvicorn -workers > 1`. This is called out directly in the module rather than left implicit.

Chapter 7

JSON API Routes

Three routers make up the JSON API, each mounted with its own prefix in `app/main.py`. A fourth router (`pages_router`) serves HTML and is documented in Chapter 26.

7.1 `app/routes/api_health.py` — `/api/health`

A single `GET /api/health` endpoint, returning a `HealthStatus` Pydantic model (Section 8). It performs three live checks on every call rather than returning a cached/static value: `DatasetService(settings).demo` (does the demo image folder exist on disk), `load_model_artifacts(settings).model_available` (can the SVM/scaler/label-encoder joblib files be loaded), and `settings.kb_path.exists()` (is the RAG knowledge base JSON file present). This makes `/api/health` a genuine readiness probe rather than a liveness-only ping — a deployment platform (or a human) can use it to detect “the process is up but the model failed to load” as distinct from “everything is fine”.

7.2 `app/routes/api_analysis.py` — `/api/analysis`

This is the largest and most important router; it owns the entire analyze-an-image surface. Every route is listed below with its HTTP method, path, and what it does.

Method	Path	Behavior
POST	<code>/api/analysis/single</code>	Accepts one uploaded file; validates extension, non-emptiness, and readability; routes through <code>BatchService.analyze_images([path])</code> even for a single image (explained below); returns the first <code>ImageResult</code> .
POST	<code>/api/analysis/multiple</code>	Accepts a list of uploaded files; <code>_collect_valid_uploads</code> filters to allowed extensions and readable, non-empty images, raising 400 if none survive; returns a full <code>BatchResult</code> .

Method	Path	Behavior
POST	/api/analysis/multiple-async	Same input validation as /multiple, but submits the batch to <code>BatchJobQueue.get_instance(settings)</code> and returns immediately with <code>{job_id, status: "queued", total_images}</code> instead of blocking.
POST	/api/analysis/zip	Accepts one .zip file; validates the extension is literally .zip; saves it, then extracts it via the security-hardened <code>safe_extract_zip</code> (Chapter 6 covers the zip-slip protection); filters extracted files to valid, readable images; returns a <code>BatchResult</code> .
POST	/api/analysis/zip-async	Same as /zip but queued, mirroring /multiple-async.
POST	/api/analysis/demo	Query param <code>limit</code> (1–200, default 10); pulls that many images from the bundled demo dataset via <code>DatasetService.get_demo_sample(limit)</code> ; 404s if the dataset path or any images are missing; returns a <code>BatchResult</code> .
POST	/api/analysis/demo-async	Queued version of /demo.
GET	/api/analysis/job/{job_id}	Validates <code>job_id</code> against the regex <code>^job_[0-9a-f]{12}\$</code> , then returns the job’s status dict from <code>BatchJobQueue</code> ; 404 if unknown.
GET	/api/analysis/runs	Scans <code>settings.output_dir</code> directly on the filesystem (<code>iterdir()</code>), for every <code>run_*</code> directory reads <code>batch_summary.json</code> if present, and returns a lightweight summary list. This is the filesystem-scan variant, distinct from...
GET	/api/analysis/runs/search	...this SQLite-backed variant, which delegates to <code>RunStore.list_runs(query, page, page_size, sort_by, sort_dir)</code> . The route’s own docstring explains the split: “Faster than /runs (which scans the filesystem) once there are many runs, and supports search - this is what the Runs/Batch pages’ frontend uses.”
GET	/api/analysis/run/{run_id}	Returns the raw <code>batch_summary.json</code> contents for one run, after <code>run_id</code> is validated by <code>_resolve_run_dir</code> (below).
GET	/api/analysis/run/{run_id}/image/{image_id}	Returns a combined payload of that image’s persisted JSON files (<code>features</code> , <code>prediction</code> , <code>severity</code> , <code>rag</code> , <code>explanation</code> – whichever exist) plus a <code>paths</code> dict mapping every file actually present in that image’s output directory to its public <code>/outputs/...</code> URL.

Method	Path	Behavior
GET	/api/analysis/run/{run_id}/download	Sends a ZIP bundle of the entire run directory, built on demand by <code>ExportService.create_run_bundle(run_dir)</code> .
GET	.../ image/{image_id}/report.pdf	Generates (or regenerates) a per-image PDF report via <code>generate_image_report_pdf</code> .
GET	.../run/{run_id}/report.pdf	Generates a whole-run PDF report via <code>generate_run_report_pdf</code> ; 404s if <code>batch_summary.json</code> does not exist yet (i.e. the run never completed).

7.2.1 Path-Parameter Validation: Preventing Path Traversal

Every route above that takes `run_id` or `image_id` as a path parameter routes through `_resolve_run_dir` and, where relevant, `_resolve_image_dir`, rather than concatenating the raw string directly into a filesystem path. Two module-level compiled regexes define the only shapes these IDs are allowed to take:

```

1 _RUN_ID_PATTERN = re.compile(r"^run_\d{4}(?:_\d{2}){5}_[0-9a-f]{6}$")
2 _IMAGE_ID_PATTERN = re.compile(r"^img_[0-9a-f]{6,16}$")

```

matching exactly what `app/utils/id_utils.py`'s `generate_run_id/generate_image_id` actually produce (e.g. `run_2026_07_02_14_30_05_a1b2c3`). `_resolve_run_dir` first rejects any `run_id` that doesn't match the pattern with HTTP 400, *then* resolves the path and calls `is_within_directory(settings.output_dir)(app/utils/path_utils.py)` as a second, independent check that the resolved absolute path is still inside `output_dir`. This belt-and-braces approach — a strict allow-list regex *and* a resolved-path containment check — means a request like `/api/analysis/run/../../../../etc/passwd` is rejected twice over: the regex fails first (the string doesn't match the run-id shape at all), and even if some future change relaxed the regex, the containment check would still catch a `..` escape.

7.3 `app/routes/api_chatbot.py` — `/api/chatbot`

Three routes, deliberately small:

- GET `/api/chatbot/languages` — returns `{"languages": list_supported_languages()}` from `app/services/translation_service.py`.
- POST `/api/chatbot/message` — body is a small local Pydantic model `ChatMessageRequest{text: str, language: str = "en"}` defined directly in the route file (not in `app/schemas.py` — the only request schema in the project that lives outside that central file). Rejects messages over 2000 characters with HTTP 400, then calls `ChatbotService.handle_message(text, language)`.
- POST `/api/chatbot/analyze-image` — accepts an uploaded image plus a `language` query/form parameter, validates it exactly like the analysis routes (extension, non-empty, readable), then calls `ChatbotService.handle_photo(image_path, language)` — this is the “photo-in-chat diagnosis” feature, reusing the same underlying image pipeline as the main analyze flow rather than a separate one.

The module keeps a single lazily-constructed module-level `_chatbot_service` instance (via `_get_chatbot_service` in contrast to `api_analysis.py`'s `build_services()`, which constructs fresh `RunManager/AnalysisService/BatchService` instances on every call. This asymmetry means the chatbot's retrievers and caches (which are

expensive to construct, per Chapter 20) are built once per process and reused, while the analysis services (cheap to construct; they hold no heavy in-memory state themselves) are simply rebuilt per request for code simplicity.

Chapter 8

Data Contracts: `app/schemas.py`

Every JSON shape that crosses the HTTP boundary in `api_analysis.py` and `api_health.py` is one of six Pydantic models defined in this single 69-line file (`api_chatbot.py` is the one exception, defining its own tiny `ChatMessageRequest` locally, as noted in Chapter 7).

`HealthStatus` `status`, `app_name`, `app_version`, `demo_dataset_available`, `model_available`, `kb_available` — all plain scalar fields, returned by `GET /api/health`. Uses `model_config = ConfigDict(protected_namespaces=)` to silence Pydantic’s default warning about fields starting with `model_` (here, `model_available`), which Pydantic v2 otherwise treats as reserved namespace-like names.

`RetrievedEvidence` `chunk_id`, `title`, `text`, `similarity_score` — one retrieved knowledge-base chunk, as surfaced to the API consumer.

`ExplanationTrace` `observation_query`, `key_findings` (list of strings), `retrieved_evidence` (list of `RetrievedEvidence`), `technical_explanation`, `farmer_friendly_explanation`, `advisory_notes` (list of strings) — the full RAG output for one image, built by `RagService.build_explanation_trace` (Chapter 20).

`ImageResult` The single largest and most important schema: `image_id`, `filename`, `prediction`, `confidence`, `infection_percentage`, `severity_score`, `severity_label`, `feature_values` (`Dict[str, float]`), `output_paths` (`Dict[str, str]`), `explanation_trace` (nested `ExplanationTrace`), and three `Optional` fields added across the v3.0 upgrade: `shap_contributions`, `lime_contributions`, and `lesion_ratio_severity`. The in-code comment on these three explains why they are all `Optional` rather than required: “all `Optional` so existing serialized runs / API consumers that don’t know about these fields keep working unchanged” — i.e. this is a deliberately backward-compatible schema evolution strategy. `lesion_ratio_severity` is typed `Optional[Dict[str, Any]]`, not `Dict[str, float]`, because of the pydantic validation bug documented in Chapter 13 (the dict it holds contains a string field, `interval_method`, alongside numeric ones).

`BatchResult` `run_id`, `total_images`, `processed_images`, `failed_images`, `healthy_count`, `diseased_count`, `average_confidence`, `average_infection_percentage`, `severity_distribution` (`Dict[str, int]`), `results` (list of `ImageResult`), `failures` (list of raw dicts describing any image that failed to process, e.g. an unreadable file that slipped past the initial validation).

`ErrorResponse` `detail` (string) and an optional `context` dict — a generic error shape available for handlers that want to attach structured context beyond FastAPI’s default `{"detail": "..."} shape` (most routes in this project actually just raise `HTTPException(status_code=..., detail=...)` directly and rely on FastAPI’s default exception handler, so `ErrorResponse` is present as a documented contract more than it is exercised by every route — *not verifiable from the available project files which specific handlers, if any, construct `ErrorResponse` explicitly*

rather than relying on FastAPI's own `HTTPException` serialization).

Chapter 9

The Core Analysis Pipeline: `app/services/analysis_service.py`

9.1 Why This File Is the Heart of the System

`AnalysisService.analyze_single_image` is the one function that every image — whether submitted through `/api/analysis/single`, `/multiple`, `/zip`, `/demo`, their async variants, or the chatbot's `/api/chatbot/analyze-image` — ultimately passes through exactly once. There is no second code path that classifies an image; every entry point converges here. Understanding this one method (264 lines) is equivalent to understanding the entire diagnostic pipeline.

9.2 Step-by-Step Trace

The method takes an `image_path: Path` and an optional `run_context`. If no run context is supplied, it creates one via `self.run_manager.create_run()` (Section 10.1) — this is what lets `/single` call this method with a fresh one-image run, while `BatchService` calls it repeatedly with a shared `run_context` so all images in a batch land in the same run folder.

1. **Logging and ID generation.** A per-run logger is fetched via `get_logger("analysis_service", run_context.log_file)`, writing to that run's own log file on disk. `generate_image_id()` (`app/utils/id_utils.py`) produces a short random hex ID (`img_a1b2c3d4e5`), and `run_manager.create_image_image_id` creates `data/outputs/run_.../images/img_.../` on disk.
2. **Preserve the original.** `copy_file(image_path, image_output_dir, filename="original" + suffix)` copies the as-uploaded file into the run folder unmodified, before any processing, so the original image is always retrievable later regardless of what preprocessing does to it.
3. **Read and assess quality.** `read_image_cv(image_path)` loads the file as an OpenCV BGR array; `assess_image_quality` (`image_processing/quality_checks.py`) computes quality metrics (blur, brightness, size) and a `quality_status` independent of everything downstream — this runs *before* preprocessing so it reflects the image as actually uploaded.
4. **Preprocess.** `preprocess_image(image_bgr, settings)` (`image_processing/preprocess.py`, Chapter 12) resizes to `settings.image_size` (128×128 by default), optionally applies CLAHE and denoising per `settings.enable_clahe/enable_denoising`, and returns an `enhanced_rgb` array, a `grayscale` array, an `hsv` array, and metadata describing what was applied.
5. **Segment the leaf.** `segment_leaf(preprocess_result .enhanced_rgb)` (`image_processing/segmentation.py`) runs HSV color thresholding plus morphological open/close plus largest-contour extraction to isolate the leaf from its background, returning a binary `mask`, a `segmented_rgb` visualization, a `contour_visualization_rgb`, `leaf_area_ratio`, `contour_count`, `used_fallback` (true

- if no leaf-shaped contour was found and a fallback mask was used instead), and `success.assess_segmentation_quality(mask)` (`image_processing/segmentation_quality.py`) then independently scores how good that mask looks.
6. **Enhance/detect symptoms.** `enhance_symptoms(image_rgb, leaf_mask)` (`image_processing/symptom_enhance.py`) runs Canny edge detection, Sobel-based gradient magnitude, and a Laplacian response, combines them into a `symptom_mask`, and computes `abnormal_color_score`. This is the step that actually looks for *disease-like* texture/color, as distinct from segmentation (which only separates leaf from background).
 7. **Extract the feature vector.** `extract_handcrafted_features` (`image_processing/feature_extraction.py`, Chapter 12.5) consumes the enhanced RGB image, grayscale, HSV, the symptom edge map, and the leaf mask, and returns a `feature_vector` (an ordered list of floats, the exact input shape the trained model expects) and a `feature_dict` (the same values, named, for display/J-SON).
 8. **Classify.** `predict_with_svm(feature_vector, settings)` (`ml/infer_svm.py`, Chapter 16) loads the persisted `svm_model.joblib + scaler.joblib + label_encoder.joblib` (via `ml/model_loader.py`'s caching loader) and returns `prediction` ("Healthy" or "Diseased"), `confidence`, `class_probabilities`, `inference_mode` ("trained_model" or "heuristic_fallback"), and the exact `feature_vector_used`.
 9. **Explain with SHAP.** `explain_prediction(feature_vector, prediction, settings)` (`ml/shap_explainer.py`) returns a `Dict[str, float]` of per-feature SHAP contribution values, or `None` on failure — explanation generation is never allowed to fail the whole request.
 10. **Explain with LIME (v3.0).** `explain_with_lime` mirrors the SHAP call as a second, independent local explanation using a different algorithm family (perturbation-based, not game-theoretic), again returning `None` rather than raising if the optional `lime` package or the SHAP background reference is unavailable.
 11. **Estimate severity (original formula).** `estimate_severity(feature_dict, symptom_mask, leaf_mask, settings, prediction)` (`image_processing/severity.py`, Chapter 13) applies the fixed weighted formula from `settings.severity_weights`, producing `infection_percentage`, `severity_score`, `severity_label`, `severity_confidence`, `symptom_region_ratio`, and a `reasoning_trace` list explaining the computation in words. Critically, this function takes `prediction` as an input specifically so it can suppress severity to zero/"Healthy" whenever the model said "Healthy" — this is the mechanism behind the README's claim that the UI never shows a contradictory "Healthy – Mild severity" state.
 12. **Estimate severity (lesion-ratio, v3.0).** `estimate_lesion_ratio_severity(symptom_mask, leaf_mask)` (`image_processing/severity_v2.py`) computes a second, independent severity signal directly from the lesion-to-leaf area ratio, with a bootstrap-based uncertainty interval, returned as a dataclass whose `__dict__` is serialized directly into both the persisted JSON and the `ImageResult.lesion_ratio_severity` field.
 13. **Generate heatmaps.** `generate_heatmaps(...)` (`image_processing/heatmaps.py`) turns the edge map, gradient magnitude, Laplacian response, symptom mask, and leaf mask into four visual artifacts: an edge heatmap (BGR, for direct OpenCV save), an edge overlay (RGB, blended onto the original leaf), a severity heatmap (BGR), and a severity overlay (RGB).
 14. **Build the RAG explanation.** `self.rag_service.build_explanation_trace(prediction, confidence, severity_label, severity_score, feature_values, symptom_findings, segmentation_success)` (Chapter 20) is called with a `symptom_findings` dict built from `abnormal_color_score` and `symptom_region_ratio` — every downstream RAG query is grounded in these actual computed values, not a generic template independent of the specific image.

15. **Persist everything to disk.** Nine artifacts are written into the image’s output directory: five images (`preprocessed.jpg`, `segmented.jpg`, `segmentation_contour.jpg`, `symptom_highlighted.jpg`, `leaf_mask.png`) plus four heatmap/overlay images (`edge_heatmap.jpg`, `edge_overlay.jpg`, `severity_heatmap.jpg`, `severity_overlay.jpg`), and five JSON files (`features.json`, `prediction.json`, `severity.json`, `rag.json`, `explanation.json`). Each JSON file’s contents are shown in full in Section 9.3.
16. **Log a one-line summary** of the whole run (prediction, confidence, severity, quality, segmentation quality) to the run’s log file, then construct and return the `ImageResult` that becomes the route’s HTTP response body.

9.3 Persisted JSON Files Per Image

The five JSON files written for every analyzed image are not simply a serialization of the `ImageResult` response — they carry *more* detail than what goes back over HTTP, because they are also the system’s durable audit trail. This is why `GET /api/analysis/run/{run_id}/image/{image_id}` (Chapter 7) reads these five files back off disk rather than recomputing anything.

`features.json` `image_id`, `filename`, `feature_vector`, `feature_values`, `diagnostics` (from feature extraction), `preprocess_metadata`, `quality_assessment` (`status/warnings/metrics/is_small_image`), and `segmentation` (`area ratio`, `contour count`, `fallback flag`, `success`, plus the independent segmentation-quality `status/score/warnings/metrics`).

`prediction.json` `image_id`, `filename`, `prediction`, `confidence`, `probabilities`, `inference_mode`, `feature_vector_used`, `shap_contributions`, `lime_contributions`.

`severity.json` `image_id`, `infection_percentage`, `severity_score`, `severity_label`, `severity_confidence`, `symptom_region_ratio`, `reasoning_trace`, and `lesion_ratio_severity` (the full lesion-ratio dataclass dump, including its bootstrap interval and `interval_method` string).

`rag.json` `observation_query`, `retrieved_evidence` (each evidence item’s `model_dump()`), `technical_explanation`, `farmer_friendly_explanation`, `advisory_notes`, and a fixed `status`: `"active_local_rag"` marker.

`explanation.json` The full `ExplanationTrace` model dump — effectively a superset/restatement of `rag.json` in the exact schema shape used by the API response’s `explanation_trace` field.

This documents a real, minor duplication in the system: `rag.json` and `explanation.json` carry overlapping information (both derive from the same `explanation_trace` object) rather than one being computed from the other on read. *The specific historical reason two separate files were chosen instead of one is not verifiable from the available project files*; functionally, `explanation.json` matches the API’s `ExplanationTrace` schema exactly, while `rag.json` adds the `status` marker field.

Chapter 10

Run Records, Batch Orchestration, and Background Jobs

10.1 `app/services/run_manager.py` — `RunManager` and `RunContext`

`RunContext` is a small `@dataclass` carrying six fields: `run_id`, `run_dir`, `images_dir`, `charts_dir`, `logs_dir`, `log_file`. `RunManager.create_run()` is the only place a `run_id` is generated (via `generate_run_id()` in `app/utils/id_utils.py`, producing the `run_YYYY_MM_DD_HH_MM_SS_XXXXXX` shape validated elsewhere by the regex in Chapter 7), and it eagerly creates all four subdirectories (`images/`, `charts/`, `logs/`, plus the run root itself) via `ensure_dir`, so nothing downstream needs to check for their existence. `create_image_dir(run_context, image_id)` is a one-line helper that creates and returns `.../images/{image_id}/`.

10.2 `app/services/batch_service.py` — `BatchService`

`BatchService.analyze_images(image_paths, progress_callback=None)` is the second convergence point in the system (after `AnalysisService.analyze_single_image` itself): every multi-image entry point (`/multiple`, `/zip`, `/demo`, and their async variants) calls this one method, and even `/single` routes through it (with a one-element list) so every analysis — single or batch — always produces a `batch_summary.json`, a CSV export, and chart images uniformly.

Its trace: create one shared `RunContext`; loop over every image path, calling `analysis_service.analyze_single_image(run_context)` inside a `try/except Exception`, so one corrupt or unprocessable image cannot abort the whole batch — failures accumulate in a `failures` list as `{filename, error}` dicts, and processing continues to the next image. A `progress_callback`, if supplied (only by `BatchJobQueue`, below), is invoked after every image regardless of success/failure, itself wrapped in its own `try/except` so a broken callback can never break the batch either.

After the loop, it computes `healthy_count/diseased_count` (by string-comparing `item.prediction.lower()` against `"healthy"`), `average_confidence` and `average_infection_percentage` (simple means over processed results, rounded to 4 decimal places, guarded against division by zero when `processed_images` is 0), and a `severity_distribution` histogram (a `Dict[str, int]` counting how many images fell into each severity label). It assembles the `BatchResult`, writes it to `batch_summary.json`, calls `export_service.export_batch_csv(...)` and `export_service.generate_batch_charts(...)` (Section 11.4), then **derives the run's created_at timestamp by parsing it back out of the run_id string itself** rather than calling `datetime.now()` at this point — the in-code comment explains why: this is “relevant if this ever runs from a queued background job with some processing delay”, i.e. so a run submitted at 10:00 but not actually processed until 10:05 (because

it sat in the single-worker job queue behind another batch) is still recorded with its true 10:00 creation time. Finally it calls `self.run_store.upsert_run(...)` to write/update the SQLite index row for this run.

10.3 `app/services/run_store.py` — `RunStore`

`RunStore` is the project's only use of a real database (`sqlite3`, standard library, no external DB server). Its own class docstring is explicit about its role: "This is additive, not a replacement for the JSON files under `data/outputs/<run_id>/` - those remain the source of truth for a run's full detail. This index exists purely so the Runs/Batch pages can search, filter, sort, and paginate without scanning the filesystem on every request." The database file lives at `settings.data_dir / "prsv_index.sqlite3"`.

Schema

Two tables, created idempotently via `CREATE TABLE IF NOT EXISTS` on every `RunStore` construction:

- `runs` — `run_id` (primary key), `created_at`, `total_images`, `processed_images`, `failed_images`, `healthy_count`, `diseased_count`, `average_confidence`, `average_infection_percentage`, `source` (defaults to `'batch'`), `filenames` (a single space-joined text blob, not a normalized child table — a deliberate simplicity trade-off for a field that is only ever substring-searched, never queried structurally). An index `idx_runs_created_at` speeds up the default sort order.
- `jobs` — `job_id` (primary key), `status`, `created_at`, `updated_at`, `run_id` (nullable until the job completes), `total_images`, `processed_images`, `error` (nullable).

`upsert_run`

Uses SQLite's `ON CONFLICT(run_id) DO UPDATE SET ...` clause, so calling it twice for the same `run_id` (which cannot actually happen in practice, since `run_id` embeds a timestamp down to the second plus a random suffix, but the method is written to tolerate it safely regardless) updates in place rather than erroring on the primary-key collision.

`list_runs` — `search/filter/sort/pagination`

This is what backs `GET /api/analysis/runs/search` (Chapter 7). `page/page_size` are clamped (`page_size` capped at 200) before being used in a `LIMIT ? OFFSET ?` clause. `sort_by` is checked against a hardcoded `allowed_sort_columns` set before being interpolated into the SQL string — this is the project's SQL-injection defense for this one place where a column name (which cannot be parameterized with a `?` placeholder in standard SQL) comes from user input: rather than parameterizing it (which SQLite does not support for identifiers), the code whitelists it against a fixed set of known-safe column names and silently falls back to `"created_at"` for anything else. The actual search term `query`, by contrast, *is* safely parameterized via `run_id LIKE ? OR filenames LIKE ?` with `?` placeholders.

Job tracking (`create_job/update_job/get_job`)

`update_job` builds its `SET` clause dynamically from only the fields actually passed (each optional parameter defaults to `None` and is skipped if still `None`), so a progress update (`processed_images` only) and a completion update (`status="done"`, `run_id`, `processed_images` all at once) both go through the same method without needing separate SQL statements for each combination of fields.

10.4 `app/services/job_queue.py` — `BatchJobQueue`

A process-wide singleton (`get_instance(settings)`, guarded by a class-level `threading.Lock`) wrapping a `ThreadPoolExecutor(max_workers=1)`. The class docstring gives the explicit architectural rationale for not using Celery/Redis: “this app is designed to run as a single process on one machine (a Windows laptop, per this project’s deployment target), so a full distributed task queue would add operational complexity ... without a matching benefit,” and notes the migration path stays open — “If this ever needs to scale beyond one machine, swapping this class for a Celery-backed implementation behind the same submit/status interface is a contained change.”

`submit(batch_service, image_paths)` generates a `job_YYYY...` style ID (`f"job_{uuid4().hex[:12]}"`, matching the `^job_[0-9a-f]{12}$` regex checked in `api_analysis.py`), records a “queued” row via `run_store.create_job`, then submits an inner `_run()` closure to the single-worker executor and returns the `job_id` immediately — this is precisely what makes `/api/analysis/multiple-async` etc. non-blocking. Inside `_run()`: status flips to “processing”, then `batch_service.analyze_images(image_paths, progress_callback=_progress)` runs on the worker thread, where `_progress(done, total)` updates `processed_images` in the `jobs` table after every single image so a polling client sees live progress, not just a final result. On success, status becomes “done” with the resulting `run_id` attached; on any exception, status becomes “error” with the exception message stored — the comment on the `except` clause is explicit that this error *is* surfaced to the polling client rather than swallowed, unlike most other “never crash on this” exception handlers elsewhere in the codebase. `get_status(job_id)` is a thin pass-through to `run_store.get_job(job_id)`.

Because `max_workers=1`, batches submitted through this queue process strictly one at a time, in submission order — a second async batch submitted while the first is still running will sit queued in the executor, and its `jobs` row will stay at “queued” until the worker thread picks it up.

Chapter 11

Supporting Services: Cache, Cleanup, Dataset, Export

11.1 `app/services/cache_service.py` — `CacheService`

The module’s own top-of-file docstring states its purpose directly: “caches repeated chatbot/RAG queries so the same FAQ-style question asked by many farmers doesn’t re-run retrieval (and, if configured, an LLM generation call) every single time.” `make_cache_key(*parts)` joins its string arguments with “||” and returns “prsv:” + `sha256(joined)[:32]` — a fixed-length, collision-resistant key regardless of how long or unusual the input query text is.

`CacheService.__init__` attempts a Redis connection only if `settings.redis_url` is non-blank: it imports `redis`, constructs a client, and calls `client.ping()` to confirm connectivity *at construction time*, not lazily on first use — if the ping fails for any reason (package missing, server unreachable, bad URL), the whole `try` block’s exception is swallowed and `self._backend` stays “memory”. This means a misconfigured `REDIS_URL` degrades silently to the in-memory fallback rather than crashing service construction. `_InMemoryCache` is a plain dict keyed by cache key, storing `(expires_at, value)` tuples, with expiry checked lazily on `get` (an expired entry is popped and treated as a miss) rather than via a background sweep. `get_json/set_json` wrap `json.dumps/json.loads` around the raw string `get/set` methods, each of which independently tries the Redis path first (again inside a `try/except` that falls back to the memory cache on any Redis error at call time, not just at construction) before falling back to `_memory_cache`.

Used by `RagService` (Chapter 20) and `ChatbotService` (Chapter 23) to cache repeated RAG explanation traces and chatbot answers keyed on the query text and retrieval mode.

11.2 `app/services/cleanup_service.py` — `run_cleanup`

A single function, `run_cleanup(settings, retention_days=None, dry_run=False)`, shared by two callers: the manual/cron-style `scripts/cleanup_outputs.py` script and the automatic background thread started in `app/main.py`’s lifespan (Chapter 4). It parses each `run_*` directory name back into a `datetime` using `_parse_run_timestamp` (matching the exact `run_YYYY_MM_DD_HH_MM_SS_XXXXXX` format from `generate_run_id`), and for any run older than `retention_days` (default: `settings.output_retention`), deletes the entire folder via `shutil.rmtree(run_dir, ignore_errors=True)` and removes its row from the SQLite index via `run_store.delete_run(run_dir.name)`. `retention_days <= 0` short-circuits to a no-op immediately, matching the config comment that 0 disables age-based cleanup. The `dry_run` flag lets the function compute what *would* be deleted without touching the filesystem — useful for the standalone script to be run safely first with `-dry-run` before actual deletion

(the exact CLI flag name is defined in `scripts/cleanup_outputs.py` itself). The function returns a summary dict `{"deleted": N, "kept": N, "unparseable": N}`, where "unparseable" counts any `run_*` directory whose name doesn't match the expected timestamp shape (e.g. manually created or corrupted folders) — these are conservatively left alone rather than deleted.

11.3 `app/services/dataset_service.py` — `DatasetService`

The smallest service in the project (32 lines): `demo_dataset_exists()` checks that `settings.demo_dataset_path` exists and is a directory; `list_demo_images()` lists every file in that directory whose suffix (case-insensitively) is in `settings.allowed_extensions`, sorted alphabetically; `get_demo_sample(limit=10)` returns the first `limit` of that sorted list. There is no randomization or shuffling — “the demo dataset” as served by `/api/analysis/demo` is always the same deterministic, alphabetically-first `limit` images from `Original Images/` for a given `limit` value.

11.4 `app/services/export_service.py` — `ExportService`

Three independent responsibilities:

- `export_batch_csv(run_dir, batch_result)` — builds a `pandas.DataFrame` with one row per successfully processed image (`image_id`, `filename`, a *display-formatted* prediction label via `prediction_display_label` from `app/utils/display_labels.py`, `confidence`, `infection_percentage`, `severity_score`, `severity_label`), and writes it to `batch_summary.csv` in the run directory.
- `create_run_bundle(run_dir)` — calls `shutil.make_archive(base_name=run_dir/"export_bundle", format="zip", root_dir=run_dir)`, zipping the *entire* run directory (images, JSON, charts, logs, the CSV — everything) into `export_bundle.zip` inside that same run directory. This is what `GET /api/analysis/run/{run_id}/download` (Chapter 7) streams back as a `FileResponse`.
- `generate_batch_charts(run_dir, batch_result)` — uses `matplotlib` with the non-interactive "Agg" backend (set once at module import time, appropriate for a headless server process with no display) to render two PNG bar charts into the run's `charts/` subdirectory: a two-bar Healthy-vs-“PRSV Suspected” prediction distribution, and a per-severity-label distribution bar chart (with an explicit “No severity data” fallback message drawn instead of an empty chart when `severity_distribution` is empty, e.g. a batch where every image failed to process). Each bar is annotated with its exact count value directly above it via `ax.text(...)`.

Part III

The Computer Vision and Machine Learning Pipeline

Chapter 12

Image Preprocessing, Segmentation, and Symptom Detection

12.1 Shared Constants: `image_processing/constants.py`

A small module of fixed constants shared across the whole `image_processing/` package: `FEATURE_NAMES` (the ordered list of 12 feature names — the 7 original color/edge features plus 5 v2.9-added texture features, discussed below); `DEFAULT_LEAF_HSV_LOWER` = (20, 20, 20) and `DEFAULT_LEAF_HSV_UPPER` = (95, 255, 255) (the HSV threshold range used for leaf segmentation); `CANNY_THRESHOLD_1` = 50 and `CANNY_THRESHOLD_2` = 150 (Canny edge detector hysteresis thresholds); `MORPH_KERNEL_SIZE` = 5 (the square structuring element size for morphological open/close); and the GLCM/LBP texture parameters (`GLCM_DISTANCES` = [1, 2], four `GLCM_ANGLES` at 0/45/90/135 degrees, `LBP_RADIUS` = 2, `LBP_N_POINTS` = 16).

A comment directly above the texture feature names in `FEATURE_NAMES` explains their addition: “Ring-spot lesions have a distinct local texture (pitted, irregular) that global color/edge averages miss - GLCM and LBP capture that texture directly. This is a cheaper accuracy lever than a CNN.” This is a real, stated design rationale (texture over deep learning, for cost/complexity reasons) preserved verbatim from the source.

12.2 `image_processing/preprocess.py`

`preprocess_image(image_bgr, settings)` returns a `PreprocessResult` dataclass carrying *every* intermediate array produced along the way (`original_bgr`, `original_rgb`, `resized_rgb`, `resized_bgr`, `grayscale`, `hsv`, `normalized_rgb`, `denoised_rgb`, `enhanced_rgb`), not just the final output — this is a deliberate “keep everything visible” style used throughout `image_processing/`, which makes every stage independently inspectable/debuggable (and is exactly what `image_processing/visual_debug.py` relies on to save intermediate-stage debug images when `settings.enable_debug_visuals` is on).

Trace: reject an empty/`None` input immediately; resize to `settings.image_size` (128×128 by default) using `cv2.INTER_AREA` interpolation (chosen because it is the recommended OpenCV interpolation method for shrinking an image, avoiding moire artifacts that `INTER_LINEAR` can introduce when downsizing); derive grayscale and HSV from the resized RGB; normalize to a [0, 1] float array; conditionally denoise via `cv2.fastNlMeansDenoisingColored`(img, None, 10, 10, 7, 21) (Non-Local Means denoising, parameters: filter strength 10 for luminance, 10 for color, template window 7, search window 21 — OpenCV’s own commonly-used defaults for this function) gated by `settings.enable_denoising`; conditionally apply CLAHE via `apply_clahe_to_rgb` gated by `settings.enable_clahe`. `apply_clahe_to_rgb` converts to LAB color space, applies

`cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))` to the L (lightness) channel only, then converts back to RGB — applying contrast enhancement to lightness only, not to the color channels, avoids introducing color distortion while still improving local contrast (a standard CLAHE-on-LAB pattern). A `metadata` dict records the original/resized/grayscale/HSV shapes and the normalized array’s actual min/max (a lightweight built-in sanity check that normalization produced values genuinely inside `[0, 1]`).

12.3 `image_processing/segmentation.py`

`segment_leaf(image_rgb)` is the segmentation method actually used by the live pipeline (as opposed to the three classical alternatives in `segmentation_classical.py`, discussed in Section 15.4, which are implemented and tested but not wired into `AnalysisService`).

1. Convert to HSV; threshold with `cv2.inRange` against `DEFAULT_LEAF_HSV_LOWER/UPPER` — a hue/saturation/value range tuned to typical green leaf tissue.
2. Clean the resulting binary mask with morphological `MORPH_OPEN` then `MORPH_CLOSE` using a `5×5` all-ones kernel (open removes small noise specks; close fills small holes inside the leaf region).
3. Find the largest external contour (`cv2.findContours(..., cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`, then `max(contours, key=cv2.contourArea)`) and fill it to produce the final mask.
4. **Two independent fallback triggers**, both of which set `used_fallback=True`, `success=False`, and replace the mask with a full-white (whole-image) mask:
 - No contour was found at all (`largest` is `None`) — e.g. an image with no green region matching the HSV range.
 - A contour *was* found, but the resulting `leaf_area_ratio` (fraction of the image the mask covers) is below `0.08` — i.e. segmentation technically succeeded but found an implausibly tiny “leaf,” which is treated as a failure and the whole image is used instead of a near-empty mask.
5. Build the visual outputs: `segmented_rgb` (the original image with everything outside the mask zeroed via `cv2.bitwise_and`), and `contour_visualization_rgb` (the original image with the detected contour drawn in blue, `(255, 0, 0)` in RGB order, 2px thick — only drawn when a real contour was found and the fallback was *not* used, so a fallback-mode image never shows a misleading contour outline).

Every downstream stage (symptom enhancement, feature extraction, severity) receives this same `mask`, so a fallback to a full-image mask does not crash anything downstream — it simply means the “leaf region” those stages operate over is the entire image rather than a tightly cropped leaf, a graceful (if less precise) degradation rather than a pipeline failure.

12.4 `image_processing/symptom_enhancement.py`

`enhance_symptoms(image_rgb, leaf_mask)` produces the evidence maps that actually look for disease-like irregularities, all masked to the segmented leaf region so background pixels never contribute:

- **Edge map** — `cv2.Canny(gray, 50, 150)`, masked to the leaf.
- **Gradient magnitude** — Sobel derivatives in x and y (`ksize=3`), combined via `cv2.magnitude`, masked, then normalized to `uint8`.

- **Laplacian response** — `cv2.Laplacian(gray, CV_32F, ksize=3)`, absolute value, masked, normalized.
- **Chlorosis map** — a from-scratch color-based heuristic (`_compute_chlorosis_map`), not a standard OpenCV function: $\text{yellow_score} = \frac{R+G}{2} - B$ and $\text{green_loss} = \max(0, R - G) + \max(0, B - G)$, combined as $0.6 \times \text{yellow_score} + 0.4 \times \text{green_loss}$. This directly encodes the visual intuition that chlorotic (yellowing) tissue is relatively yellow (high R+G relative to B) and has lost its normal green dominance — a hand-designed, explainable heuristic rather than a learned feature. Its per-pixel mean over the leaf mask becomes the scalar `abnormal_color_score`, later fed into the RAG explanation’s `symptom_findings` (Chapter 9).
- **Combined symptom mask** — a fixed-weight blend: $0.35 \times \text{gradient} + 0.30 \times \text{laplacian} + 0.35 \times \text{chlorosis}$, normalized to `uint8`. This is the mask used by both severity estimators (Chapter 13) and the heatmap generator.
- **Symptom highlight visualization** — the symptom mask is painted into the red channel of a same-shape overlay image, then blended onto the original with `cv2.addWeighted(original, 0.72, red_overlay, 0.45, 0)` and re-masked to the leaf, producing `symptom_highlighted.jpg`.

12.5 `image_processing/feature_extraction.py` and `texture_features.py`

`extract_handcrafted_features` computes the original 7 global features, each restricted to leaf-mask pixels only (`_masked_pixels`):

Feature	Definition
<code>brightness</code>	Mean grayscale value over masked pixels, normalized by 255.
<code>green_ratio</code>	Mean of $G/(R+G+B+\epsilon)$ over masked pixels ($\epsilon = 10^{-6}$ avoids division by zero on pure-black pixels).
<code>hue_mean</code>	Mean HSV hue over masked pixels, normalized by 179 (OpenCV’s HSV hue range is 0–179, not 0–359).
<code>saturation_mean</code>	Mean HSV saturation over masked pixels, normalized by 255.
<code>edge_density</code>	Fraction of masked pixels that are nonzero in the (already leaf-masked) Canny edge map.
<code>color_variance</code>	Variance of masked RGB pixel values, normalized by 255^2 .
<code>entropy</code>	<code>skimage.measure.shannon_entropy</code> of the masked grayscale pixels, normalized by 8 (the maximum possible entropy of an 8-bit image, $\log_2(256) = 8$).

These are merged with the 5 texture features from `extract_texture_features` (below) into one 12-entry `feature_dict`, and `feature_vector` is built by indexing that dict in the exact order of `FEATURE_NAMES` — this ordering is what the trained SVM’s input columns actually correspond to, so any change to `FEATURE_NAMES`’s order without retraining the model would silently scramble every downstream prediction; nothing in the code enforces this invariant beyond the shared constant being imported from the one place.

Texture features (`texture_features.py`)

`extract_texture_features(grayscale, mask)` crops to the mask’s bounding box, zeroes out-of-mask pixels within that box, requantizes to 32 gray levels (GLCM needs a small, tractable number of levels), and computes:

- **GLCM contrast/homogeneity/energy/correlation** — `skimage.feature.graycomatrix` at distances $\{1, 2\}$ and angles $\{0^\circ, 45^\circ, 90^\circ, 135^\circ\}$ (8 combinations), `symmetric=True`, `normed=True`; each property is averaged across all 8 matrices via `graycoprops`. Contrast is additionally normalized by dividing by 32^2 (levels squared) and clamped to ≤ 1.0 , since GLCM contrast is otherwise unbounded, to keep it in a comparable range to the other $\sim 0-1$ features. Correlation is clamped to $[-1, 1]$ and any `NaN` (which GLCM correlation can produce on a perfectly uniform region) is replaced with 0 via `np.nan_to_num`.
- **LBP uniformity** — `skimage.feature.local_binary_pattern` with `method="uniform"`, 16 points, radius 2; a histogram is built over the masked LBP values, and “uniformity” is defined as $\sum_i \text{hist}_i^2$ — the sum of squared bin proportions, which is higher when the histogram is concentrated in fewer bins (a more regular texture) and lower when it is spread out (a more irregular, pitted texture). The in-code comment is explicit that this irregularity is “exactly the kind ... ring-spot lesions introduce.”

Both GLCM and LBP computation are individually wrapped in `try/except Exception`, each with the comment “texture extraction must never break the pipeline” — on any failure, the corresponding feature values default to 0.0 rather than propagating an exception up through `AnalysisService`.

Chapter 13

Severity Estimation

13.1 `image_processing/severity.py` — The Original Weighted Formula

`estimate_severity(feature_dict, symptom_mask, leaf_mask, settings, prediction)` is an explicit, unsupervised (not learned) weighted sum, not a second machine-learning model. Its own docstring states the key design rule directly: the `prediction` argument (the classifier's Healthy/Diseased call) “is used to keep the severity label consistent with the diagnosis: a leaf the model calls Healthy is always reported in the ‘Healthy’ severity band, even if the raw unsupervised formula below would otherwise compute a small non-zero score from ordinary lighting/color variation. This avoids UI states like ‘Healthy - Mild severity’ that read as contradictory to the user.”

The five weighted components

$$\text{raw_score} = 0.30 \cdot \text{inv_green} + 0.20 \cdot \text{edge_density} + 0.15 \cdot \text{entropy} + 0.20 \cdot \text{abnormal_color} + 0.15 \cdot \text{symptom_ratio}$$

where $\text{inv_green} = 1 - \text{green_ratio}$ (a leaf that has lost its normal green dominance scores higher), `edge_density` and `entropy` are read directly from the already-computed `feature_dict`, `abnormal_color_score` is recomputed here as the mean symptom-mask intensity over the leaf mask (independently of the `abnormal_color_score` computed in symptom enhancement, though both derive from the same chlorosis-weighted symptom mask), and `symptom_region_ratio` is the fraction of leaf pixels where the symptom mask exceeds intensity 160. The weights (0.30/0.20/0.15/0.20/0.15, summing to 1.0) come from `settings.severity_weights` (Chapter 5) and are fixed, hand-tuned constants — *the specific historical process by which these five weight values were chosen is not verifiable from the available project files; there is no fitting/ calibration script for them in the codebase.*

The raw 0–1 score is scaled to 0–100 and clamped. If `prediction` (case-insensitively) equals “healthy”, the score is hard-forced to 0.0 and the label to “Healthy” — regardless of what the raw formula computed — which is exactly the suppression mechanism described in the docstring. Otherwise, the label is looked up via `_severity_label`, which does a linear scan of `settings.severity_thresholds` for the first band whose (*low*, *high*) range contains the score.

`severity_confidence` is a *separate* formula (not the same value as `severity_score`):

$$\text{confidence} = \text{clamp}(40 + 25 \cdot \text{edge_density} + 20 \cdot \text{symptom_ratio} + 15 \cdot \text{abnormal_color})$$

— a floor of 40 plus contributions from the same three symptom-strength signals, meant to express how much visual evidence backs the severity estimate (more edges/symptom area/abnormal color $\Rightarrow$ higher confidence in the estimate), not how large the estimate itself is.

The returned `reasoning_trace` dict is written verbatim into `severity.json` and includes a "formula_mode": "weighted_image_processing_based" tag and an explicit "suppressed_for_healthy_prediction" bool flag — so any consumer of the persisted JSON (or a developer debugging a run) can see directly, without inferring it, whether this particular result was formula-computed or forced to zero by the Healthy-suppression rule.

13.2 `image_processing/severity_v2.py` — Lesion-Ratio Severity with Bootstrap Interval

This module’s own docstring is unusually candid about its scope and is worth preserving close to verbatim, since it is the clearest first-person statement in the codebase of what was and was not achievable given the data actually available:

“Two upgrades from the wishlist, implemented honestly within what the current pipeline can actually support: 1. Lesion-area-ratio severity ... This does NOT require a trained instance-segmentation model (SAM/U-Net) - the wishlist’s fuller version of this item explicitly depends on Section 1’s lesion segmentation work landing first. What’s implemented here uses the classical HSV-threshold symptom mask the pipeline already computes. 2. Conformal prediction interval: ... this pipeline has no expert-annotated ground-truth severity labels yet ..., so `calibrate_from_residuals()` is the real, reusable conformal machinery, and `bootstrap_interval()` is what’s used by default today: a distribution-free bootstrap interval ..., which is a defensible interim proxy ..., not a substitute for genuine expert-calibrated coverage.”

`compute_lesion_area_ratio`

A single-line core computation: $\text{ratio} = \frac{|\{p : \text{symptom_mask}(p) > 160 \wedge \text{leaf_mask}(p) > 0\}|}{\max(1, |\{p : \text{leaf_mask}(p) > 0\}|)}$
— exactly “what fraction of leaf pixels are symptom pixels,” using the same threshold-160 convention as `severity.py`.

`bootstrap_interval`

Runs $n = 25$ resamples with a *fixed* random seed (`np.random.default_rng(seed=42)`, so results are deterministic and reproducible run-to-run for the same mask, not a source of run-to-run noise) where each resample perturbs the symptom-pixel threshold by a random integer jitter in $[-20, +20]$ around the base threshold of 160, recomputes the lesion ratio at that jittered threshold, and collects 25 ratio values. The interval is then the empirical $\frac{1-\text{coverage}}{2}$ and $1 - \frac{1-\text{coverage}}{2}$ quantiles of those 25 values (`coverage=0.90` by default, i.e. the 5th and 95th percentiles). This measures *threshold sensitivity* — how much the estimate would change if “what counts as a symptom pixel” were drawn slightly differently — which is a real and useful signal, but the docstring is explicit that it is **not** a calibrated conformal interval over actual ground-truth severity error.

`calibrate_from_residuals` — **built, unused**

A genuine split-conformal calibration function: given a list of $|\text{predicted} - \text{true}|$ residuals from a held-out calibration set, it computes $q_{\text{level}} = \min(1, \frac{\lceil (n+1) \cdot \text{coverage} \rceil}{n})$ and returns the q_{level} -quantile of the absolute residuals — the textbook split-conformal interval half-width formula. This function is never called anywhere else in the codebase (confirmed by inspection: no import of `calibrate_from_residuals` appears outside its own module and its test file). It exists as ready-to-use machinery for the moment expert-labeled ground-truth severity scores become available, exactly as its docstring states; until then, the system always uses `bootstrap_interval` instead.

This is documented in this project's own `ROADMAP_v3.md` as a Category A item (blocked on data, not on missing code).

Output

`estimate_lesion_ratio_severity` returns a `LesionRatioSeverity` dataclass: `lesion_area_ratio`, `severity_score` (the ratio $\times 100$), `interval_low`, `interval_high` (also scaled to a 0–100 percentage and clamped into `[0, 100]`), `interval_method` (the fixed string `"threshold_perturbation_bootstrap"`), and `coverage` (0.90). Its `__dict__` is what gets serialized into `ImageResult.lesion_ratio_severity` and `severity.json` — and, as documented in Chapter 9, is exactly why that Pydantic field had to be retyped from `Dict[str, float]` to `Dict[str, Any]` (Error 1 in the project's own v3.0 changelog): this dataclass mixes numeric fields with the string field `interval_method`.

Unlike the original `severity.py` formula, **this second severity number is never suppressed for a Healthy prediction** — it is computed unconditionally and reported alongside the (correctly suppressed) primary `severity_score/severity_label`. This is a real, observable asymmetry in the shipped system: a Healthy-predicted image's `severity_label` will always read "Healthy" with `severity_score: 0`, while its `lesion_ratio_severity.severity_score` can still be a small nonzero number reflecting incidental symptom-mask pixels. *Whether this asymmetry is an intentional design choice (the secondary metric is meant to always reflect the raw signal, unfiltered) or an oversight is not verifiable from the available project files* — no comment in either module addresses the interaction between the two.

Chapter 14

Heatmaps and Quality Assessment

14.1 `image_processing/heatmaps.py`

`generate_heatmaps(...)` produces the two pairs of visual explanation artifacts saved for every analyzed image. Both use the same two-step pattern: build a grayscale “intensity” map, colorize it with `cv2.applyColorMap(uint8_map, cv2.COLORMAP_JET)` (OpenCV’s classic blue→red “jet” colormap, the same one common in heatmap visualizations across scientific computing), then blend it onto the original leaf image with `cv2.addWeighted`.

- **Edge heatmap:** colorizes the leaf-masked Canny edge map directly (`masked_edge = cv2.bitwise_and(edge_map, edge_map, mask=leaf_mask)`), then overlays it at $\alpha = 0.35$.
- **Severity heatmap:** builds a *different* weighted combination than either the symptom mask or the severity-score formula — $0.25 \cdot \text{edge} + 0.30 \cdot \text{gradient} + 0.20 \cdot \text{laplacian} + 0.25 \cdot \text{symptom_mask}$, masked to the leaf — then overlays at $\alpha = 0.40$. This is a third distinct weighted blend of the same underlying edge/gradient/laplacian/symptom signals (alongside the symptom-mask blend in `symptom_enhancement.py` and the severity-score blend in `severity.py`), tuned independently for visual clarity as a heatmap rather than for numerical severity accuracy — these three blends use different weights from each other, and nothing in the code ties them to a single shared formula.

Both overlay images are re-masked to the leaf region after blending, so the background of `edge_overlay.jpg` and `severity_overlay.jpg` is black rather than showing a faint heatmap tint over non-leaf pixels.

14.2 `image_processing/quality_checks.py`

`assess_image_quality(image_bgr)` runs *before* any preprocessing, directly on the as-uploaded image, using three classical, well-established quality heuristics:

- **Blur** — variance of the Laplacian (`cv2.Laplacian(gray, CV_64F).var()`), a standard blur-detection heuristic (a sharp image has high-variance Laplacian response; a blurry one is smoothed out and has low variance). Threshold: below 40 triggers an “Image appears blurry” warning.
- **Brightness** — mean grayscale value. Below 55 triggers “too dark”; above 210 triggers “too bright.”
- **Contrast** — standard deviation of grayscale values. Below 25 triggers “low contrast.”

Additionally, any image narrower or shorter than 128px in either dimension is flagged `is_small_image=True` with its own warning. The overall `quality_status` is derived purely from *how many* warnings fired: zero $\rightarrow$ "good", exactly one $\rightarrow$ "acceptable", two or more $\rightarrow$ "poor" — a simple count-based rule, not a weighted score. This result flows into `features.json`'s `quality_assessment` block and (per Chapter 26) into the result page's UI warnings, but — confirmed by inspection — **does not itself block or alter the analysis pipeline in any way**; a "poor" quality image is still fully processed and classified, with the warnings surfaced only for the human viewing the result to take into account.

14.3 `image_processing/segmentation_quality.py`

`assess_segmentation_quality(mask)` is the mask-quality counterpart to the image-quality check above, run *after* segmentation. It starts a score at 100 and subtracts penalties for four independent failure modes, each checked and penalized separately (so a mask can accumulate multiple penalties):

Condition	Penalty	Warning
<code>leaf_coverage_ratio < 0.08</code>	−35	"Leaf coverage is very low in the segmented mask."
<code>0.08 ≤ leaf_coverage_ratio < 0.18</code>	−15	"Leaf coverage is limited in the segmented mask."
<code>contour_count > 6</code>	−20	"Segmentation mask is fragmented into many regions."
<code>3 < contour_count ≤ 6</code>	−10	"Segmentation mask shows moderate fragmentation."
<code>largest_contour_ratio < 0.06</code>	−25	"Largest segmented region is too small."
<code>0.06 ≤ largest_contour_ratio < 0.15</code>	−10	"Largest segmented region is smaller than expected."
<code>leaf_coverage_ratio > 0.85</code>	−15	"Segmented region occupies most of the image and may include background."

The final score is clamped to $[0, 100]$ and bucketed: $\geq 75 \rightarrow$ "good", $\geq 50 \rightarrow$ "acceptable", otherwise "weak". Note that the 0.08 leaf-coverage threshold here duplicates the 0.08 fallback threshold in `segment_leaf` itself (Chapter 12) — both were chosen independently at the same value, but this function does not call into or share code with the segmentation fallback logic; a mask that already triggered the segmentation module's own fallback (and is therefore a full-image mask with `leaf_coverage_ratio` exactly 1.0) will instead trip the *opposite* penalty here (> 0.85 , "occupies most of the image"), which is the correct and consistent outcome even though the two modules never directly reference one another.

14.4 `image_processing/visual_debug.py` — An Unwired Utility Module

This 35-line module defines four thin wrapper functions (`save_debug_rgb`, `save_debug_gray`, `save_debug_mask`, `save_debug_bgr`) around the shared image-save helpers in `app/utils/image_utils.py`. A direct search of the codebase confirms it is **never imported by any other module in the project**, including `app/services/analysis_service.py` (which would be the natural caller, since it is the only place that saves per-stage debug images today, and does so by calling `save_image_pil/save_image_cv` directly rather than through these wrappers) and **never referenced by any test file**. Relatedly, `settings.enable_debug_visuals` (Chapter 5) is likewise declared in `app/config.py` but — confirmed by a codebase-wide search for the string

`enable_debug_visuals` — is never read anywhere outside its own field definition. Put plainly: this is a real, present piece of scaffolding for a per-stage debug-image toggle that was set up (a config flag plus a small save-helper module) but never wired into `AnalysisService`'s actual save logic, which today unconditionally saves its fixed set of nine output images regardless of this flag's value. This is stated here as an observed fact about the shipped code, not as a criticism to be resolved — *whether this was an intentional partial implementation left for future use, or simply superseded by `AnalysisService` always saving everything anyway, is not verifiable from the available project files.*

Chapter 15

Extended (Opt-In, Not Live-Wired) Image Processing Modules

Four v3.1/v3.2 modules in `image_processing/` share a common, explicitly-stated architectural pattern: they are real, fully implemented, individually tested computer-vision code, but are **deliberately not called by** `AnalysisService` and therefore never run during a normal `/api/analysis/*` request. Each module's own top-of-file docstring states this opt-in status directly, and the reason is the same in every case: **the shipped, trained SVM model was fit on exactly the original 12-feature vector** (`ml/feature_schema.py`), so silently changing what features get computed by default would desynchronize the live pipeline from the model it was trained against. These modules exist for exploratory analysis, a future retraining run with a richer feature set, or as alternative strategies a caller could opt into explicitly — not as silent replacements of the production path.

15.1 `image_processing/texture_extended.py`

Docstring: “Extended texture descriptors (v3.1), additive to the existing single-radius GLCM/LBP features ... Returned as a separate, opt-in feature dict rather than folded into the production 12-feature vector ... these are available for a future retraining run that wants a richer feature set, and for exploratory analysis via `ml/feature_quality.py`.”

- `multi_radius_lbp_histogram(grayscale, radii=(1,2,3), n_points_per_radius=8)` — computes uniform LBP at three radii (8, 16, 24 points respectively) and concatenates each radius's histogram into a flat dict keyed `lbp_r{radius}_bin{i}`, capturing texture at multiple spatial scales rather than the single `radius=2` used in the production `texture_features.py`.
- `gabor_filter_bank_energy(grayscale, orientations=(0,45,90,135), frequencies=(0.1,0.3))` — applies `skimage.filters.gabor` at 4 orientations $\times$ 2 frequencies (8 filters total) and records the mean squared response energy of each as `gabor_o{deg}_f{freq}_energy`.
- `tamura_features(grayscale)` — the classic Tamura coarseness/contrast/directionality triad (a well-known texture descriptor family from 1978 perceptual-texture research), implemented via multi-scale average difference for coarseness, statistical moments for contrast, and a gradient-orientation-histogram sharpness measure for directionality.
- `wavelet_texture_energy(grayscale, wavelet="db2", level=2)` — Daubechies-2 wavelet sub-band energy via the optional `PyWavelets` package, returning `Optional[Dict[str, float]]` (None if `pywt` is unavailable, though `PyWavelets` is in fact listed in the base `requirements.txt`, so in practice this key is populated).

- `local_phase_quantization(grayscale, window_size=7)` — a from-scratch LPQ implementation, motivated in-code by the fact that “LPQ isn’t in scikit-image”; LPQ is a texture descriptor specifically designed to be robust to uniform (out-of-focus) blur, complementing the blur-sensitive standard LBP/GLCM features.
- `extract_all_extended_texture_features(grayscale)` — aggregates all of the above into one combined dict.

15.2 `image_processing/color_spaces_extended.py`

Same opt-in framing (“Same opt-in status as `texture_extended.py`: not folded into the production 12-feature vector”). Functions: `lab_color_histogram` (16-bin histograms per CIE-LAB channel), `ycbcr_color_histogram` (16-bin histograms per YCbCr channel), `hsi_features` (Hue-Saturation-Intensity color-space statistics, a color model distinct from OpenCV’s HSV), `vegetation_indices` (ExG/ExGR/GLI — Excess Green, Excess Green minus Excess Red, and Green Leaf Index, all standard vegetation-index formulas from remote-sensing/ precision-agriculture literature, computed directly from RGB), and `color_moments` (per-channel mean/variance/skewness — the first three statistical moments, a classic compact color descriptor). `extract_all_extended_color_features` aggregates all of these.

15.3 `image_processing/illumination_normalization.py`

Docstring: “preprocessing options that separate illumination from reflectance before feature extraction, addressing cross-device/lighting variance. Offered as opt-in alternatives alongside the existing CLAHE-based enhancement ..., not a replacement for it (CLAHE stays the default the shipped model was trained against).” Functions: `single_scale_retinex(image_rgb, sigma=80.0)` — the classic Retinex decomposition, $\text{reflectance} = \log(\text{image}) - \log(\text{gaussian_blur}(\text{image}))$, separating the illumination (low-frequency blur) from reflectance (the true surface color, independent of lighting); `multi_scale_retinex(image_rgb, sigmas=(15,80,200))` — averages single-scale Retinex at three blur scales for a more robust result; `detect_shadow_mask(image_rgb, threshold_ratio=0.6)` — a chromaticity-based heuristic shadow detector; `gray_world_white_balance(image_rgb)` — the classic gray-world assumption (average scene reflectance is gray) used to correct color cast.

15.4 `image_processing/segmentation_classical.py`

Docstring: “Classical segmentation alternatives (v3.1) to the existing HSV-threshold leaf segmentation ... still not a trained model (SAM/U-Net remain blocked on labeled data, see `ROADMAP_v3.md`), but more background-robust than a fixed HSV threshold under shadows, clutter, or uneven lighting. Offered as alternative masking strategies, selectable per image, not a silent replacement of the production path.” Each function returns an `AlternativeSegmentationResult(mask, method, leaf_area_ratio)`:

- `grabcut_segmentation(image_rgb, iterations=5)` — OpenCV’s GrabCut (`cv2.grabCut`) seeded with a rectangle covering the central ~80% of the image, refined over 5 iterations into foreground/background/probable regions.
- `watershed_segmentation(image_rgb)` — marker-based watershed transform.
- `slic_superpixel_mask(image_rgb, n_segments=100, compactness=10.0)` — SLIC (Simple Linear Iterative Clustering) superpixel oversegmentation via `skimage.segmentation.slic` (imported at call time inside the function).

As reported in this documentation’s Executive Summary and in the project’s own build history, these three were verified against a real dataset image (`Original Images/RingSpot(160).jpg`) during development, producing leaf-area ratios of 0.0028 (GrabCut), 0.073 (Watershed), and 0.782 (SLIC) on that specific image — a wide spread illustrating that these three classical methods disagree substantially with each other (and, by implication, may each disagree with the production HSV-threshold method) on what fraction of a given image actually constitutes “leaf,” which is itself informative context for anyone considering promoting one of them into the live pipeline.

15.5 `image_processing/weather_fusion.py` — v3.2

Unlike the four modules above, weather fusion is not opt-in-but-unused — it is simply not on the critical analysis path because it needs an external location input the core leaf-analysis pipeline never collects. Its docstring is the most extensively self-documented “honesty note” in the entire codebase and is worth quoting in full:

“Honesty note on what’s genuinely proven vs. still a research question: the API call and data-fetching code below is real and makes an actual HTTP request to a live weather service (verify by running `fetch_current_weather()` from an environment with outbound internet access - this sandbox’s egress allowlist doesn’t include `api.open-meteo.com`, so the call itself is untested from within this container, but the request/response handling follows Open-Meteo’s documented, stable API contract exactly). What remains a genuine open research question, explicitly not resolved here, is the *risk model*: the coefficients in `estimate_spread_risk()` are illustrative placeholders based on the general agronomy principle that aphid activity (PRSV’s vector) rises with warm, humid conditions - they are NOT calibrated against real PRSV outbreak data, because no such calibration dataset exists in this project. Treat the output as a demonstration of the *pipeline*, not a validated forecast.”

`fetch_current_weather(latitude, longitude, timeout_seconds=10)` calls GET `https://api.open-meteo.com/v1` with `current=temperature_2m,relative_humidity_2m,precipitation, wind_speed_10m`, parses the response into a `WeatherSnapshot` dataclass, and returns `None` on *any* exception (network, timeout, malformed JSON, missing key) — the comment explicitly places this in the same “best-effort, never fatal” family as SHAP, LIME, and translation.

`estimate_spread_risk(weather)` computes four independent factors — a temperature factor peaking at 28°C ($\max(0, 1 - |T - 28|/15)$), a humidity factor saturating at 80% relative humidity, a rain-penalty term (heavy rain washes aphids off plants), and a wind-penalty term (wind disrupts aphid flight) — and combines them as $\text{risk} = T_{\text{factor}} \cdot H_{\text{factor}} \cdot (1 - 0.6 R_{\text{penalty}}) \cdot (1 - 0.3 W_{\text{penalty}})$, clamped to $[0, 1]$ and bucketed into “low” (< 0.3), “moderate” (< 0.6), or “elevated”. A human-readable `reasoning` string is generated describing which factors drove the result, always ending with the same fixed disclaimer sentence: “This is an illustrative heuristic, not a validated epidemiological model.” — baked into the function’s own output, not just its docstring, so the caveat travels with the result even if the caller never reads the source code. `get_spread_risk_for_location(latitude, longitude)` chains the two functions and returns `None` end-to-end if the weather fetch failed. Confirmed by codebase search: this module is not imported by any route or service in the live application — it is exercised only by its own test file, `tests/test_weather_fusion.py`, which mocks `requests.get` rather than hitting the live API (as noted in the Errors/troubleshooting history behind this project’s development).

Chapter 16

Classification: Model Loading and Inference

16.1 `ml/feature_schema.py`

Three lines of real substance: `EXPECTED_FEATURE_NAMES = FEATURE_NAMES.copy()` and `EXPECTED_FEATURE_DIM = len(EXPECTED_FEATURE_NAMES) (12)`. This module exists purely so `ml/infer_svm.py` can validate an incoming feature vector’s length without importing `image_processing/constants.py` directly — a thin indirection layer that keeps the `ml/` package’s dependency on `image_processing/`’s constant explicit and named for what it means in the ML context (“the shape the model expects”), rather than what it means in the image-processing context (“the names of the seven-plus-five computed features”).

16.2 `ml/model_loader.py` — `load_model_artifacts`

`LoadedModelArtifacts` bundles `model`, `scaler`, `label_encoder` (each `Any | None`), `metadata` (a dict), and a single derived boolean `model_available`. `_safe_load_joblib(path)` returns `None` if the path does not exist, rather than raising — so a missing *any one* of the three joblib files degrades to `None` for that artifact specifically, not a crash. `model_available` is defined as `model is not None and scaler is not None` — notably, the `label_encoder` is *not* part of this check; `_model_predict` (below) tolerates a missing label encoder by falling back to the raw (numeric-string) prediction label instead. `load_model_artifacts` also loads `models/metadata.json` if present, giving callers access to the training-time metadata described in Section 16.4 below.

This function is called from three separate places in the live application, each time re-loading from disk rather than caching the result in memory: `app/main.py`’s startup warning check, `/api/health`, and every single call to `predict_with_svm` (i.e. once per analyzed image). *Whether this repeated disk I/O and joblib deserialization per request is a deliberate simplicity choice or an unaddressed performance opportunity is not verifiable from the available project files* — there is no caching decorator, no module-level singleton, and no comment discussing the trade-off anywhere in `model_loader.py` itself. Given that the model files are small (single-digit megabytes) and joblib deserialization of a scikit-learn `SVC` is fast, this is unlikely to be a practical bottleneck at this project’s scale, but it is a real, observable repeated-work pattern worth noting for anyone optimizing request latency further.

16.3 `ml/infer_svm.py` — `predict_with_svm`

The single entry point, `predict_with_svm(feature_vector, settings)`, is a two-way dispatcher: if `artifacts.model_available` and `artifacts.model` is not `None`, it calls `_model_predict`; otherwise it calls `_heuristic_predict`. Both paths return the same `InferenceResult` shape (`prediction`, `confidence`, `probabilities`, `inference_mode`, `feature_vector_used`), so nothing downstream needs to know or care which path actually ran — the `inference_mode` field is the only observable difference, and it is exactly what `app/main.py`’s startup banner and the README warn about.

`_model_predict` — the real, trained path

`_validate_feature_vector` first asserts the incoming vector has exactly `EXPECTED_FEATURE_DIM` (12) entries, raising `ValueError` immediately if not — a hard fail-fast check rather than silent truncation/padding, appropriate because a length mismatch here would indicate a real bug (e.g. `FEATURE_NAMES` and the trained model’s expected input having drifted out of sync) rather than a normal runtime condition to degrade gracefully from. The vector is reshaped to (1, -1) (a single sample, scikit-learn’s expected 2D input shape), passed through `artifacts.scaler.transform(...)` (the persisted `StandardScaler`, fit at training time — this is why `scaler.joblib` must travel with `svm_model.joblib`: the model was trained on standardized features and would produce meaningless predictions on raw ones), then `artifacts.model.predict(...)`. The raw numeric prediction is decoded back to a string label via `label_encoder.inverse_transform` if a label encoder was loaded, else used as-is. If the model exposes `predict_proba` (true for the shipped `SVC(probability=True)`), per-class probabilities are decoded the same way into a `{"Healthy": p, "Diseased": p}` dict, and `confidence` is the maximum class probability; if not (a model without probability estimates), `confidence` falls back to a fixed 0.75 placeholder.

`_heuristic_predict` — the fallback path

This is a small, self-contained, hand-tuned formula, entirely independent of any trained artifact, using *only the first 7* entries of the feature vector (the original color/edge features, explicitly *not* the 5 texture features added later) — its own comment explains why: “the heuristic formula was hand-tuned against those 7 and isn’t recalibrated for the new ones; the trained model uses the full vector via `_model_predict`.” The formula:

$$\text{disease_signal} = 0.30(1 - \text{green_ratio}) + 0.25 \text{edge_density} + 0.20 \text{entropy} + 0.15 \text{color_variance} + 0.10 \text{saturation}$$

clamped to `[0, 1]`. A threshold of 0.42 separates “Diseased” from “Healthy”; above threshold, the diseased probability is $0.55 + 0.45 \times \text{signal}$ (mapping the above-threshold signal range into a 0.55–1.00 probability band); below threshold, it is $0.15 + 0.60 \times \text{signal}$ (mapping into a 0.15–0.75 band). This ensures the reported probability is always consistent with which side of the 0.42 threshold the raw signal actually fell on. `inference_mode` is hard-set to the literal string “`heuristic_fallback`” — the exact string the README and startup banner tell an operator to watch for.

16.4 The Actual Shipped Model: What Is In `models/`

Unlike most of this documentation, this section describes a specific, concrete artifact rather than general code behavior: the real trained model files shipped inside this exact archive, as recorded in `models/metadata.json`, `models/training_summary.json`, and `models/evaluation/evaluation_metrics.json`.

- **Model type:** `SVM_RBF` (an `sklearn.svm.SVC` with an RBF kernel).

- **Feature names** (12, in order): exactly `FEATURE_NAMES` from Chapter 12 — `brightness`, `green_ratio`, `hue_mean`, `saturation_mean`, `edge_density`, `color_variance`, `entropy`, `glcm_contrast`, `glcm_homogeneity`, `glcm_energy`, `glcm_correlation`, `lbp_uniformity`.
- **Label classes**: `["Diseased", "Healthy"]`.
- **Dataset split**: 608 training images, 153 test images (a fixed, recorded held-out test set — `metadata.json` lists all 153 test filenames explicitly, e.g. `"Healthy(125).jpg"`, `"RingSpot(10).jpg"`, drawn from the same `Original Images/` naming convention the demo dataset itself uses).
- **Class weighting**: `class_weight="balanced"` — scikit-learn’s automatic inverse-frequency class weighting, used because the dataset is not perfectly balanced between the two classes (the test-set class support below, 107 vs. 46, confirms a real class imbalance favoring the Healthy/majority class as labeled here).
- **Hyperparameters**: $C = 2.0$, $\gamma = 0.1$, selected via a real, recorded `GridSearchCV` over a 4×5 grid of $C \in \{0.5, 1, 2, 5, 10\} \times \gamma \in \{"scale", "auto", 0.01, 0.1\}$ (20 candidates total, every one of which is recorded in `metadata.json`’s `cross_validation.params_per_candidate/mean_test_score_per_candidate/std_test_score_per_candidate` arrays), with 5-fold cross-validation scored on `f1_macro`. The winning configuration’s cross-validated score was 0.9652 (macro-F1).
- **Held-out test-set performance** (from `evaluation_metrics.json`, on the 153-image held-out test set, which was never used for training or hyperparameter selection):

Metric	Value
Accuracy	0.9542
Precision (Diseased)	0.9149
Recall (Diseased)	0.9348
F1-score (Diseased)	0.9247
Specificity	0.9626
Matthews Correlation Coefficient (MCC)	0.8920
Cohen’s Kappa	0.8919
ROC AUC	0.9844

The classification report additionally breaks this down per class: class "0" (mapped in the underlying `LabelEncoder`’s sorted order, which alphabetizes `["Diseased", "Healthy"]` such that class 0 is `Diseased` and class 1 is `Healthy` — *this exact index-to-label mapping is asserted here from the alphabetical ordering convention scikit-learn’s `LabelEncoder` uses; it is not restated a second time anywhere in `evaluation_metrics.json` itself, so a reader should treat the class-index-to-label correspondence as inferred, not directly labeled in that file*) shows precision 0.9717/recall 0.9626/F1 0.9671 with support 107, and class "1" shows precision 0.9149/recall 0.9348/F1 0.9247 with support 46 — consistent with the $607/153 = 608+153=761$ total dataset having roughly a 2:1 to 3:1 imbalance toward the majority class in the held-out split specifically (107 vs. 46 out of 153 test images).

Three static evaluation plots are shipped alongside these numbers: `models/evaluation/confusion_matrix.png`, `roc_curve.png`, and `precision_recall_curve.png` — pre-rendered images generated by whatever training/evaluation run produced this specific model (most directly `ml/evaluate.py`, described in Section 17.2), not regenerated live by the running web application.

Chapter 17

Model Training, Evaluation, and SHAP Explainability

17.1 `ml/train_svm.py` — `train_svm_from_feature_csv`

The standalone training entry point (invoked via `scripts/retrain_model.py`, per the README’s retraining instructions). `validate_training_dataframe` first asserts the input CSV has a `filename` column, a `label` column, and all 12 `EXPECTED_FEATURE_NAMES` columns, and that `label` has at least two distinct values — fast, explicit validation before any expensive work begins. Training then: splits into train/test (default `test_size=0.2`, i.e. the real 608/153 split recorded in `metadata.json`); fits a `StandardScaler` on the training features only (never on test data, avoiding train/test leakage); label- encodes the string class names via `LabelEncoder`; runs `GridSearchCV` over `SVM_PARAM_GRID` ($C \in \{0.5, 1, 2, 5, 10\}$, $\gamma \in \{\text{scale, auto, 0.01, 0.1}\}$) with `StratifiedKFold(n_splits=5)` (stratified so each fold preserves the overall class ratio, important given the real class imbalance) scored on `f1_macro`. A module comment explains the grid’s size was kept “intentionally small ... so training stays fast on a CPU-only laptop with a few hundred images” — again reflecting the project’s stated Windows- laptop deployment target, not a cloud GPU cluster. The winning estimator is refit on the full training set (scikit-learn’s default `GridSearchCV(refit=True)` behavior), evaluated on the held-out test set via `ml/evaluate.py`’s `evaluate_binary_classifier`, and every artifact (model, scaler, label encoder, metadata JSON including the full grid-search history) is written to `model_output_dir` via `joblib.dump/save_json`.

17.2 `ml/evaluate.py` and `ml/metrics.py`

`evaluate_binary_classifier` in `evaluate.py` is what produces exactly the metrics dict shown in Section 16.4 — accuracy, precision, recall, F1, specificity, MCC, Cohen’s kappa, ROC AUC, and the full `sklearn.metrics.classification_report` dict — plus the three saved evaluation plots (confusion matrix, ROC curve, precision-recall curve), rendered via `matplotlib` with the same `Agg` non-interactive backend convention used in `export_service.py`. `ml/metrics.py` supplies `compute_specificity` (true negatives over true negatives plus false positives — not directly available as a named function in scikit-learn, so implemented from the raw confusion matrix here) and `compute_binary_metrics`, the shared building blocks `evaluate.py` composes into its full report.

17.3 ml/shap_explainer.py — explain_prediction

The explainability path used by every live analysis (Chapter 9, step 10). Its design centers on one performance concern, addressed with a process-level cache:

Explainer state caching

`_explainer_state_cache` is a plain module-level dict, explicitly *not* `functools.lru_cache` — the code comment states why: “Settings/pydantic objects aren’t hashable,” so `lru_cache` (which needs hashable arguments) cannot be used directly on a function taking `settings`. Instead, `_load_explainer_state` manually keys the cache on `str(settings.models_dir)` (a plain, hashable string) and builds the expensive `shap.KernelExplainer` object (loading the k-means-summarized SHAP background sample from `shap_background.joblib`) exactly once per process, reused for every subsequent call — this is precisely the “7 second first-use cost” that `app/main.py`’s `_warm_up_shap_explainer` is designed to pre-pay at startup rather than on a user’s first request. If model artifacts, the scaler, or the SHAP background file are missing, or building the explainer raises any exception, the cache stores `None` and every subsequent call to `explain_prediction` short-circuits to returning `None` (no SHAP output) rather than repeatedly retrying a known-broken setup.

Computing the explanation

Given a state, `explain_prediction` scales the input feature vector with the same persisted `StandardScaler`, then calls `explainer.shap_values(scaled, nsamples=200, silent=True)`. The comment on `nsamples=200` is explicit about the trade-off: “kept small and fixed (rather than SHAP’s ‘auto’, which can run into the thousands) so a single image’s explanation stays fast enough for a synchronous request - this is an approximation, not an exact Shapley value, which is an accepted trade-off for interactive use.” This is a real, stated accuracy-for-latency trade-off, not an oversight.

The function then selects the SHAP values for the *predicted* class specifically (not, say, always class 0): it maps the string prediction (“Healthy”/“Diseased”) to a binary class index (1 for anything not literally “healthy”, 0 otherwise), matches that against `model.classes_`, and indexes into whichever shape SHAP returned — the code explicitly handles *two* different possible SHAP output shapes (a list of per-class arrays, the older SHAP convention; or a single (1, `n_features`, `n_classes`) 3D array, a newer SHAP convention), which is the same kind of version-compatibility handling documented as Error 3 in this project’s own v3.1 changelog for the related `evaluation/shap_interactions.py` module (Chapter 24) — both modules had to account for SHAP’s API shape changing between versions. The resulting per-feature values are rounded to 6 decimal places and zipped against `EXPECTED_FEATURE_NAMES` into the returned dict; a positive value means that feature pushed the prediction toward the predicted class, a negative value means it pushed away from it (stated directly in the function’s own docstring). The entire body after state retrieval is wrapped in one `try/except Exception`, again explicitly commented “explainability is best-effort, never fatal.”

Chapter 18

Calibration, Ensembling, and Extended Explainability

18.1 `ml/calibration.py` — `calibrate_model`

Module docstring states the exact statistical problem being fixed: `SVC(probability=True)`'s built-in `predict_proba` “derives its ... output from an internal 5-fold Platt scaling pass, but that pass is fit on the *same* training data the SVM itself was fit on, which tends to produce overconfident probabilities.” `CalibratedClassifierCV` fixes this by fitting the calibration mapping on held-out folds instead. `calibrate_model(X_train, y_train, method="isotonic", cv=5, base_estimator=None)` defaults `base_estimator` to a fresh `SVC(kernel="rbf", class_weight="balanced", random_state=42)` if none is supplied, and *adaptively reduces* the number of CV folds (`effective_cv = max(2, min(cv, n_per_class))`) when the smallest class has fewer samples than the requested fold count — guarding against a `StratifiedKFold` construction error on small or heavily imbalanced datasets. The function's own docstring explains the `method` choice directly: “`method="sigmoid"` is Platt scaling (better for small datasets / few hundred samples ...). `method="isotonic"` is more flexible but needs more calibration data (roughly 1000+ samples) ... callers with small datasets should pass `method="sigmoid"` explicitly” — i.e. the module's own default (“`isotonic`”) is documented as *not* necessarily the right choice for this project's actual ~760-image dataset, and callers are explicitly told to override it. Critically, the docstring at the top of the module also states this module “intentionally does not touch `ml/train_svm.py`'s default flow (it stays the well-tested baseline)” — calibration is additive, used by `ml/train_ensemble.py` and `evaluation/statistical_tests.py`'s calibration report, never silently substituted into the main training script.

18.2 `ml/ensemble.py` — `StackingClassifier`

`build_stacking_ensemble` constructs a real scikit-learn `StackingClassifier` with three base learners — an RBF SVM (same configuration as the baseline model, `probability=True`, `class_weight="balanced"`), a `RandomForestClassifier` (300 trees, unlimited depth, `class_weight="balanced"`), and a `GradientBoostingClassifier` (200 estimators, learning rate 0.05, max depth 3) — combined via a `LogisticRegression` meta-learner trained on `stack_method="predict_proba"` (the meta-learner sees each base learner's predicted class probabilities, not just its hard label) using 5-fold (or fewer, adaptively, mirroring `calibrate_model`'s pattern) stratified cross-validation for the internal out-of-fold predictions that train the meta-learner. `passthrough=False` means the meta-learner sees *only* the base learners' predictions, not the original 12 features directly alongside them. The module docstring is explicit that this is “a real `StackingClassifier` (scikit-learn), not soft-voting - the meta-learner learns

how much to trust each base model’s opinion rather than averaging them blindly,” and that it is kept as “a separate, additive model (`models/ensemble_model.joblib`) rather than replacing the SVM pipeline, so the original model stays available for direct comparison” via McNemar’s test (Chapter 24).

18.3 `ml/train_ensemble.py` — Real, Executed Results

This CLI script (`-feature-csv`, `-model-dir`) trains the calibrated SVM and the stacking ensemble, distills a surrogate tree from the ensemble, statistically compares the ensemble against the original baseline SVM (McNemar’s test), and logs everything through `mlops/experiment_tracking.py`. This script was actually run during this project’s own development against the real 761-image feature CSV, producing the following recorded, real (not illustrative) results: ensemble test-set accuracy **94.77%**; McNemar’s test comparing the ensemble against the baseline SVM’s predictions on the same held-out set yielded {statistic : 0.0, *p*-value : 1.0, significant_at_0.05 : false} — i.e. **no statistically significant difference** was detected between the ensemble and the single-SVM baseline on this dataset’s held-out split, a real, honestly-reported negative/null result, not a claim that the ensemble is definitively better; and the distilled surrogate tree achieved **0.9539 fidelity** to the ensemble’s own predictions. Output is written to `models/evaluation/ensemble_comparison.json`.

18.4 `xai/lime_explainer.py` — `explain_with_lime`

Its docstring directly addresses a specific scope question the upgrade wishlist raised (“LIME on image superpixels”), and is worth reproducing because it states a real architectural decision and its rationale plainly:

“This pipeline is deliberately NOT a pixel-level CNN - it classifies from 12 hand-engineered numeric features ..., which is the whole point of the project’s ‘lightweight, explainable-by-design, non-deep-learning’ approach Superpixel-LIME requires a model that consumes raw image pixels; applying it here would need bolting on a separate pixel classifier no one asked to add, which would quietly change the project’s core design decision. What genuinely applies to *this* architecture is LIME’s tabular mode ..., which perturbs the 12 engineered features around an instance and fits a local linear model SHAP already covers global feature attribution, LIME adds a second, model-agnostic cross-check on the *local* explanation for one specific prediction.”

Structurally, this module closely mirrors `ml/shap_explainer.py`: a module-level cache (`_lime_explainer_cache`, keyed the same way on `str(settings.models_dir)`), a background reference reused directly from the *same* `shap_background.joblib` file SHAP uses (“rather than requiring a second artifact”), `None`-on-any-failure at every stage, and the same explicit “best-effort, never fatal” framing. `LimeTabularExplainer` is constructed with `discretize_continuous=True` (LIME bins continuous features into quartile-like ranges for more interpretable rule descriptions in its output, e.g. “`edge_density > 0.34`” rather than a raw linear coefficient) and `class_names=["Healthy", "Diseased"]`. `explain_with_lime(feature_vector, settings, num_features=6)` scales the feature vector with the persisted scaler, wraps the model’s `predict_proba` as LIME’s required black-box prediction function, calls `explainer.explain_instance(..., num_features=min(6, 12))` (returning only the 6 most locally-influential features by default, not all 12), and returns the result as {`feature_description`: `weight`} where `feature_description` is LIME’s own human-readable condition string (from `explanation.as_list()`), not a bare feature name — this is a structurally different key format from SHAP’s plain-feature-name keys, a detail relevant to anyone consuming `lime_contributions` and `shap_contributions` from the same `ImageResult` and expecting matching key sets.

18.5 `xai/surrogate_tree.py` — `distill_surrogate_tree`

Its docstring frames the goal precisely: “give a plain-language rule summary of what the SVM has generally learned, e.g. ‘IF `edge_density > 0.18` AND `green_ratio < 0.42` THEN `Diseased`’.” The critical implementation detail, stated in the function’s own docstring, is that `surrogate_labels = base_model.predict(X_train_scaled)` — **the tree is trained on the base model’s own predictions, not the original ground-truth labels**: “that’s what makes it a surrogate of the model rather than a second, independent classifier.” A shallow `DecisionTreeClassifier` (`max_depth=4`, `min_samples_leaf=max(5, 2% of training set)`) is fit on those surrogate labels, and **fidelity** — how often the tree’s own predictions agree with the labels it was trained to imitate, computed via `accuracy_score(surrogate_labels, tree_predictions)` — is reported alongside the human-readable rule text (`sklearn.tree.export_text`) and per-feature importances. The module docstring is explicit about how to interpret a low fidelity score: “a low-fidelity surrogate should not be trusted as a rule summary.” The real, recorded fidelity from this project’s own `ml/train_ensemble.py` run (distilling the stacking ensemble specifically) was **0.9539**, reported in Section 7.3 above — high enough that the resulting rule text is a reasonably faithful plain-language summary of the ensemble’s overall decision logic, per the module’s own stated fidelity-interpretation standard.

Chapter 19

Feature Quality, Active Learning, Federated Learning, and LoRA Groundwork

19.1 `ml/feature_quality.py` — `analyze_feature_quality`

Runnable directly against the existing `data/training_features.csv` with no new data needed. `analyze_feature_quality(X, y, feature_names)` returns a `FeatureQualityReport` combining four independent statistical analyses of the 12-feature representation: PCA (`run_pca_analysis` — explained variance ratio per component and how many components are needed to reach 95% cumulative variance, revealing how much redundancy exists among the 12 features), per-feature mutual information with the target label (`run_mutual_information`, via `sklearn.feature_selection.mutual_info_c`), Recursive Feature Elimination with Cross-Validation (`run_rfecv`, using an `SVC` estimator, returning which features RFECV would keep and every feature’s elimination ranking), and Linear Discriminant Analysis class separation (`run_lda_separation` — a single scalar score of how linearly separable the two classes are in the 12-feature space). This module answers “are these 12 features actually pulling their weight,” a question distinct from (and complementary to) the SHAP/LIME/surrogate-tree explainability machinery, which explains *a trained model’s* behavior rather than the raw feature set’s statistical properties.

19.2 `ml/active_learning.py` — **Built, Not Yet Applicable**

The module’s own docstring states its real-world applicability honestly: “these techniques are only *useful* once a pool of unlabeled field images exists (e.g. photos submitted through the chatbot without a confirmed outcome yet) - the shipped demo dataset is fully labeled, so there’s nothing to actively sample from today.” It also explains a library choice: rather than adding the `modAL` package as a dependency, the functions are implemented directly with `numpy`, because the underlying math is, in the docstring’s words, “a few lines of `numpy` over `predict_proba` outputs already available from this project’s models.”

- `uncertainty_sampling(model, X_unlabeled, n_samples=10)` — ranks unlabeled samples by how close their predicted probability is to the decision boundary (maximum uncertainty $\approx$ probability near 0.5), returning the `n_samples` most uncertain as `UncertaintySample` records.
- `query_by_committee(models, X_unlabeled, n_samples=10)` — given multiple trained models (e.g. the ensemble’s individual base learners), scores samples by how much the committee *disagrees* with itself, a classic active-learning strategy distinct from single- model uncertainty.
- `self_training_pseudo_labels(...)` — generates pseudo-labels for unlabeled samples the

model is confident about, gated by a confidence threshold, for semi-supervised bootstrapping.

Confirmed by codebase search: none of these three functions is called anywhere in the live application or any script with actual unlabeled data — they are exercised only by `tests/test_feature_quality_and_active_learning.py` against synthetic/held-out data, consistent with the module’s own stated honesty note that there is currently no real unlabeled pool to run them against.

19.3 `ml/federated_learning.py` — A Real, Executed Simulation

This module’s docstring is the clearest example in the codebase of a result being reported with an explicit, precise boundary around what it does and does not prove:

“Honesty note on what this does and doesn’t prove: real federated learning’s privacy property comes from training across genuinely separate physical devices that never share raw data with each other or a central server - that requires actual farmer devices, which this project has none of. What’s implemented and *run* here is Flower’s simulation mode: the real `flwr` federated-averaging algorithm, the real client/server API, executed against `n_clients` simulated partitions of the existing shipped feature dataset This proves the FL *training mechanics* work correctly on this project’s feature representation ... - genuine progress over ‘not started.’ It does not, and cannot, prove the cross-device privacy property that only matters once `n_clients` are physically separate farmer phones instead of `array[i::n]` partitions of one CSV in one process.”

The docstring also explains a specific model-choice constraint: `SVC` (used everywhere else in this project) has no `partial_fit` method, so it cannot participate in federated averaging, which works by exchanging and averaging model parameters across communication rounds. `SGDClassifier(loss="hinge")` — “mathematically a linear SVM trained via stochastic gradient descent” — *does* support `partial_fit`, and its `coef_/intercept_` arrays are exactly the parameters `FedAvg` averages. This is therefore not the same model family as the shipped RBF-kernel SVM; it is a separate linear approximation used specifically because it is FL-compatible.

`partition_data(X, y, n_clients, seed=42)` performs a class-stratified split of the dataset into `n_clients` partitions — the docstring notes partitioning is by row index, “standing in for ‘different farms/sessions’ since the dataset has no actual per-image source field yet,” explicitly cross-referencing the same missing-metadata gap noted for `evaluation/fairness_audit.py` (Chapter 24). `run_federated_simulation(X, y, n_clients=4, n_rounds=10, ...)` wraps each partition’s data in a real `fl.client.NumPyClient` subclass (`SklearnFlowerClient`) and runs `fl.server.strategy.FedAvg` over `fl.simulation.start_simulation`, returning a `FederatedRoundResult` per round (`aggregated_accuracy`, `per_client_accuracy`).

This was actually executed during this project’s development, against the real shipped feature dataset, with 4 simulated clients over 5 rounds, producing the real, recorded accuracy trajectory:

[0.3026, 0.8470, 0.8651, 0.8848, 0.8749]

— a low first-round accuracy (each client starts from a randomly initialized linear model trained on only its own partition, before any aggregation) rising sharply after the first `FedAvg` aggregation round and continuing to improve through round 4, with a small dip at round 5 (a normal, expected pattern in federated optimization, not evidence of a bug). Getting this to run required installing `flwr[simulation]` specifically (rather than plain `flwr`), which pulls in the `ray` distributed-computing library as `flwr`’s simulation backend — documented in this project’s own development history as Error 5, an `ImportError` that was resolved by installing the `[simulation]` extra.

19.4 `rag/qa_dataset_builder.py` and `ml/lora_finetune.py` — Fine-Tuning Groundwork

`rag/qa_dataset_builder.py`'s docstring identifies exactly which roadmap prerequisite it closes: LoRA/QLoRA fine-tuning “needs ... a curated Q&A dataset to fine-tune on.” `generate_qa_pairs(kb_path)` applies a fixed set of category-specific question templates (`_CATEGORY_QUESTION_TEMPLATES`, e.g. a “symptoms”-category KB chunk generates “What are the symptoms of {title}?” and “How can I recognize {title} on my papaya plant?”) to each of the real 30 curated knowledge-base chunks in `rag/kb/prsv_knowledge.json`, producing one or more Alpaca-format `{"instruction", "input", "output", "source_chunk_id", "category"}` pairs per chunk, where “output” is always the KB chunk’s own verbatim curated text. The module’s honesty note states plainly: “this produces genuine training pairs (not synthetic/LLM-hallucinated content - every answer is verbatim from the human-reviewed knowledge base), but 30 source chunks yields a modest few dozen pairs - usable for a first LoRA fine-tuning pass, but small by fine-tuning-dataset standards.” **This was actually run**, producing 41 real Q&A pairs, persisted to `data/lora_qa_dataset.json` and tracked by DVC.

`ml/lora_finetune.py`'s own docstring is unambiguous about its execution status: “genuinely written to run, NOT executed in this environment. Be clear-eyed about that distinction: this is real, correct code following the standard PEFT/QLoRA recipe, but it needs a GPU (this sandbox has none) and a `transformers/peft/bitsandbytes` + PyTorch install ..., also impractical in this sandbox’s constrained network egress.” `load_alpaca_dataset` converts the 41-pair JSON into a HuggingFace `datasets.Dataset`; `run_lora_finetune(qa_dataset_path, output_dir, base_model_name="microsoft/Phi-3-mini-4k-instruct", lora_rank=16, ...)` configures a 4-bit quantized load (`BitsAndBytesConfig`) of the base model and a `LoraConfig` targeting the attention projection matrices (`q_proj/k_proj/v_proj/o_proj`) — the standard QLoRA recipe from the PEFT library, correctly assembled but never actually invoked in this sandbox. A `main()` CLI entry point and a usage comment (`pip install torch transformers peft bitsandbytes accelerate`, then `python -m ml.lora_finetune -qa-dataset data/lora_qa_dataset.json -output-dir models/lora_adapter`) document exactly how to run it once GPU infrastructure is available.

Part IV

Retrieval-Augmented Generation and the Chatbot

Chapter 20

The Knowledge Base and Core RAG Pipeline

20.1 `rag/kb/prsv_knowledge.json` — The Knowledge Base

The single source of truth for every piece of PRSV domain knowledge the system can surface is a hand-curated JSON file containing exactly **30 knowledge chunks**. Each chunk (modeled by the `KnowledgeChunk` dataclass in `rag/schemas.py`) has: `chunk_id` (e.g. "prsv_001"), `title` (e.g. "PRSV overview"), `category`, `audience`, `text` (the actual curated explanation, one to a few sentences), and `tags` (a list of keywords). The 30 chunks span 14 distinct categories, confirmed by direct inspection of the file: `disease_description`, `symptoms`, `symptom_interpretation`, `severity`, `transmission`, `impact`, `prevention`, `treatment`, `management`, `general_care`, `schemes`, `advisory`, `explanation_support`, `tool_usage`, and `chat_faq` (the tool-usage and FAQ categories reflecting that this KB is also used to answer questions *about the app itself*, not only about the disease). This is the entire knowledge the RAG and chatbot subsystems can draw on — **there is no larger external knowledge source**; every explanation, chatbot answer, and retrieved-evidence citation anywhere in the system ultimately traces back to one of these 30 human-written chunks.

20.2 `rag/knowledge_loader.py`

`load_knowledge_base(kb_path)` is the one function that turns the raw JSON file into a list of `KnowledgeChunk` objects, raising `FileNotFoundError` immediately if the file is missing (this is what `/api/health's kb_available` check ultimately depends on). Every retriever in the `rag/` package (TF-IDF, BM25, dense, hybrid) is constructed from this same loaded chunk list.

20.3 `rag/schemas.py`

Two dataclasses: `KnowledgeChunk` (the raw KB entry, above) and `RetrievedChunk` (a `KnowledgeChunk` plus retrieval-time metadata: `similarity_score`, and three v3.0-added fields — `retrieval_method` (default "tfidf"), `sparse_rank`, `dense_rank` (both `Optional[int]`, default `None`). The in-code comment on these three explains the default choice: "Defaults keep every existing caller (TF-IDF-only retriever, existing tests) working unchanged" — the same backward-compatible schema-evolution pattern seen in `ImageResult` (Chapter 8).

20.4 `rag/retriever.py` — `LocalTfidfRetriever`

The original (pre-v3.0) retrieval method, and still the ultimate fallback of last resort if every optional retrieval dependency is unavailable. Builds one `sklearn.feature_extraction.text.TfidfVectorizer`

(English stop words removed) over a synthetic “document” per chunk — `f"{title} {category}{' '.join(tags)} {text}"`, i.e. title and tags are concatenated *ahead of* the body text, weighting them into the same bag-of-words representation as the text itself. A query is vectorized with the same fitted vectorizer, scored against every chunk by cosine similarity, and the top-*k* by score are returned.

20.5 `rag/query_builder.py` — Turning Pipeline Output Into a Query

Two functions convert the numeric output of the image-analysis pipeline into text the retriever can search against. `build_observation_query` produces one long, formulaic sentence embedding the prediction, severity label/score, four specific feature values (green ratio, edge density, entropy, saturation mean), the two symptom-finding scores (abnormal color score, symptom region ratio), and ends with a fixed suffix — `"possible chlorosis, mosaic, discoloration, texture irregularity, PRSV symptoms."` — deliberately injecting PRSV-relevant vocabulary directly into the query text so the retriever, whichever backend is active, has strong lexical/semantic overlap with the KB’s own disease-symptom vocabulary. `build_key_findings` produces a flat list of human-readable strings (`"Prediction: Diseased"`, `"Confidence: 0.9123"`, etc.) — this is exactly what populates `ExplanationTrace.key_findings`, a separate, simpler summary from the two prose explanations described next.

20.6 `rag/generator.py` — The Deterministic Template Generator

The always-available fallback generator (used whenever LLM generation is off, unconfigured, or fails). `generate_technical_explanation` and `generate_farmer_friendly_explanation` both branch on whether the prediction is Healthy or not (producing one of two fixed opening-sentence templates each), then append the same numeric feature summary (technical) or a plainer guidance summary (farmer-friendly), each ending with an `_compose_evidence_summary` call — which simply joins the `text` of the top 2 retrieved chunks with a space, or returns “No supporting PRSV evidence was retrieved.” if none were found. `generate_advisory_notes` returns a fixed base list of three general-purpose notes (inspect nearby leaves; monitor for aphids; keep the report for records), **conditionally appending** a fourth (“Isolate or mark PRSV-suspected plants ...”) only if the prediction is not Healthy, and a fifth (“Seek agricultural expert review quickly ...”) only if the severity label is one of `{"moderate", "moderate to severe", "severe"}` (case-insensitive) — so a mildly-diseased leaf gets 4 notes and a severely-diseased leaf gets 5, while a Healthy leaf always gets exactly the base 3.

20.7 `rag/hybrid_retriever.py` — HybridRetriever

The v3.0 upgrade to retrieval, and the retriever actually used by `RagService` whenever `settings.rag_use_hybrid_` is true (the default). Its docstring states the rationale for combining two retrieval signals directly: “BM25 (exact keyword recall - important for chemical/product names a farmer might type verbatim, e.g. ‘imidacloprid’) with dense embedding retrieval (semantic recall - handles paraphrase and, after translation to English, cross-lingual queries) via Reciprocal Rank Fusion (RRF), which is the standard, parameter-light way to combine two differently-scaled ranking signals without having to hand-tune a score weighting.”

Three-stage graceful degradation

The docstring states the fallback chain explicitly: (1) if both `rank_bm25` and the dense embedding stack (`sentence-transformers` + optionally `faiss`) are available, run full hybrid RRF; (2) if only one of the two is available, use that one alone; (3) if neither is available, fall back to the original

`LocalTfidfRetriever` — “so this module is a strict upgrade, never a regression, on any machine including ones without the new optional deps.” This is implemented via the `active_mode` property, which inspects `self._sparse.is_available()` and `self._dense.is_available()` and returns one of the four literal strings `"hybrid_rrf"`, `"dense_only"`, `"bm25_only"`, or `"tfidf_fallback"` — exactly the string surfaced by `RagService.retrieval_mode` and used as part of the RAG cache key (Section 11.1), so cached results are automatically invalidated/segmented per retrieval mode (a cache entry computed under `"bm25_only"` is never returned for a query under `"hybrid_rrf"`, since the cache key includes the mode string).

`_Bm25Sparse`

A thin wrapper around `rank_bm25.BM25Okapi`, built over the same `title+category+tags+text` document concatenation as `LocalTfidfRetriever`, tokenized by a trivial `text.lower().split()` (whitespace tokenization, not a proper tokenizer library — adequate for a small, English-language 30-document corpus).

Reciprocal Rank Fusion (`_fuse`)

For each chunk that appears in either the sparse or dense candidate list (pooled from `candidate_pool` candidates each, default 15, per `settings.rag_candidate_pool_size`), an RRF score is accumulated:

$$\text{RRF}(c) = \sum_{\text{source} \in \{\text{sparse}, \text{dense}\}} \frac{1}{k + \text{rank}_{\text{source}}(c)}$$

with $k = 60$ (`RRF_K`), the standard damping constant from the original Reciprocal Rank Fusion paper (Cormack et al.), noted directly in a code comment. A chunk that appears in *both* lists accumulates score from both terms (naturally rewarding cross-signal agreement); a chunk missing from one list contributes only its other term. The final ranking sorts by this fused score descending and returns the top `top_k`, each tagged `retrieval_method="hybrid_rrf"` and carrying both its `sparse_rank` and `dense_rank` (whichever apply) for full traceability of how each result was found.

20.8 `app/services/rag_service.py` — `RagService`

The orchestrator that ties the knowledge base, retriever, generator, and cache together, and the sole entry point `AnalysisService` calls (Chapter 9). Its own docstring is an accurate, compact summary of the whole subsystem:

“Retrieval: uses `HybridRetriever` ... when `settings.rag_use_hybrid_retrieval` is `True` and the optional packages are installed; otherwise falls back to the original TF-IDF retriever, so behavior on a minimal install is identical to v2.9. Generation: attempts a real, grounded Claude API call (`rag/llm_generator.py`) when `settings.rag_use_llm_generation` is `True` and `ANTHROPIC_API_KEY` is configured. On any failure ... it falls back to the original deterministic template generator ..., which always works and is what the technical/advisory-notes text used elsewhere in the pipeline still uses for consistency. Caching: repeated observation queries ... are cached via `CacheService` to avoid redundant retrieval + generation work.”

`build_explanation_trace` — the full trace

1. Build the observation query text (`build_observation_query`).
2. Compute a cache key from (`"rag_trace"`, `observation_query`, `self.retrieval_mode`) and check the cache — **note this cache key does not include prediction, confidence, or any feature value directly**; it is derived entirely from the already- query-string-encoded

observation text (which itself embeds those values as formatted numbers) and the retrieval mode. Two images that happen to produce byte-identical observation query strings (i.e. identical prediction, severity label/score to 2 decimals, and the four embedded feature values to 4 decimals) will hit the same cache entry and receive an identical cached `ExplanationTrace`, including identical LLM-generated text if that path was used originally — an efficiency-motivated behavior explicitly intended by the caching design, not a bug.

3. On a cache miss: retrieve chunks via the active retriever; build `retrieved_evidence` and `key_findings`; generate the technical explanation via the deterministic template generator (**always**, regardless of LLM configuration — the technical explanation path never calls the LLM); attempt the farmer-friendly explanation via `generate_grounded_explanation` (Chapter 21) *only if* `settings.rag_use_llm_generation` is true, falling back to the deterministic `generate_farmer_friendly_explanation` if the LLM call returns falsy (`None` or empty string) for any reason; generate advisory notes (always deterministic, never LLM-based).
4. Assemble the `ExplanationTrace`, cache it, and return it.

This asymmetry — the **technical** explanation and advisory notes are always template-generated, while only the **farmer-friendly** explanation ever goes through the optional LLM path — is a real, observable architectural choice in the shipped code: it means the technical explanation (which quotes precise feature values) is always fully deterministic and reproducible, while only the more conversational, plain-language explanation can vary between an LLM call and a template, depending on configuration.

Chapter 21

Advanced RAG: Dense Retrieval and Grounded LLM Generation

21.1 rag/embeddings_retriever.py — DenseEmbeddingRetriever

Adds semantic (meaning-based, not just keyword-based) retrieval. The module docstring frames the model choice explicitly: “Uses `sentence-transformers` with a small multilingual model (`paraphrase-multilingual-MiniLM-L12-v2`, ~118M params, CPU-friendly) rather than a larger model like `bge-m3`, so it stays usable on the same modest hardware the rest of this project targets. Swapping the model name is a one-line change (`EMBEDDING_MODEL_NAME` below) if more accuracy is needed.” The same multilingual model is reused by `rag/reranker.py`... actually reused independently is `nlp/intent_classifier.py`, which imports the identical `EMBEDDING_MODEL_NAME` constant rather than hardcoding its own model string, so the two modules cannot silently drift onto different embedding spaces.

Backend selection: FAISS vs. plain numpy

`_build()` always tries to load the sentence-transformer model and encode every KB chunk into a normalized embedding matrix first; *then*, separately, it tries to build a FAISS `IndexFlatIP` index over those embeddings (inner product over L2-normalized vectors is mathematically equivalent to cosine similarity). If FAISS import/construction fails, it falls back to `self._backend = "numpy"` rather than failing the whole retriever — because at this KB’s actual size (30 chunks), a FAISS index provides no measurable performance benefit over a plain `embeddings @ query_vector` matrix multiply; the docstring states this directly: “FAISS mainly pays off once the KB grows past a few thousand chunks, which is exactly the trigger condition called out in the upgrade wish-list.” If the *outer* try (loading `sentence-transformers` itself, or encoding) fails for any reason — package missing, model download failure, out-of-memory — the backend becomes `"unavailable"` and `is_available()` returns false, triggering `HybridRetriever`’s next fallback stage.

`retrieve(query, top_k=5)` encodes the query with the same model, searches either the FAISS index or the numpy matrix (branching on `self._backend`), and returns `RetrievedChunk` objects tagged `retrieval_method="dense"` with a `dense_rank` (1-indexed) set — `sparse_rank` is left at its default `None` on these results, exactly what `HybridRetriever._fuse` expects when combining dense hits with sparse hits.

21.2 rag/llm_generator.py — generate_grounded_explanation

The one place in the entire codebase that makes a real Anthropic API call. Its docstring is explicit about the opt-in, fail-safe design: “No ANTHROPIC_API_KEY configured, or the `anthropic` package missing, or the API call errors/times out → returns `None`, and callers ... fall back to the original deterministic template generator. The app therefore works identically to v2.9 out of the box, and only produces LLM-generated text once an API key is explicitly set.”

`is_configured()` is a one-line check: `bool(os.environ.get("ANTHROPIC_API_KEY"))` — reading directly from the process environment, not from `settings`, which is a subtle but real distinction: even though `app/config.py` does not declare an `anthropic_api_key` field on `Settings` at all, this module still picks up the key correctly because `pydantic-settings` loads `.env` into the actual process environment via `python-dotenv` semantics as part of `Settings`’ own construction, and `os.environ.get` then sees it directly — *whether this was a deliberate design choice (avoid ever holding the API key as a typed `Settings` field, reading it only at the point of use) or simply an omission from `Settings` is not verifiable from the available project files.*

The grounding system prompt

The full, fixed system prompt is worth reproducing verbatim, since it is the entire mechanism by which this project attempts to keep LLM output factually anchored to the curated KB rather than the model’s own general training knowledge:

“You are an agricultural assistant explaining a Papaya Ring Spot Virus (PRSV) leaf-image diagnosis to a farmer or an agronomy reviewer. Base your answer ONLY on the structured findings and retrieved knowledge chunks provided below. Do not introduce facts, chemical names, or treatment claims that are not present in the provided context. If the context does not fully answer something, say so plainly instead of guessing. Keep the answer concise, practical, and non-alarmist.”

`_format_context` builds the user-facing context block: a “Structured findings” section (prediction, confidence, severity label/score) followed by a “Retrieved PRSV knowledge” section listing each retrieved chunk as “- [{title}] {text}”, or the literal string “(no knowledge chunks retrieved)” if retrieval returned nothing — so the model is never left to guess what evidence exists; an empty-retrieval case is stated to it explicitly. An `audience_instruction` switches the register: “farmer” (the only value ever passed by `RagService`) gets “Write for a farmer with no technical background: short sentences, plain language, no jargon”; any other value gets a more technical instruction — though in practice, confirmed by inspection of every call site, `audience` is always literally “farmer” throughout the live application, so the technical-audience branch is real, working code that is never actually exercised by the shipped app’s own call sites.

The API call itself

`model=GENERATION_MODEL` (the fixed string “claude-sonnet-4-6”), `max_tokens=500`, a single user message combining the audience instruction, the formatted context, and a fixed closing instruction (“Explain this diagnosis and what to do next.”). The response’s content blocks are filtered to just the “text”-type blocks and joined; an empty result (after stripping whitespace) is normalized to `None` rather than an empty string, so the caller’s falsy-check (if not `farmer_friendly_explanation:`) in `RagService` correctly triggers the template fallback even on a technically-successful-but-empty API response. The entire call is wrapped in one `try/except Exception`, consistent with every other optional external dependency in this codebase.

Reachability note: since this sandbox environment has no `ANTHROPIC_API_KEY` configured and the code intentionally never disables egress checks, `is_configured()` returns `False` in this environment and `generate_grounded_explanation` always short-circuits at its very first line; live

end-to-end verification of an actual Claude API round-trip from within this specific sandbox is therefore *not verifiable from the available project files* — what is verifiable is that the code path, request construction, and fallback wiring are all present, correctly structured, and covered by `tests/test_llm_generator.py` (which mocks the `anthropic` client rather than making a live call).

Chapter 22

RAG Quality Layers: Reranking, Expansion, Correction, Graph, Structured Output, Routing, Evaluation

Seven further v3.1 modules round out the RAG subsystem. None of them is called by `RagService` in the live analysis pipeline (confirmed by codebase search of `app/services/rag_service.py` and `app/services/chatbot_service.py` — neither imports `reranker.py`, `query_expansion.py`, `corrective_rag.py`, `knowledge_graph.py`, `structured_output.py`, or `agent_orchestrator.py`); each is complete, real, independently tested code, exercised through its own test file and directly importable for exploratory use or a future integration, following the same “real machinery, not wired into the live request path” pattern already seen for `image_processing/segmentation_classical.py` and its siblings (Chapter 15).

22.1 `rag/reranker.py` — Cross-Encoder Reranking

`rerank(query, candidates, top_k=None)` re-scores an already-retrieved candidate list with `sentence_transformer` (model `cross-encoder/ms-marco-MiniLM-L-6-v2`), cached in a module-level dict the same way the reranker and SHAP/LIME explainers are. The docstring explains the accuracy/cost trade-off against the bi-encoder retrievers already in use: a cross-encoder “scores a (query, passage) pair jointly, rather than encoding them separately ... Cross-encoders are slower per-pair but meaningfully more accurate ..., which is exactly what a small top-K candidate list benefits from” — i.e. cross-encoders don’t scale to searching a whole corpus, but are well suited to re-ordering the 15-candidate pool `HybridRetriever` already narrows to. `retrieval_method` on each reranked result is suffixed with “+reranked” (e.g. “`hybrid_rrf+reranked`”), so the provenance trail records both which retrieval method *found* the chunk and that reranking subsequently reordered it.

22.2 `rag/query_expansion.py` — `expand_query`

A two-tier fallback, stated directly in the docstring’s motivating example: rewriting a query like “*ring spots wat 2 do*” into something closer to “*PRSV ring spot symptoms treatment recommendations.*” Tier 1 is an LLM rewrite (only attempted if `rag.llm_generator.is_configured()`), using a small, single-purpose system prompt instructing the model to output *only* the rewritten query, capped at 100 tokens. Tier 2, `_lexical_expand`, is a fully offline fallback: a small hand-written `_SYNONYM_MAP` (e.g. “`spots`” → [`“lesion”`, `“spot”`, `“mark”`, `“patch”`], `“cure”` → [`“treatment”`, `“remedy”`, `“management”`], and even common shorthand like `“wat”` → [`“what”`])

and "2" → ["to"], an explicit nod to informal chat-message spelling) plus a substring-matching pass against the KB's own tag vocabulary (if `cleaned in tag` or `tag in cleaned`), appending any matched expansion terms to the original query. If neither tier produces anything, the original query is returned unchanged.

22.3 `rag/corrective_rag.py` — `grade_and_decide`

Grades retrieval quality *before* generation, explicitly framed against the current production behavior: “rather than the current `rag_service.py` behavior of only falling back to the template generator on API/key failure regardless of retrieval quality.” It reuses `rag.rag_eval._similarity` (below) rather than adding a second scoring mechanism, with a stated rationale: “a full LLM-based relevance grader would cost an extra API call per turn just to decide whether to make a second one.” Each retrieved chunk is scored against the query; chunks scoring ≥ 0.15 (`relevance_threshold`) are kept as `relevant`. If at least 34% (`min_relevant_fraction`) of retrieved chunks are relevant, the decision is `action="generate"` using *all* of them; if fewer than 34% are relevant but at least one is, the decision is *still* `"generate"`, but restricted to only the relevant subset (the code comment: “Some signal, but weak - still worth generating from the relevant subset rather than discarding everything”); if *none* clear the threshold, or nothing was retrieved at all, the decision is `action="fallback"`.

An observed discrepancy worth noting explicitly: the module's own top-of-file docstring describes three possible outcomes — re-query with an expanded query, or fall back to a safe response — and the `CragDecision.action` field's inline comment types it as `# "generate", "requery", "fallback"`. However, a direct reading of `grade_and_decide`'s actual control flow shows it **only ever returns "generate" or "fallback"** — there is no code path that produces `action="requery"` anywhere in this function. This is therefore documented scaffolding for a three-way decision (and a caller could reasonably check for a `"requery"` string that will simply never appear) rather than a bug that crashes anything; *whether the "requery" action was planned for a future revision and never finished, or the comment is simply stale relative to the implemented two-way logic, is not verifiable from the available project files.*

22.4 `rag/knowledge_graph.py` — A Real Graph Over the 30-Chunk KB

`build_graph(kb_path)` returns a `PrsvKnowledgeGraph` (nodes = the 30 `KnowledgeChunks`, edges = `GraphEdge` objects) built by a genuinely graph-structured, not arbitrary, rule: a fixed `_STAGE_ORDER` list groups the KB's 14 categories into four ordered stages — (1) `{symptoms, symptom_interpretation, disease_description}`, (2) `{transmission, impact}`, (3) `{treatment, prevention, management, general_care}`, (4) `{advisory, severity, schemes, chat_faq, tool_usage, explanation_support}` — reflecting, per the module's own comment, “the causal ordering a multi-hop ‘why’ question typically traverses: what it looks like -> why it happens / how it spreads -> what to do about it.” An edge connects two chunks when they are in *adjacent* stages *and* share at least one tag, via `_stage_of(category)` mapping each chunk to a stage index. `multi_hop_query(graph, start_chunk_id, max_hops=2)` performs a breadth-first traversal from a starting chunk out to `max_hops` edges, and `find_best_start_node(graph, query_tags)` picks the chunk with the greatest tag overlap with a set of query tags as the traversal's entry point. The docstring is candid about scope: “this is a real graph over the *existing* 30-entry knowledge base ... not a production knowledge-graph database (Neo4j, etc.), which would be overengineering for 30 entries. The graph structure and multi-hop traversal logic are genuine and testable; swapping the in-memory graph for a real graph database is a mechanical change if the knowledge base grows large enough to need one.”

22.5 rag/structured_output.py — Schema-Constrained Generation

`generate_structured_diagnosis(...)` returns a Pydantic `StructuredDiagnosis` model (`summary`, `actions` — a list of prioritized next steps, plus, per the tool schema, a confidence caveat field) instead of free text. The technique, explained in the docstring: “rather than a generic ‘constrained decoding’ library (`outlines/guidance` need direct control of token sampling, which the Anthropic Messages API doesn’t expose), this uses Claude’s structured-output-via-tool-use pattern - defining a single-purpose ‘tool’ whose input schema IS the desired output shape, and forcing the model to call it.” Concretely, `_OUTPUT_TOOL_SCHEMA` defines a tool named `emit_diagnosis_summary` with a JSON Schema `input_schema`; the API call passes `tool_choice={"type": "tool", "name": "emit_diagnosis_summary"}`, forcing Claude to respond with a tool call matching that schema rather than free text, and the tool call’s parsed input is validated directly into the `StructuredDiagnosis` Pydantic model — giving schema-guaranteed JSON without needing a constrained- decoding library. Same fail-safe pattern: `None` on any failure (no key, package missing, API error, or Pydantic validation failure).

22.6 rag/agent_orchestrator.py — route

A per-chatbot-message router deciding between three `OrchestratorAction` values — `ANALYZE_PHOTO`, `RETRIEVE_KNOWLEDGE`, `DIRECT_ANSWER`. The module docstring is candid about what “agent” means here versus a full autonomous agent loop: “a full LangChain/LlamaIndex agent loop lets an LLM freely decide which of N tools to call, in what order, for how many steps What’s implemented below is a real, testable, and honest middle ground: a deterministic router when no LLM is configured ..., and a genuine LLM-driven tool-choice call (Claude tool use) as a strictly-better upgrade path when a key IS configured. Both paths converge on the same [action space], so swapping between them doesn’t change what the caller ... sees.” `_rule_based_route` uses two small fixed keyword sets — `_HAS_PHOTO_HINTS` ("photo", "picture", "image", "attach", "upload", "check this") and `_HAS_ANALYSIS_KEYWORDS` ("diagnose", "diagnosis", "what disease", "is this prsv", "check my plant") — to pick a route without any model call at all; `_llm_route` offers Claude the same three actions as callable tools and lets the model choose, used only when configured. Like the six other advanced-RAG modules in this chapter, `route()` is not invoked anywhere in `app/services/chatbot_service.py`’s actual message-handling flow today (Chapter 23 describes that flow’s real, simpler routing logic).

22.7 rag/rag_eval.py — RAGAS-Lite Evaluation

The evaluation harness whose `_similarity` function `corrective_rag.py` reuses. Its docstring states the scope trade-off against real RAGAS directly: “the real RAGAS library scores faithfulness and answer relevance by using an LLM-as-judge to decompose the answer into claims and check each one against the retrieved context. That gives the most reliable score but costs an LLM call per evaluation and needs an API key. This module implements the same four metrics with a lightweight, dependency-light approximation (embedding cosine similarity when `sentence-transformers` is available, falling back to token-overlap/Jaccard otherwise) so evaluation can run for free, offline, in CI.” `_similarity(text_a, text_b)` tries `_try_embedding_similarity` first (encoding both texts with a shared, cached `SentenceTransformer` instance and computing cosine similarity), falling back to `_jaccard` over stop-word-filtered token sets (`_tokenize`, which drops a small fixed `_STOPWORDS` set and any token of length ≤ 2) if embeddings are unavailable.

The four RAGAS-lite metrics — **faithfulness** (is the generated answer supported by retrieved context, a hallucination proxy), **answer_relevance** (does the answer address the query), **context_precision** (what fraction of retrieved chunks are actually relevant), and **context_recall**

(how much of the query’s information need is covered by the union of retrieved chunks) — are all computed with this same similarity primitive rather than four independent scoring mechanisms. `evaluate_rag_response` scores one query/answer/context triple; `evaluate_batch` aggregates over many. A separate, higher-fidelity `evaluate_with_llm_judge()` is offered as the option to prefer “for a final research report” once an LLM is configured, again following the consistent two-tier (offline approximation → LLM-backed upgrade) pattern used throughout this RAG subsystem.

Chapter 23

The Chatbot: Service, Translation, and NLP

23.1 `app/services/chatbot_service.py` — `ChatbotService`

The class docstring states its scope and a deliberate consistency decision plainly: “retrieval-grounded Q&A over the same PRSV knowledge base the main analysis pipeline uses, plus the ability to run a photo straight through the existing analysis pipeline and explain the result conversationally. No external LLM call - answers are composed from retrieved knowledge chunks, consistent with how the rest of this app already works (kept deliberately consistent rather than mixing a templated system with an LLM-backed chat feature that behaves differently).” This is worth underscoring precisely because Chapter 21 showed that an LLM generation path *does* exist elsewhere (`rag/llm_generator.py`, used by `RagService` for the per-image farmer-friendly explanation) — `ChatbotService` deliberately does *not* call it, keeping chat answers fully template/retrieval-based regardless of whether `ANTHROPIC_API_KEY` is configured. The constructor builds its own independent `HybridRetriever` (or `LocalTfidfRetriever` fallback) over the same 30-chunk KB, its own `CacheService`, and its own `RunManager/AnalysisService/BatchService` chain (reused for photo-in-chat analysis) — all separate instances from whatever `RagService` or the `/api/analysis` routes construct, per-request, elsewhere in the app; as noted in Chapter 7, this one service instance is built once per process and reused (via `api_chatbot.py`’s module-level singleton), unlike the analysis routes’ per-request service construction.

`handle_message(text, language)` — **full trace**

1. **Language validation**: an unsupported `language` code silently falls back to `DEFAULT_LANGUAGE` (“en”) rather than erroring.
2. **NLP analysis of the incoming message** (v3.1, real, not stubbed): `classify_intent(text)` (empty text short-circuits to `Intent.QUESTION` without calling the classifier), `assess_urgency(text)`, and `extract_symptom_findings(text)` (only if `text` is non-empty) are all computed *before* branching on what to do — every response includes `intent` and `urgency.level` in its payload, regardless of which reply path is taken.
3. **Empty message**: a fixed English prompt (“Please type a question, or attach a leaf photo to check it.”) is used as `reply_en` and falls through to the shared translation step at the end.
4. **Greeting**: triggered by *either* the NLP classifier returning `Intent.GREETING` *or* the original, still-present `_GREETING_PATTERN` regex matching `(^(\hi|hello|hey|namaste|namaskar|vanakkam|help)\b,`

case insensitive) — the v3.1 classifier was added *alongside* the original regex rather than replacing it, so either signal alone is sufficient. On a greeting, the function returns **immediately** (a distinct early **return**, not falling through to the shared translation-and-response-assembly code at the bottom) with a phrasebook-sourced reply (`get_phrase("greeting", language)`) — notably `used_live_translation` is hardcoded `True` here with the comment “phrasebook is pre-translated, always ‘available’” — a deliberate semantic choice that “translation succeeded” for a phrasebook lookup means “a pre-translated string exists for this language,” not “a live translation API call succeeded.”

5. **Ordinary question:** `_to_english` translates the incoming text to English (a no-op returning the text unchanged, with `translation_ok=True`, if `language` is already “en”); `_compose_answer_from_chunks` (below) produces the English answer; if `urgency.level == "urgent"`, a fixed referral sentence (“This sounds serious, so please prioritize contacting a local agricultural extension officer as soon as possible. ”) is **prepended** to the answer — this is the concrete, observable effect of the urgency-detection module actually changing chatbot output, not just being logged.
6. **Shared finishing step** (for both the empty-message and ordinary-question paths, but *not* the greeting path, which already returned): `_from_english` translates the English reply back to the target language; if that translation itself failed (`translation_ok_out=False`), a phrasebook “translation_unavailable” note is **appended** to the reply text, so the user sees an explicit, human-readable notice rather than silently receiving English text when they asked in another language. The final response dict includes `reply` (translated, with any note appended), `reply_english` (always the original English, for debugging/logging/consumers that want the untranslated text), `language`, `used_live_translation`, `source` (“knowledge_base” for this path), `intent`, `urgency`, and `symptom_findings` (a nested {`symptoms`, `plant_parts`} dict, or `None` if extraction wasn’t run).

`_compose_answer_from_chunks` — the actual answer composition

Cached (via `CacheService`) on the normalized, lowercased English query text — the code comment notes this cache key is intentionally scoped to the *English* query, “translation happens outside this method, so the cached English answer is reused regardless of which language the farmer originally typed in,” i.e. a Hindi and a Tamil farmer asking the semantically-translated-equivalent question share one cache entry. Retrieval returns the top 3 chunks; results are filtered to `audience == "farmer"` chunks specifically (falling back to the unfiltered set if no farmer-audience chunk was retrieved) — since the KB also contains “technical”-audience chunks not meant for direct farmer consumption. If nothing at all was retrieved, a fixed “I don’t have specific information on that yet ...” message is returned. Otherwise, the answer is composed from the **best-matching chunk’s text**, with the **second-best chunk’s text appended** only if it isn’t already a substring of the first (if `farmer_chunks[1].text` not in `primary`) — avoiding a redundant answer when two retrieved chunks happen to overlap heavily.

`handle_photo(image_path, language)` — photo-in-chat

Routes the uploaded image through **the exact same** `batch_service.analyze_images([image_path])` call used by every other entry point in the system (Chapter 10) — a chatbot photo submission produces a real, full analysis run with its own `run_id`, output directory, and persisted JSON/image artifacts, not a lightweight “chat-only” analysis path. If analysis fails (empty results), a translated error message is returned with `source: "photo_analysis_error"`. On success, the response is composed from a phrasebook intro (“result_healthy” or “result_diseased”, selected by the actual prediction), the display-formatted prediction, confidence percentage, severity label/score, **the same farmer_friendly_explanation** the main analysis pipeline’s RAG service already generated for this image (not a second, separate generation), and up to the first 3 advi-

sory notes — translated as one combined English string, then translated to the target language as a single unit (not sentence-by-sentence), with the same “translation unavailable” fallback note pattern as `handle_message`. The response additionally carries `prediction`, `confidence`, `severity_label`, `severity_score`, `image_id`, and `run_id` — meaning a chat client can deep-link back to the full `/run/{run_id}` web page for the complete visual result (heatmaps, SHAP/LIME breakdowns, etc.), not just the chat-summarized text.

23.2 `app/services/translation_service.py`

`SUPPORTED_LANGUAGES` is a fixed dict of exactly **10 languages**: English plus 9 Indian languages — Hindi, Telugu, Tamil, Kannada, Malayalam, Bengali, Marathi, Gujarati, Punjabi — each entry carrying its English name, native-script name, and a BCP-47 locale string for the Web Speech API (e.g. `"hi-IN"`, `"te-IN"`), confirming this is exactly the “9 Indian languages” referenced throughout this project’s documentation and consistent with its stated target audience of Indian papaya farmers. `translate_text(text, target_lang, source_lang="en")` uses `deep_translator.GoogleTranslator` — a free, keyless wrapper around Google’s public translate endpoint. Its docstring is explicit about the failure mode: “This requires outbound internet access at runtime; if the request fails (offline, blocked network, rate-limited, etc.), the original text is returned unchanged with `used_live_translation=False` so the chatbot degrades gracefully instead of erroring out.” `get_phrase(key, lang)` (a separate function, not shown in full here) backs the phrasebook fallback used for greetings and the translation-unavailable notice — these are pre-translated fixed strings for a small, known set of phrase keys (`"greeting"`, `"result_healthy"`, `"result_diseased"`, `"translation_unavailable"`), available even with zero network access, which is precisely why the greeting path in `handle_message` can claim `used_live_translation=True` with no actual API call: the phrase was translated once, by a human, ahead of time, and simply looked up.

23.3 `nlp/` — The Four NLP Modules, Each With an Explicit Honesty Note

Every one of the four v3.1 NLP modules opens with a docstring explicitly distinguishing what it implements from what a fully-trained model would be, and stating why the simpler approach was chosen. This pattern is consistent enough across the module that it is worth presenting together.

`nlp/intent_classifier.py` `classify_intent(text)` returns an `Intent` enum (`GREETING`, `THANKS`, `PHOTO_REQUEST`, `COMPLAINT_URGENT`, `QUESTION`). Rather than fine-tuning DistilBERT or using zero-shot BART-MNLI (the wishlist’s suggestions, both requiring a large `transformers` model download), it reuses the *same* small multilingual embedding model already loaded for dense retrieval (`EMBEDDING_MODEL_NAME` from `rag/embeddings_retriever.py`): each intent is defined by a handful of example utterances, embedded once into per-intent centroids (`_get_centroids`), and an incoming message is classified by nearest centroid in embedding space — “a real, standard zero-shot technique (embedding-based nearest-centroid classification) - not as strong as a purpose-built NLI model, but it needs zero additional model downloads beyond what v3.0 already requires.” `_keyword_fallback` is used if the embedding model is unavailable.

`nlp/symptom_ner.py` `extract_symptom_findings(text)` returns `SymptomFindings(symptoms, plant_parts)` via dictionary/pattern matching against `_SYMPTOM_VOCAB` (e.g. “yellow spots”, “mosaic”, “ring spot”, “chlorosis”, “curling”, “wilting”, “stunted”) and a corresponding plant-part vocabulary, both sourced from the same KB tag vocabulary used elsewhere. Docstring: “This is a legitimate, commonly-used *rule-based* NER approach - a defensible and fully honest first step - but

it is not a statistical model and won't generalize to symptom descriptions using vocabulary outside its dictionary the way a trained NER model would."

`nlp/urgency_detection.py` `assess_urgency(text)` returns `UrgencyAssessment(level, score, matched_signals)` via a fixed `_URGENT_KEYWORDS` set ("whole field", "everything is dying", "emergency", "desperate", "ruined", etc.) plus a `_DISTRESS_PUNCTUATION_SIGNAL` tuple ("!!!", "???", "please help"). Docstring: "this is a keyword + heuristic scorer, not a trained sentiment model It's a legitimate, transparent first pass - each signal is inspectable - but will miss distress phrased outside its keyword list."

`nlp/dialogue_state.py` `DialogueState` / `DialogueStateStore` implement session-scoped memory (a `Turn` history, a remembered "current topic") and `resolve_coreference(text)`, replacing pronouns like "it"/"that plant"/"the same one" (`_COREFERENCE_PRONOUNS`) with the tracked `last_topic`. Its docstring states the integration gap directly and without euphemism: "`chatbot_service.py` currently treats every message independently (each `handle_message()` call is stateless). This module provides the session-scoped memory structure; wiring it into `chatbot_service.py`'s request handling is the next integration step (it needs a `session_id` concept in the API layer, which is a routing change, not just a service-layer addition - noted here rather than silently left out)." Confirmed by direct inspection of `app/routes/api_chatbot.py` and `chatbot_service.py`: this is accurate — `ChatMessageRequest` carries no session identifier of any kind, and `handle_message/handle_photo` take no session argument, so every chat turn in the live application is genuinely independent of every other turn, exactly as the module's own docstring says.

Part V

Evaluation Rigor, MLOps, and Sustainability

Chapter 24

Statistical Evaluation and Evaluation Rigor Modules

None of the seven modules in `evaluation/` is imported by the live web application; all are standalone analysis tools invoked directly (by a script or by a developer) against trained models and feature data.

24.1 `evaluation/statistical_tests.py`

`mcnemar_test` — comparing two classifiers correctly

The docstring states precisely what this test answers and why it is the right one: “Tests whether classifier A and classifier B disagree in a systematically different way (not whether one is more accurate overall) - the correct test for ‘is this new model meaningfully different from the old one’, rather than eyeballing a difference in accuracy percentages.” It builds a 2×2 contingency table of where the two classifiers agree or disagree with each other relative to ground truth (n_{00} : both wrong, n_{01} : A wrong/B right, n_{10} : A right/B wrong, n_{11} : both right), and branches on the number of **discordant** pairs ($n_{01} + n_{10}$): if zero, it returns a fixed result (statistic 0, p-value 1, not significant) with the interpretation “Classifiers made identical correctness patterns on every test sample”; if fewer than 25 (the standard small-sample threshold for McNemar’s test), it uses `scipy.stats.binomtest` (an exact binomial test) rather than the chi-square approximation, which is unreliable with small discordant counts; otherwise it uses the continuity-corrected chi-square statistic $\frac{(|n_{01}-n_{10}|-1)^2}{n_{01}+n_{10}}$. A human-readable `interpretation` string is generated either way, stating which model corrected more of the other’s errors if the result is significant. This is exactly the test that produced the real, recorded null result described in Chapter 17 for the ensemble-vs-baseline comparison.

`calibration_report` — is confidence trustworthy?

Computes the Brier score (`sklearn.metrics.brier_score_loss`, a standard proper scoring rule for probabilistic predictions) and a reliability diagram (`sklearn.calibration.calibration_curve` with `strategy="quantile"` binning, meaning bins have roughly equal sample counts rather than equal probability-width, more robust to unevenly distributed predicted probabilities). If `output_dir` is given, a matplotlib plot is saved comparing the model’s actual calibration curve against the diagonal “perfectly calibrated” reference line.

`compare_models_report`

A convenience wrapper bundling McNemar’s test plus per-model calibration reports for two named models (default names `"baseline_svm"` and `"stacking_ensemble"`) — exactly what `ml/train_ensemble.py` calls to produce `models/evaluation/ensemble_comparison.json`.

24.2 `evaluation/holdout_eval.py` — Honest-by-Design

The docstring identifies this as addressing what was “repeatedly flagged across the upgrade list as ‘the single highest-value validation upgrade’: the model’s reported accuracy so far comes from a random train/test split of one dataset (same cameras, same sessions), which overstates real-world generalization.” `evaluate_holdout(holdout_csv, model_dir, in_distribution_metrics_path)` is designed to compare the shipped model’s performance against a *second*, genuinely independent-source feature CSV. Its very first behavior is the one worth documenting most carefully, because it was confirmed by an actual run during this project’s own v3.1 development: **when no such holdout CSV exists**, it does not error, does not fabricate a plausible-looking result, and does not silently skip — it returns a structured, honest status:

```

1 {
2   "status": "no_independent_source_data",
3   "message": "No file found at ... This is expected until a second "
4             "batch of images from a different camera/region/session has been "
5             "collected ... Reported model accuracy until then reflects only "
6             "same-distribution performance and should be captioned as such."
7 }
```

This exact behavior — a real, recorded, non-error early return — is the clearest concrete example in the codebase of the “say so explicitly rather than fabricate” principle this documentation itself follows. **No independent-source holdout CSV exists anywhere in this project’s shipped data/ directory**, so this module has never progressed past this early-return status in this project’s actual history — confirmed by inspection: only `data/training_features.csv` and `data/demo_manifest.csv` exist, neither of which is a second, independently-sourced dataset. When a holdout CSV *is* provided, the function loads the persisted model/scaler/label-encoder, computes the same metric suite as `ml/evaluate.py`, and — if an in-distribution metrics file is also given — computes a `generalization_gap` per metric (in-distribution minus holdout), with the interpretation explicitly stating the expected direction: “Positive gap values indicate the model performs worse on the independent-source holdout than on the original same-distribution test split - the expected direction if any overfitting ... occurred.”

24.3 `evaluation/stress_testing.py`

`run_stress_test(image_paths, predict_fn, read_fn)` applies four independent, deliberately damaging perturbations to a sample of real images and measures how much the model’s predictions change: **blur** (`cv2.GaussianBlur`, kernel size 9), **brightness shift** (a fixed -60 delta, simulating underexposure), **JPEG compression** (quality 15, aggressive compression artifacts), and **partial occlusion** (a random rectangular block covering 25% of the image, simulating a leaf partly out of frame or blocked by a hand/leaf overlap). For each perturbation, `StressTestResult` records `prediction_flip_rate` (fraction of images whose predicted class changed after perturbation) and `mean_confidence_drop` — a direct, quantitative characterization of failure modes under realistic photography conditions, rather than reporting only best-case clean-image accuracy.

24.4 `evaluation/fairness_audit.py`

Docstring’s honesty note states the exact blocker directly: “this project’s shipped feature CSV (`data/training_features.csv`) doesn’t currently have a subgroup/source column - the demo dataset’s provenance metadata wasn’t tracked per-image. This module is real, functioning infrastructure that computes a genuine per-subgroup breakdown the moment such a column exists (it’s a one-column addition to the feature extraction manifest, see `scripts/prepare_manifest.py`); until then, calling it with a synthetic or absent subgroup column will report that honestly rather than fabricate a ‘no bias found’ result.” `audit_by_subgroup` computes `compute_binary_metrics` independently per distinct subgroup value; `largest_subgroup_gap` identifies which two subgroups differ most on a chosen metric. This is the same missing per-image provenance metadata gap cross-referenced by `ml/federated_learning.py`’s docstring in Chapter 18 — two separate v3.1/v3.2 modules independently point at the same real, unaddressed data-collection gap.

24.5 `evaluation/trust_scores.py` — Jiang et al. (2018)

`TrustScorer`, backed by `sklearn.neighbors.NearestNeighbors`, implements the specific, named “trust score” formulation from Jiang et al., 2018 (“To Trust Or Not To Trust A Classifier”), cited directly in the module docstring: for a given prediction, compare the distance to the nearest *same*-predicted-class training point against the distance to the nearest *different*-class training point. A ratio greater than 1 means the point sits closer to a different class than to its own predicted class — “a red flag regardless of what `predict_proba` says.” This directly targets the failure mode where a model is confidently wrong because it is extrapolating into a sparsely-represented region of feature space rather than interpolating within well-represented training data — a distinct concern from calibration (which asks “is the stated confidence numerically accurate on average”) and from SHAP/LIME (which explain *why* a prediction was made, not whether the input region is well-represented at all).

24.6 `evaluation/pdp_ice.py`

`compute_pdp_ice` and `compute_pdp_for_all_features` wrap `sklearn.inspection.partial_dependence` with `kind="both"` — returning both the averaged Partial Dependence curve (how the average prediction changes as one feature varies, holding others at their observed values) and per-sample Individual Conditional Expectation curves (the same, but for each individual training instance separately, revealing whether the averaged PDP curve is hiding heterogeneous per-instance behavior). This complements SHAP/LIME’s single-instance, single-prediction explanations with a view of model behavior across a feature’s *entire* observed range, as the docstring states directly.

24.7 `evaluation/shap_interactions.py`

The docstring explains a real technical constraint of the SHAP library itself, not a limitation this project introduced: “SHAP’s exact interaction-value computation (`shap_interaction_values`) is only implemented for `TreeExplainer` in the `shap` library, not for the `KernelExplainer` used elsewhere in this project for the SVM This module therefore runs interaction values against the tree-based members of the v3.1 stacking ensemble (`ml/ensemble.py`’s Random Forest / Gradient Boosting), which is the correct, supported way to get this feature.” `compute_shap_interactions(tree_model, X, feature_names, top_k_pairs=10)` identifies which *pairs* of features jointly drive predictions (the docstring’s example: “does high entropy only matter when `green_ratio` is also low?”), something a per-feature-alone SHAP attribution cannot reveal. This is the exact module where Error 3 (Chapter 1, Problem Solving) was found and fixed — the installed `shap` version’s

`TreeExplainer(...)` `.shap_interaction_values(X)` returned a single 4D array (samples, features, features, class) rather than the list-of-per-class-arrays shape the code originally assumed, causing a scalar-cast crash; the fix added explicit handling for both possible shapes, and `tests/test_shap_interactions.py` was written afterward as a regression guard.

Chapter 25

MLOps: Experiment Tracking, Model Registry, Drift Detection, ONNX Export

25.1 `mlops/experiment_tracking.py` — MLflow Wrapper

`tracked_run(settings, run_name)` is a context manager used by both `ml/train_svm.py` and `ml/train_ensemble.py`. `is_available()` checks whether `mlflow` can be imported at all; if not, `tracked_run` yields `None` and exits — the training script’s own logging calls (`log_params`, `log_metrics`, `log_artifact`, `log_sklearn_model`, each independently guarded by the same `is_available()` check) become complete no-ops. When available, it optionally sets the tracking URI from `settings.mlflow_tracking_uri` (blank means MLflow’s own default, a local `./mlruns` folder needing no server) and sets the experiment name from `settings.mlflow_experiment_name` (default “prsv-analyzer”). The docstring clarifies a subtlety worth restating precisely: “‘disabled’ here really only guards against `mlflow` not being installed” — i.e. `mlflow_tracking_uri` being blank does *not* disable tracking outright; it just means MLflow’s own local-folder default is used instead of a remote server. `log_params` additionally stringifies any non-primitive parameter value (lists, nested dicts from a hyperparameter grid) before logging, since “MLflow rejects non-primitive values” and the comment states this is done “rather than crashing a training run over a logging call.”

25.2 `mlops/model_registry.py` — `ModelRegistry`

A self-contained, dependency-free (no MLflow, no database) versioned model registry built directly on the filesystem plus a JSON manifest. The docstring frames its scope honestly against what a larger system would use: “a production system at real scale would use MLflow’s own Model Registry or a dedicated service. This JSON-file-based version is appropriate for this project’s single-machine deployment target (consistent with the existing SQLite-over-Postgres, in-process-queue-over-Celery choices already made elsewhere in this codebase) and is fully functional - not a stub - for that scale.” This is a real, explicit statement tying together three separate architectural decisions across the codebase (the `RunStore` SQLite index, the `BatchJobQueue` in-process executor, and this registry) under one consistent “lightweight infrastructure appropriate to a single-machine deployment” philosophy.

Structure

`registry_dir/registry_manifest.json` holds a flat JSON list of `ModelVersion` records (`version_id`, `model_filename`, `registered_at`, `metrics`, `notes`, `is_active`); `registry_dir/versions/` holds the

actual copied model binary files, each renamed with its version ID as a prefix. `register(model_path, metrics, notes)` generates a microsecond-resolution timestamp plus a 6-character UUID suffix as the version ID (`f"v_{timestamp}_{unique_suffix}"`) — **this exact microsecond-plus-UUID scheme is the fix for Error 2** described in Chapter 1’s Problem Solving history: the original second-resolution timestamp collided when two registrations happened within the same second, corrupting rollback targeting, caught by `tests/test_mlops_additions.py::test_model_registry_rollback` failing. `register` copies the model file into `versions/`, deactivates every existing version (`v["is_active"] = False` for all), appends the new version marked active, and saves the manifest. `rollback(version_id, deploy_to)` marks the target version active, deactivates all others, copies its artifact to `deploy_to` (typically the live `models/svm_model.joblib` path), and raises `ValueError` if the version ID is not found. `champion_challenger_compare(challenger_metrics, primary_metric="accuracy")` compares a candidate model’s metric against the currently active version’s recorded metric and returns a `{"recommendation": "promote"|"hold", "reason": ...}` dict — the docstring is explicit that this is only “the decision-support half of a champion/challenger deployment pattern; the traffic-splitting half needs a live deployment with real request routing to be meaningful, which is noted as the integration point rather than faked here.”

25.3 `mlops/drift_detection.py`

The docstring explains a deliberate library-avoidance choice: “the wishlist names **evidently** or **whylogs**. Both are real, well-built libraries, but pull in a fair amount of their own dependency tree for what is, at its statistical core, two well-established tests: Population Stability Index (PSI) and the Kolmogorov-Smirnov two-sample test. Implementing those directly here (a few dozen lines of `numpy/scipy`) gives the same statistical rigor without the extra dependency weight.” `population_stability_index(reference, current, n_bins=10)` computes the standard PSI formula (binning both distributions into the same 10 bins and summing $(\text{current}\% - \text{reference}\%) \times \ln(\text{current}\% / \text{reference}\%)$ across bins) — `_severity_from_psi` then buckets the resulting PSI value into the industry-standard interpretation thresholds (a PSI below roughly 0.1 indicates no significant shift, 0.1–0.25 indicates moderate shift, above 0.25 indicates major shift — this project’s exact numeric cutoffs are implemented in `_severity_from_psi`). `detect_feature_drift` computes both PSI and `scipy.stats.ks_2samp` (a nonparametric two-sample distribution comparison test, giving a KS statistic and p-value) per feature, and `summarize_drift` aggregates a list of per-feature `FeatureDriftResults` into an overall drift summary. Like the other MLOps modules, this operates on two feature arrays supplied by the caller (a reference/training distribution and a current/live distribution) — it is not wired into any live request path, since the application does not currently persist a rolling window of live inference feature vectors for this module to compare against; it is invocable infrastructure, exercised in `tests/test_mlops_additions.py` against synthetic reference/ current arrays.

25.4 `mlops/onnx_export.py` — Verified, Executed

The docstring pinpoints exactly which part of the earlier “needs a mobile app” roadmap blocker this actually closes: “the earlier roadmap noted ‘needs a target mobile app to embed the runtime in,’ but the *model conversion* itself needs no mobile app at all - it needs no phone, no Android/iOS project.” `export_pipeline_to_onnx(scaler_path, model_path, output_path, n_features)` wraps the persisted `StandardScaler` and `SVC` into a single `sklearn.pipeline.Pipeline`, then converts the whole pipeline (not just the classifier) to ONNX via `skl2onnx.convert_sklearn` — the comment explains why scaling is baked into the same exported graph: “a mobile app just feeds in raw 12-dim feature vectors and gets predictions back - no separate scaler step to reimplement in Kotlin/Swift.” The `options={id(model): {"zipmap": False}}` argument requests plain array probability output rather than `skl2onnx`’s default dict-like “zipmap” output, again for

easier on-device consumption. `verify_onnx_matches_sklearn(onnx_path, scaler_path, model_path, X_test, tolerance=1e-4)` is the numerical proof step: it runs the *same* test vectors through both the original scikit-learn pipeline and an `onnxruntime.InferenceSession` over the exported graph, and checks `sklearn_predictions` exactly match `onnx_predictions` (`np.array_equal`) *and* the maximum absolute difference in predicted probabilities is below `tolerance` (10^{-4}).

This was actually executed against the real shipped `svm_model.joblib/scaler.joblib` and 50 real rows sampled from `data/training_features.csv`, producing an ONNX file of **8,974 bytes** with `predictions_match=True` and `max_probability_diff` $\approx 3.6 \times 10^{-7}$ — six orders of magnitude tighter than the 10^{-4} tolerance, i.e. the ONNX export is numerically indistinguishable from the original scikit-learn model at the precision that matters for a real diagnostic decision.

25.5 `sustainability/energy_benchmark.py`

Docstring: “formally measures the SVM pipeline’s inference cost, turning the project’s ‘lightweight by design’ choice into a measurable, citable number rather than an assumed advantage.” `_try_codecarbon_measure` attempts a real joule measurement via the optional `codecarbon` package, which itself needs OS-/hardware power-draw access (Intel RAPL, NVIDIA-SMI, etc.) not available in every environment; the honesty note is explicit that this “aren’t available in every environment (notably: sandboxed containers, some cloud VMs).” `benchmark_inference_cost` falls back to “a CPU-time-based proxy (wall-clock inference time x number of CPU cores used), which is a standard, defensible stand-in used throughout the Green-AI literature when direct power measurement isn’t accessible - not pretending to be the same precision as a direct joule measurement, and labeled as such in the output.” `compare_models_energy` compares the recorded `EnergyBenchmarkResults` for multiple models (e.g. the baseline SVM vs. the stacking ensemble) side by side — directly answering whether the added complexity of the v3.0 ensemble (three base learners instead of one) comes with a measurably higher inference cost, a question this project’s own Green-AI framing (referenced by `xai/lime_explainer.py`’s docstring in Chapter 17) explicitly cares about.

Part VI

The Frontend

Chapter 26

Server-Rendered Pages: `app/routes/pages.py` and Templates

26.1 No JavaScript Framework — By Architecture, Not Omission

As stated in Chapter 2, the entire user interface is server-rendered Jinja2 HTML (`app/templates/`) plus hand-written vanilla JavaScript (`app/static/js/`) for client-side interactivity — there is no React/Vue/Angular/Svelte anywhere in the project, and no build step (webpack, Vite, esbuild) transforms the frontend assets; the browser loads `style.css`, `chatbot.css`, and the JS files directly as static files, with one external script loaded from a CDN in `base.html`: `Chart.js` 4.4.4 from `cdnjs.cloudflare.com` (used for the run-history dashboard’s charts, in `dashboard.js`).

26.2 `app/routes/pages.py` — The HTML Page Router

Twelve routes: eight `GET` routes rendering a template, four `POST` routes that are HTML-form equivalents of the JSON `/api/analysis/*` endpoints, returning a `RedirectResponse` to the resulting run’s detail page rather than a JSON body — so the browser’s native form submission flow (no JavaScript `fetch` required) still works end-to-end.

Method	Path	Template	Behavior
GET	/	<code>home.html</code>	Landing page: app name, count of demo dataset images, the 5 most recent runs (<code>_list_runs(limit=5)</code>).
GET	<code>/analyze</code>	<code>analyze.html</code>	The upload page; accepts optional <code>message/level</code> query params so a redirect from a failed form POST can surface an error/status message on reload.

Method	Path	Template	Behavior
GET	/status	status.html	An operational status/ diagnostics page: demo dataset availability, model availability, KB availability, writable-directory checks, the 5 most recent runs, <i>and</i> security posture — <code>api_key_enabled</code> , <code>rate_limit_requests</code> , <code>rate_limit_window_seconds</code> , <code>output_retention_days</code> — i.e. this page surfaces whether API-key protection is currently on, directly from <code>settings</code> .
GET	/methodology	methodology.html	A static explanation page (only <code>app_name</code> passed as context) describing the system’s approach — the human-facing counterpart to the technical README.
GET	/results	result.html	The searchable/filterable run list: <code>search</code> (substring match on <code>run_id</code>), <code>status_filter</code> (healthy/diseased/mixed, via <code>_filter_runs</code>), <code>severity_filter</code> , <code>sort_by</code> (newest/oldest). Note this filtering happens by scanning <i>every</i> run folder’s <code>batch_summary.json</code> (<code>_list_runs()</code> with no limit) and filtering in Python, in contrast to <code>/api/analysis/runs/search</code> ’s SQLite-backed server-side filtering (Chapter 10) — these are two independent implementations of similar run-listing/filtering logic , one filesystem-scan-based (this page) and one SQLite-based (the JSON API route), which could in principle diverge if a run’s <code>batch_summary.json</code> and its RunStore SQLite row ever fell out of sync.
GET	/batch-results	batch_result.html	All runs, no filtering.
GET	/run/{run_id}	run_detail.html	404s if <code>batch_summary.json</code> doesn’t exist; otherwise renders the full batch summary for one run. Note: unlike <code>api_analysis.py</code> ’s <code>_resolve_run_dir</code> , this page route does not apply the regex/containment validation described in Chapter 7 — it builds the path directly as <code>settings.output_dir / run_id / "batch_summary.json"</code> and relies only on the file existing or not. This is a real, observable difference in validation rigor between the JSON API and the HTML page router for structurally the same operation, discussed further in Chapter 27.
GET	/run/{run_id}/image/{image_id}	image_detail.html	Same pattern (direct path construction, no regex/containment check); loads all five persisted JSON files for that image plus a <code>paths</code> dict of every file in that image’s output directory.

Method	Path	Template	Behavior
POST	/analyze/single	—	Form equivalent of POST <code>/api/analysis/single;</code> redirects to <code>/run/{run_id}/image/{image_id}</code> on success, or back to <code>/analyze?message=...&level=error</code> on failure.
POST	/analyze/multiple	—	Redirects to <code>/run/{run_id}</code> .
POST	/analyze/zip	—	Extracts and analyzes, redirects to <code>/run/{run_id}</code> .
POST	/analyze/demo	—	Runs the demo dataset, redirects to <code>/run/{run_id}</code> .

26.3 Shared Helpers in `pages.py`

`_list_runs(limit=None)` scans `settings.output_dir` exactly like `/api/analysis/runs`, reading each run’s `batch_summary.json` if present (silently defaulting `processed_images/failed_images` to 0 on a JSON read error, rather than excluding the run entirely). `_filter_runs(runs, search, status_filter, severity_filter, sort_by)` applies each filter as an independent, chainable list comprehension: `status_filter == "healthy"` keeps only runs with `healthy_count > 0` *and* `diseased_count = 0` (a run must be *entirely* healthy, not merely contain a healthy image, to match); `"diseased"` keeps any run with at least one diseased image; `"mixed"` requires both counts positive. `severity_filter` checks whether the given label key exists in the run’s `severity_distribution` dict at all (any count, including a count of the filter label being present at all, not a minimum threshold). `_build_services()` mirrors `api_analysis.py`’s identically-named helper — both files independently construct a fresh `RunManager/AnalysisService/BatchService` triple per request, confirming this “rebuild services per call” pattern is used consistently across both the JSON API and the HTML form-submission routes, not just one of them.

26.4 Template Structure

`templates = Jinja2Templates(directory=ROOT_DIR/"app"/"templates")` is constructed once at module import time. `base.html` is the shared layout every other template extends (via Jinja2’s `{% extends "base.html" %}` / `{% block ... %}` mechanism): it defines the `<head>` (favicons, PWA icon, the `style.css/chatbot.css` stylesheets, the Chart.js CDN script), an inline, non-deferred, pre-paint dark-mode script (reading a `"prsv-theme"` key from `localStorage` and setting a `data-theme` attribute *before* first paint, specifically to avoid a flash of the wrong theme on page load), the site header/navigation, and — via `partials/chatbot_widget.html`, included in the base layout — the chatbot widget, which is therefore present and available on *every* page of the site, not just a dedicated chat page. The nine other page templates (`home.html`, `analyze.html`, `result.html`, `run_detail.html`, `image_detail.html` — the largest template at 401 lines, since it renders every heatmap image, feature value, SHAP/LIME breakdown, and RAG explanation for one image — `batch_result.html`, `status.html`, `methodology.html`, `error.html`) each extend `base.html` and fill in the page-specific body content.

26.5 Client-Side JavaScript

`app.js` (254 lines) Site-wide behavior shared across every page: the dark/light theme toggle (writing back to the same `"prsv-theme"` `localStorage` key `base.html`’s inline script reads), nav-

igation/menu interactivity, and other cross-page UI wiring.

upload.js (365 lines) Drag-and-drop file upload handling for the `/analyze` page: `ALLOWED_TYPES` (a fixed MIME-type allowlist mirroring, but independently maintained from, the server-side `settings.allowed_extensions`), `MAX_FILE_MB = 25` (matching `settings.max_upload_size_mb`'s default), client-side validation (`isValidImageFile`) that rejects empty or oversized files and unrecognized MIME types *before* they are ever submitted to the server — a genuine client-side pre-check, though (as Chapter 27 discusses) not a substitute for the server's own validation, since a client-side check is trivially bypassable. `setupDropzone` wires up a drag-and-drop target with a live thumbnail preview grid.

analyze.js (1 line) Contains only `console.log("PRSV analyze module initialized.");` — a near-empty placeholder file. All of the `/analyze` page's actual interactive behavior (drag-and-drop, validation, previews) lives in `upload.js` instead; `analyze.js` is loaded (confirmed by a `<script>` tag in `analyze.html`) but contributes no real functionality beyond a one-line startup log message. *Whether this file was scaffolding for logic that was later moved into `upload.js` and never cleaned up, or intentionally kept as a minimal page-load marker, is not verifiable from the available project files.*

dashboard.js (262 lines) Builds the Chart.js visualizations for run-history/dashboard views — consuming the same `severity_distribution/healthy_count/diseased_count` data structures already described in Chapter 10, rendered client-side rather than as the static matplotlib PNGs `export_service.py` generates server-side (the two chart-generation approaches — server-side matplotlib PNG and client-side Chart.js — coexist in this project for different purposes: the matplotlib PNGs are permanent, embeddable, downloadable artifacts saved into a run's own folder and PDF reports; the Chart.js charts are live, interactive, in-browser visualizations that never touch disk).

chatbot.js (242 lines) Drives the chat widget included on every page: sending messages to `POST /api/chatbot/message`, handling photo uploads to `POST /api/chatbot/analyze-image`, language selection (populated from `GET /api/chatbot/languages`), and — per the project's stated “voice input/output” feature — the browser's native Web Speech API, using the BCP-47 locale strings (“hi-IN”, “te-IN”, etc.) that `translation_service.py`'s `SUPPORTED_LANGUAGES` table supplies precisely for this purpose.

26.6 CSS

style.css (1,237 lines) is the primary site stylesheet, defining the design system (colors, typography, layout, dark/light theme variables consumed by the `data-theme` attribute set in `base.html`'s inline script) for every page. **chatbot.css (246 lines)** is a dedicated, separate stylesheet scoped to the chat widget — kept apart from `style.css` because the widget is a shared, injected component present on every page (via the base-layout partial) rather than a page-specific element.

Part VII

Security, Testing, and Operations

Chapter 27

Security Documentation

This chapter follows the same discipline as the rest of this document: every mechanism described here is either **implemented** (with the exact module and behavior cited), **partially implemented**, or **absent** — stated plainly, without implying protection that does not exist in the shipped code.

27.1 Implemented

Optional API-key gating `ApiKeyAndRateLimitMiddleware` (Chapter 6) checks `X-API-Key` against `settings.api_key` for every `/api/*` route, when `API_KEY` is set. **Caveat:** the comparison is a plain Python `!=` string comparison, not a constant-time comparison (e.g. `hmac.compare_digest`); a timing side-channel that could theoretically help an attacker guess the key character-by-character is not ruled out by the current implementation, though this is a fairly low-severity concern for a research/demo deployment on typical network latency.

Rate limiting A sliding-window limiter (default 60 requests/60s per client IP) on `/api/*` routes, explicitly documented as single-process, in-memory, non-distributed.

Path traversal protection (run/image IDs) `_resolve_run_dir` and `_resolve_image_dir` in `api_analysis.py` combine a strict regex allowlist with a resolved-path containment check (Chapter 7).

Path traversal protection (ZIP extraction) `safe_extract_zip` (`app/utils/zip_utils.py`) sanitizes every member filename via `sanitize_filename` (stripping any directory components — `Path(member.filename).name` discards the archived path entirely, keeping only the base filename, which alone defeats a classic `../../etc/passwd`-style zip-slip payload), generates a collision-safe unique target path via `ensure_unique_path`, and then *independently re-verifies* with `is_within_directory(extract_target_path)` before writing any bytes, raising `ValueError("Unsafe ZIP content detected.")` if that check ever fails — the same belt-and-braces double-check pattern used for run/image IDs.

Upload content-type/readability validation Every upload path checks the file extension against `settings.allowed_extensions` (`is_allowed_extension`), rejects empty files (`validate_non_empty_file`), and — critically — **actually opens and verifies the file as a real image** via `validate_image_readable` (`PIL.Image.open(path).verify()`), not merely trusting the extension or a client-supplied MIME type. This means a file renamed from `malware.exe` to `malware.jpg` would pass the extension check but fail `validate_image_readable` and be rejected.

Debug mode off by default `settings.debug` defaults to `False`; `app/config.py`'s own comment states the reason directly (stack traces should not be returned to arbitrary callers).

Filename sanitization `sanitize_filename` (`app/utils/file_utils.py`) and `ensure_unique_path` are applied to every uploaded/extracted file before it touches the filesystem, independent of the ZIP-specific protections above.

27.2 Partially Implemented

Upload size limits `settings.max_upload_size_mb` (default 25) and `settings.max_zip_size_mb` (default 200) are declared, documented config fields, and a corresponding `validate_file_size(file_size_bytes, max_size_mb)` function exists in `app/utils/validation_utils.py`. **However, a direct codebase-wide search confirms neither `settings.max_upload_size_mb` nor `settings.max_zip_size_mb` is read anywhere outside its own field declaration, and `validate_file_size` is never called anywhere in the live application code** (only its own field/function definitions exist; no route, service, or utility function invokes it). The *only* enforcement of these limits anywhere in this project is **client-side**, in `app/static/js/upload.js`'s `MAX_FILE_MB = 25` constant and its `isValidImageFile` check — which runs in the user's browser before a request is sent, and is therefore trivially bypassable by anyone submitting a request directly (via `curl`, a script, or a modified/disabled client) rather than through the web UI's own JavaScript. **In the shipped v3.2 server code, there is no server-side enforced maximum upload or ZIP file size.** A sufficiently large upload would be constrained only by whatever limits FastAPI/Starlette/Uvicorn, the underlying OS, or an external reverse proxy (e.g. a platform like Render, mentioned in `render.yaml`) might impose — none of which are part of this project's own code. This is stated here plainly, as a verified fact about the shipped code, not a hypothesis.

Rate limiting scope Only `/api/*` routes are rate-limited; the HTML form-submission routes under `pages.py` (which perform the identical underlying analysis work, including the potentially expensive ZIP-extraction and batch-analysis paths) are **not** covered by `ApiKeyAndRateLimitMiddleware` at all, since that middleware's very first check exempts any path not starting with `/api/`. A client could bypass the JSON API's rate limit entirely by submitting the equivalent request through the HTML form endpoints (`POST /analyze/single`, etc.) instead.

Run/image ID path validation on HTML pages As noted in Chapter 26, `/run/{run_id}` and `/run/{run_id}/image/{image_id}` (the HTML page routes) construct filesystem paths directly from the URL path parameters without the `regex-allowlist-plus-containment-check` pattern the JSON API's equivalent routes apply. In practice this is a narrower gap than it might first appear, because `run_id/image_id` are used only to build a path under `settings.output_dir` and then checked for existence via `.exists()` — Python's `pathlib` `/` operator does not itself execute anything, and a path like `settings.output_dir / "../../etc/passwd"` would indeed resolve outside `output_dir`, but the resulting file would then need to parse as one of the five expected JSON filenames (`features.json`, etc.) or otherwise render meaningfully in the template to be useful to an attacker as anything beyond a 404 or a template error — still, this remains a real, structural inconsistency between the two routers' validation rigor for what is functionally the same operation, and is documented here rather than left unmentioned.

27.3 Not Implemented / Absent

User authentication / accounts There is no login system, no user accounts, no session/cookie-based authentication anywhere in the codebase. The single `API_KEY` mechanism is a shared secret for machine-to-machine `/api/*` access, not a multi-user auth system; every visitor to the HTML pages (which are never gated by the API key) sees the same, fully public interface with no per-user distinction.

Authorization / access control With no user concept, there is correspondingly no autho-

rization model — every run, every uploaded image, and every report is visible to anyone who can reach the server and knows (or discovers, e.g. via `/results`) a `run_id`.

HTTPS/TLS Not configured anywhere in this project’s own code; TLS termination, if used, would be the responsibility of whatever reverse proxy or hosting platform sits in front of the app (e.g. Render’s platform-level HTTPS, per `render.yaml`) — this is a standard and reasonable division of responsibility (an application server does not typically terminate TLS itself), stated here only to be explicit about scope.

Encryption at rest Uploaded images, analysis outputs, and the SQLite index are all stored as plain files/database rows on disk with no application-level encryption.

CSRF protection The HTML form-submission routes (`POST /analyze/single` etc.) have no CSRF token mechanism. Given that there is no authenticated session for a CSRF attack to ride on top of (per the “no authentication” point above), the practical impact of this absence is limited compared to a traditional session-authenticated application, but it is still a genuine absence worth recording precisely rather than glossing over.

Distributed rate limiting / DDoS protection Explicitly out of scope per the middleware’s own docstring and design (single-process, in-memory).

Part VIII

Testing, Installation, and Deployment

Chapter 28

Testing Documentation

28.1 Overview and Philosophy

The project ships a real, executable pytest suite: **125 test functions** across **50 test files** under `tests/`, confirmed by direct enumeration (`grep -c "def test_" tests/*.py`, summed). This is not a token or placeholder test directory — every module documented in Parts II through VI of this report that is reachable by an automated test has one, and the suite is run on every push via GitHub Actions (Section 28.5).

Two things are worth stating plainly, in keeping with this project’s own “honesty note” style (Section 27):

1. The `README.md` shipped with the project still says “40 tests as of v2.9” in its testing instructions. That count is stale — it describes an earlier release. The actual, current, v3.2 test count verified directly against the shipped `tests/` directory is 125. This report uses the verified 125, not the README’s outdated figure, and flags the discrepancy here so a reader who cross-checks the README is not confused by the mismatch.
2. Not every module documented in this report has a dedicated test file. Modules whose behavior is exercised only indirectly (e.g. through a route test that happens to call them) or that have no test coverage at all are called out explicitly below, rather than implying uniform coverage across the entire codebase.

28.2 Test Framework and Configuration

The suite runs on plain `pytest`, with a single project-level configuration file, `pytest.ini`:

```
1 [pytest]
2 markers =
3     slow: marks tests that take noticeably longer (e.g. spinning up a
4     Flower/Ray simulation) - still run by default, but can be excluded
5     with '-m "not slow"' for a fast local loop.
```

This registers exactly one custom marker, `slow`, which `pytest` would otherwise warn about as “unknown marker” under strict-marker settings. The marker is applied to the federated-learning simulation test(s) in `tests/test_federated_learning.py`, whose Flower/Ray simulation takes measurably longer to run than the rest of the suite (spinning up multiple simulated client processes, per Section 19.3). Critically, the marker’s docstring states the slow test is *still run by default* — it is opt-out, not opt-in. A developer who wants a fast local feedback loop excludes it explicitly:

```
1 pytest -q -m "not slow"
```

while ordinary `pytest -q` (as used in CI, see below) runs the full 125-test suite including the slow one.

28.3 Running the Suite Locally

From inside `prsv_project/` with the virtual environment activated (Section 29):

```
1 pytest -q
```

The `-q` (quiet) flag is the convention used consistently across this project’s own README, CI workflow, and (implicitly) its test authoring style — it produces a compact dot-per-test summary rather than a verbose per-test log, appropriate for a 125-test suite where a full verbose transcript would be unwieldy. A developer who wants to isolate a single file or test uses `pytest`’s ordinary node-id syntax, e.g. `pytest -q tests/test_severity.py` or `pytest -q tests/test_severity.py::test_severity_output_keys` (exact test names vary per file; see the per-file breakdown below).

28.4 Test File to Module Mapping

The table below maps every one of the 50 test files to the production module(s) it exercises and gives the exact test-function count in that file (verified by direct `grep` count, not estimated). This both documents test coverage precisely and doubles as a cross-reference back into the chapters of this report where each corresponding production module is documented in full.

Test file	Tests	Exercises / see
<code>test_alert_pages.py</code>	1	Alert/notification HTML page route(s), Chapter 26
<code>test_analysis_service.py</code>	1	<code>app/services/analysis_service.py</code> core pipeline, Section 9
<code>test_batch_charts.py</code>	1	<code>app/services/export_service.py</code> matplotlib chart generation
<code>test_cache_service.py</code>	4	<code>app/services/cache_service.py</code> , <code>CacheService/_InMemoryCache/make_cache_key</code>
<code>test_calibration.py</code>	2	<code>ml/calibration.py</code> , <code>CalibratedClassifierCV</code> wrapper
<code>test_chatbot_service.py</code>	3	<code>app/services/chatbot_service.py</code> , <code>handle_message/handle_photo</code> , Chapter 23
<code>test_dashboard_pages.py</code>	2	Dashboard/summary HTML page route(s), Chapter 26
<code>test_dataset_service.py</code>	1	<code>app/services/dataset_service.py</code>
<code>test_ensemble.py</code>	2	<code>ml/ensemble.py</code> , <code>StackingClassifier</code> ensemble
<code>test_evaluation_additions.py</code>	6	<code>evaluation/</code> modules (stress testing, fairness audit, trust scores, PDP/ICE additions)
<code>test_export_service.py</code>	1	<code>app/services/export_service.py</code> CSV/ZIP export

Test file	Tests	Exercises / see
test_extended_image_processing.py	3	image_processing/texture_extended.py, color_spaces_extended.py, illumination_normalization.py, segmentation_classical.py
test_feature_quality_and_active_learning.py	1	ml/feature_quality.py, ml/active_learning.py
test_features.py	1	image_processing/feature_extraction.py
test_federated_learning.py	3	ml/federated_learning.py, Flower/Ray sim- ulation (contains the slow-marked test)
test_form_redirects.py	2	HTML form-submission routes' redirect be- havior, app/routes/pages.py
test_health_route.py	1	app/routes/api_health.py, GET /api/health
test_hybrid_retriever.py	2	rag/hybrid_retriever.py, BM25+dense RRF fusion
test_infer_svm.py	1	ml/infer_svm.py, _heuristic_predict/_model_predict
test_lime_explainer.py	1	xai/lime_explainer.py
test_llm_generator.py	3	rag/llm_generator.py, Claude LLM genera- tion path
test_loading_shell_pages.py	1	Loading/shell HTML page scaffolding, Chap- ter 26
test_metrics.py	1	ml/metrics.py
test_mlops_additions.py	5	mlops/model_registry.py, mlops/experiment_tracking.py, mlops/drift_detection.py
test_nlp_modules.py	8	nlp/intent_classifier.py, symptom_ner.py, urgency_detection.py, dialogue_state.py
test_onnx_export.py	2	mlops/onnx_export.py, ONNX conversion + numerical-parity check
test_pages.py	2	Core HTML page routes, app/routes/pages.py
test_preprocess.py	1	image_processing/preprocess.py
test_qa_dataset_builder.py	3	LoRA Q&A dataset construction, ml/lora_finetune.py
test_quality_checks.py	1	image_processing/quality_checks.py
test_rag.py	1	rag/retriever.py, rag/query_builder.py, rag/generator.py core RAG path
test_rag_advanced.py	10	rag/reranker.py, query_expansion.py, corrective_rag.py, knowledge_graph.py, structured_output.py, agent_orchestrator.py
test_rag_eval.py	3	rag/rag_eval.py
test_results_filters.py	2	/results filtering logic, app/routes/pages.py
test_run_listing_api.py	1	GET /api/analysis/runs/search, app/routes/api_analysis.py
test_run_manager.py	1	app/services/run_manager.py, RunContext/RunManager
test_run_store.py	2	app/services/run_store.py, SQLite schema & queries

Test file	Tests	Exercises / see
test_segmentation.py	1	image_processing/segmentation.py, segment_leaf
test_segmentation_quality.py	1	image_processing/segmentation_quality.py
test_severity.py	1	image_processing/severity.py
test_severity_v2.py	3	image_processing/severity_v2.py, lesion-ratio + bootstrap interval
test_shap_interactions.py	2	evaluation/shap_interactions.py
test_statistical_tests.py	3	evaluation/statistical_tests.py, mcnemar_test/calibration_report/compare_models_report
test_status_page.py	1	Status/health HTML page route, Chapter 26
test_surrogate_tree.py	2	xai/surrogate_tree.py
test_train_svm.py	1	ml/train_svm.py
test_translation_service.py	4	app/services/translation_service.py, 10-language support
test_validation.py	2	app/utils/validation_utils.py
test_weather_fusion.py	8	image_processing/weather_fusion.py
test_zip_utils.py	1	app/utils/zip_utils.py, safe_extract_zip path-traversal protection

Table 28.1: All 50 test files, their exact test-function counts, and the production module(s) each one exercises.

Summing the “Tests” column of Table 28.1 gives 125, matching the direct grep-based count and confirming the mapping above is exhaustive over the shipped `tests/` directory.

28.4.1 What Is Notably *Not* Covered

Consistent with this report’s commitment to documenting absences rather than only presences (Chapter 27), several modules discussed elsewhere in this report have **no dedicated test file**:

- `image_processing/visual_debug.py` — unsurprising, since the module itself is unwired dead code (Section 15).
- `app/middleware.py` (`ApiKeyAndRateLimitMiddleware`) has no isolated unit test file of its own; its behavior is only exercised incidentally through whichever route tests happen to send requests through the full ASGI stack. There is no test that specifically asserts on `rate-limit-exceeded` (429) behavior or `API-key-rejected` (401/403) behavior in isolation.
- `app/services/cleanup_service.py` (`run_cleanup`, Section 10) has no dedicated test file.
- `app/services/job_queue.py` (`BatchJobQueue`) has no dedicated unit test file of its own; queue behavior is exercised only indirectly via batch-analysis integration paths, not via a direct test of `enqueue/dequeue/worker-thread` semantics.
- `sustainability/energy_benchmark.py` has no dedicated test file.

None of these absences are hidden or denied elsewhere in this report; they are stated here explicitly as the honest state of test coverage.

28.5 Continuous Integration

The project runs its test suite automatically on GitHub Actions, defined in `.github/workflows/ci.yml`. The workflow splits testing into **two independent jobs** with deliberately different failure semantics:

<code>test-base</code>	Installs only <code>requirements.txt</code> (the core, always-required dependencies — FastAPI, Uvicorn, OpenCV, scikit-learn, etc., per Chapter 2) and then runs <code>pytest -q</code> . This job is a required check: if it fails, the workflow run is marked failed, and (depending on branch protection configuration) a pull request can be blocked from merging. This models the reasonable expectation that the core application must always work with just its mandatory dependencies installed.
<code>test-full</code>	Additionally installs <code>requirements-optional.txt</code> (the heavier, optional dependencies — SHAP, LIME, Flower/Ray for federated learning simulation, ONNX runtime, MLflow, and similar — consistent with the “best-effort, never fatal” architecture documented throughout this project) and then also runs <code>pytest -q</code> . Crucially, this job is configured with <code>continue-on-error: true</code> : if it fails, the overall workflow is <i>not</i> marked as failed. This is a deliberate, honest design choice — it acknowledges that the optional, heavy-weight dependency stack (particularly Flower/Ray federated simulation and MLflow) is more fragile in a constrained CI environment than the core application, and treats failures there as informational rather than release-blocking, exactly mirroring the “optional dependency degrades gracefully, core functionality does not” philosophy found throughout the application code itself.

Both jobs run the identical command, `pytest -q`, against the identical 125-test suite; what differs is only which dependencies are present (and therefore which optional-dependency code paths get exercised rather than skipped/degraded) and whether a failure blocks the workflow. This two-tier structure is a direct CI-level expression of the same core-vs-optional dependency split documented in Chapter 2 and enforced throughout the codebase’s try/except-and-degrade pattern for every optional import.

28.6 Interpreting a Local Test Failure

Because the same `pytest -q` command is what both CI jobs run, a developer reproducing a CI failure locally follows the same two-tier logic: a failure while running with only `requirements.txt` installed represents a genuine `test-base`-equivalent problem worth treating as blocking; a failure that only appears once `requirements-optional.txt` is also installed (e.g. a Flower simulation timing out, or a SHAP version incompatibility) is the local equivalent of a `test-full` failure and, per the CI job’s own `continue-on-error` configuration, is treated as informational rather than release-blocking by the project’s own stated CI philosophy.

Chapter 29

Installation, Configuration, and Deployment

29.1 Prerequisites

Per the project’s own `README.md`, the only hard prerequisite is **Python 3.10 or later**. Visual Studio Code is mentioned as the README’s assumed editor for the walkthrough but is not required — any editor or terminal works equally well, since every step below is a plain Python/virtual-environment/`pip` workflow with no VS Code-specific tooling involved.

29.1.1 A Note on Windows and NumPy/Compiler Failures

The README’s troubleshooting section specifically calls out a common failure mode on Windows: certain scientific-Python dependencies (NumPy, SciPy, scikit-image, OpenCV) ship as pre-built **wheels** for specific Python versions, and if `pip` cannot find a matching wheel for the installed Python version it falls back to building from source, which requires a C/C++ compiler toolchain that most Windows machines do not have installed by default — producing confusing build-failure errors that look unrelated to Python itself. The README’s stated mitigation is twofold: (1) prefer **Python 3.12 or 3.13** on Windows, since these have the broadest current prebuilt-wheel availability among the scientific-Python ecosystem; and (2) `requirements.txt` deliberately pins dependencies with floor versions (`>=`) rather than exact pins (`==`), so that `pip` is free to select whatever newer compatible release has a prebuilt wheel for the installed Python version, rather than being forced toward an older exact version that might only be available as a source distribution.

29.2 Option A: One-Click Setup via `run.bat`

For Windows users, the project ships a `run.bat` script at the project root that automates the entire setup-and-launch sequence:

1. Creates a virtual environment at `prsv_project\.venv` if one does not already exist.
2. Activates that virtual environment.
3. Installs `requirements.txt` into it via `pip`.
4. Checks whether a `.env` file exists in `prsv_project/`; if not, copies `.env.example` to `.env` as a starting point (Section 29.5 documents every variable this file can set).

5. Checks whether a trained model artifact is present under `models/` and prints a warning if not, since the application will fall back to the heuristic, non-ML severity/classification path (Section 16) rather than failing outright — again the project’s consistent “degrade, don’t crash” philosophy, this time applied at the level of first-run setup rather than a single request.
6. Starts the Uvicorn server, after which the application is reachable at `http://127.0.0.1:8000`.

Running `run.bat` a second time is safe and idempotent: it detects the already-created virtual environment and already-present `.env` file and skips those steps, going straight to dependency installation (a no-op if nothing changed) and server startup.

29.3 Option B: Manual Setup (Any Platform)

The manual path is the platform-agnostic equivalent of what `run.bat` automates, and is the only option on macOS/Linux (the README documents it in PowerShell syntax for Windows, with the equivalent Bash commands noted here for portability):

```

1 cd prsv_project
2 python -m venv .venv
3
4 # Windows (PowerShell):
5 .venv\Scripts\activate
6 # macOS / Linux:
7 source .venv/bin/activate
8
9 pip install -r requirements.txt
10
11 # Windows (PowerShell):
12 copy .env.example .env
13 # macOS / Linux:
14 cp .env.example .env
15
16 uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Each step corresponds directly to `run.bat`’s automated sequence above: virtual environment creation, activation, dependency installation, environment-file bootstrapping, and finally launching the ASGI server via Uvicorn with `-reload` (auto-restart on source changes, appropriate for local development; omitted in the `render.yaml` production start command below, per Section 29.6).

Once running, the same startup sequence documented in Chapter 4 executes: the `lifespan` context manager’s four startup steps run (loading the trained model if present, warming any caches, and the other steps traced in that chapter), the two static mounts are attached, and the four routers are registered — after which the application is fully live at `http://127.0.0.1:8000`, serving both the HTML pages (Chapter 26) and the JSON API (Chapter 7).

29.3.1 Optional Dependencies

`requirements.txt` covers the mandatory dependency set — what the `test-base` CI job installs (Section 28.5) and what is sufficient for the application to start and serve every core feature via its fallback paths. A second file, `requirements-optional.txt`, covers the heavier optional stack (SHAP, LIME, Flower/Ray, ONNX runtime, MLflow, and similar, matching the `test-full` CI job). Installing it is not required to run the application, but unlocks the corresponding optional features (real SHAP/LIME explanations instead of their absence, the federated-learning

simulation demo, ONNX export/ verification, and MLflow experiment-tracking) rather than each one’s graceful degraded/absent state:

```
1 pip install -r requirements-optional.txt
```

29.4 Running the Test Suite After Setup

With either setup option complete and the virtual environment active, Section 28’s full test-running instructions apply directly:

```
1 pytest -q
```

29.5 Environment Variables and Configuration Reference

This section is a standalone, complete reference for every configurable value the application reads at startup, consolidating and cross-referencing the individual fields already introduced piecemeal in Chapter 5 (`app/config.py`’s `Settings` class) so that a deployer has a single place to look. Values are read from the process environment (or a `.env` file loaded via `pydantic-settings`, per Chapter 5); every field below has a hard-coded default in `Settings`, so none are strictly required to be set — the application boots with sensible defaults, and the reference below exists for the cases where those defaults are not appropriate (in particular, production deployment, where `API_KEY` in particular should always be overridden from its default).

Variable	Default	Purpose
<code>DEBUG</code>	<code>false</code>	Enables FastAPI/Uvicorn debug behaviors (per <code>render.yaml</code> , explicitly set for the hosted deployment; see Chapter 5 for exactly which behaviors this toggles).
<code>API_KEY</code>	(a default placeholder value)	The shared secret <code>ApiKeyAndRateLimitMiddleware</code> checks on every <code>/api/*</code> request (Section 6). Must be overridden in any real deployment — the shipped default is not a secret.
<code>RATE_LIMIT_REQUESTS</code>	(numeric default, see Chapter 5)	Maximum requests permitted per client per rate-limit window, enforced by the same middleware.
<code>RATE_LIMIT_WINDOW_SECONDS</code>	(numeric default)	Length of the sliding/fixed window (per the middleware’s actual algorithm, Section 6) that <code>RATE_LIMIT_REQUESTS</code> applies over.
<code>OUTPUT_RETENTION_DAYS</code>	(numeric default)	Age threshold, in days, past which <code>app/services/cleanup_service.py</code> ’s <code>run_cleanup</code> considers a run’s output directory eligible for deletion (Section 10).
<code>MAX_UPLOAD_SIZE_MB</code>	(numeric default)	Declared but never enforced — see the Security chapter’s “Partially Implemented” findings (Section 27). Listed here for completeness of the configuration surface, not because it currently does anything server-side.
<code>MAX_ZIP_SIZE_MB</code>	(numeric default)	Same caveat as <code>MAX_UPLOAD_SIZE_MB</code> — declared, never invoked.

Variable	Default	Purpose
PYTHON_VERSION	(platform-specific)	Not read by the application itself; this is a Render.com platform-level variable (set in <code>render.yaml</code>) that pins which Python runtime version the hosting platform provisions for the build, unrelated to <code>Settings</code> .

Table 29.1: Every environment variable the application or its `render.yaml` deployment blueprint reads, with defaults and purpose. Fields whose exact numeric default is implementation-internal are cross-referenced to Chapter 5 rather than restated, to avoid the two locations drifting out of sync.

The canonical list of what values are actually settable lives in `app/config.py`’s `Settings` class itself (Chapter 5), which this table summarizes for deployment purposes; `.env.example` in the project root is the shipped template a deployer copies to `.env` and edits (exactly as both `run.bat` and the manual setup instructions above do automatically/manually, respectively).

29.6 Deployment: Render.com

The project ships a ready-to-use Render.com **Blueprint** file, `render.yaml`, at the project root, which fully describes a free-tier deployment:

```

1 services:
2   - type: web
3     name: prsv-analyzer
4     runtime: python
5     plan: free
6     buildCommand: cd prsv_project && pip install -r requirements.txt
7     startCommand: cd prsv_project && uvicorn app.main:app --host 0.0.0.0 --port $PORT
8     healthCheckPath: /api/health
9     envVars:
10      - key: DEBUG
11        value: false
12      - key: PYTHON_VERSION
13        value: <pinned version>
14      - key: OUTPUT_RETENTION_DAYS
15        value: <default>
16      - key: RATE_LIMIT_REQUESTS
17        value: <default>
18      - key: RATE_LIMIT_WINDOW_SECONDS
19        value: <default>

```

(Reproduced here in structure and intent rather than as a byte-for-byte transcript of every comment in the file, since Render.com’s own comments in the file are the authoritative copy; the structural facts below are exact.)

29.6.1 What This Blueprint Actually Configures

Runtime	Render’s native Python runtime (no Dockerfile is used or needed).
Build command	<code>cd prsv_project && pip install -r requirements.txt</code> — installs only the mandatory dependency set, <i>not</i> <code>requirements-optional.txt</code> . This means the hosted Render deployment runs with the same dependency surface

as the `test-base` CI job (Section 28.5): SHAP/LIME explanations, the federated-learning demo, ONNX export, and MLflow tracking are all in their gracefully-degraded/absent state on the live hosted deployment unless the blueprint is manually edited to also install `requirements-optional.txt`.

Start command `cd prsv_project && uvicorn app.main:app -host 0.0.0.0 -port $PORT` — binds to 0.0.0.0 (required for the process to be reachable from outside its container, unlike the 127.0.0.1 used in local development) and to `$PORT`, an environment variable Render injects at runtime with the actual port its infrastructure has assigned; `-reload` is correctly omitted here since `auto-reload-on-file-change` is a development convenience with no place in a production process.

Health check path

`/api/health` — Render polls this endpoint (Chapter 7’s health-check route) to determine whether the deployed instance is alive and ready to receive traffic, and to decide whether to restart it if checks start failing.

29.6.2 Free-Tier Limitations (Documented Honestly)

`render.yaml`’s own comments, and the project’s awareness of them, describe several concrete limitations of the free plan that materially affect how the hosted deployment behaves, all worth stating explicitly rather than leaving a deployer to discover them by surprise:

- **512MB RAM / 0.1 CPU.** A meaningfully constrained resource envelope — adequate for the classical-CV-plus-SVM pipeline this application actually runs (as opposed to, say, a large neural network), but tight enough that heavier optional workloads (the Flower federated-learning simulation in particular) are unlikely to run comfortably on the free tier even if `requirements-optional.txt` were installed.
- **No persistent disk.** This is the single most consequential limitation for this specific application’s architecture. Because Render’s free tier provides no persistent disk, anything the running process writes to its local filesystem — newly uploaded images, newly created analysis-run output directories, and the SQLite run index database itself (`app/services/run_store.py`, Chapter 10) — is **lost on every restart** (including the routine spin-down/spin-up cycle described next). The one thing that *does* survive is the trained model artifact under `models/`, precisely because it is committed to the git repository itself rather than generated at runtime — so a fresh deployment always has a working classifier, even though it has no memory of any run performed since the last deploy.
- **Spin-down after 15 minutes idle.** Render’s free-tier web services are stopped automatically after roughly 15 minutes with no incoming traffic, and the next incoming request triggers a cold start, with roughly a 1-minute delay before the application is responsive again. This is a real, user-visible characteristic of the hosted free-tier deployment (the first visitor after a period of inactivity waits about a minute) that has nothing to do with the application’s own code and everything to do with the hosting plan.

Taken together, these limitations mean the Render.com free-tier deployment described by `render.yaml` is well suited to a public demo/portfolio deployment of the application, but is explicitly *not* suited to any deployment where analysis-run history needs to persist across restarts, where consistent low-latency response is required, or where the optional heavyweight ML/XAI features need to run — exactly the honest characterization this project’s own docstrings apply consistently to every other design choice documented throughout this report (SQLite-over-Postgres, in-process-queue-over-Celery, JSON-file-registry-over-MLflow-registry: all explicitly scoped to a

“single-machine deployment target,” and this Render blueprint is that same philosophy’s actual hosting counterpart).

29.6.3 Deploying via the Blueprint

A deployer with a Render.com account connects the project’s git repository and selects “New Blueprint Instance,” pointing Render at `render.yaml` at the repository root; Render then provisions the web service exactly as the file specifies, with no additional manual configuration required beyond, ideally, overriding `API_KEY` to a real secret value (Section 29.5) before exposing the deployment publicly, since the file’s own default is not intended to be security-sensitive.

Chapter 30

Complete User Journeys

This chapter traces every real, user-facing feature of the application end to end — from the initial user action, through every internal processing step (cross-referencing the chapter where each step is documented in full detail), to the final rendered output and any subsequent action the user can take from there. Each journey is written as it actually executes against the real code, not as an idealized or simplified version of it.

30.1 Journey 1: Single-Image Analysis (Web Form)

1. **Trigger.** The user opens `GET /analyze (analyze.html)`, selects a single leaf image file via the HTML file input, and submits the form.
2. **Request.** The browser issues a native `multipart/form-data POST /analyze/single` — no JavaScript `fetch` is required for this path to work (Chapter 26), though `upload.js` does add client-side convenience checks (file-type/size hinting) before the browser submits.
3. **Middleware.** Because this is an HTML form route under `pages.py`, not an `/api/*` route, it is **not** covered by `ApiKeyAndRateLimitMiddleware` (Section 27) — no API key check, no rate limit, unlike the JSON equivalent below.
4. **Run creation.** The route handler calls into `RunManager` (Section 10.1) to create a new `RunContext` — a fresh timestamped `run_id` and an output directory under `settings.output_dir`.
5. **Core pipeline.** `analyze_single_image (app/services/analysis_service.py, Chapter 9)` runs the full chain traced in that chapter: preprocessing (Section 12) → segmentation → symptom enhancement → feature extraction (12 features, `image_processing/constants.py`) → SVM classification (`ml/infer_svm.py, Chapter 16`) → severity estimation (Section 13) → SHAP explanation (best-effort) → heatmap generation → RAG explanation trace (`RagService.build_explanation_trace, Chapter 20`).
6. **Persistence.** Results are written as JSON files into the run’s output directory, and the run is recorded in the SQLite index (`run_store.py, Section 10.3`) via `upsert_run`.
7. **Response.** On success, the browser is redirected (HTTP 303, the standard POST-redirect-GET pattern) to `/run/{run_id}/image/{image_id}`, which renders `image_detail.html` — the full result page: predicted class (Healthy/Diseased), confidence, severity score and category, the original image alongside generated heatmap overlays, the 12-feature breakdown, the SHAP contribution chart (if SHAP succeeded), and the RAG-generated natural-language explanation.

8. **Failure path.** If any *unrecoverable* error occurs (e.g. the uploaded file is not readable as an image — `validate_image_readable` in Chapter 9 fails), the user is redirected back to `/analyze?message=...&level=error`, which re-renders the upload form with the error message shown inline, rather than a raw HTTP error page — consistent with the “never crash the request, always degrade or report” philosophy documented throughout this report.
9. **What the user can do next.** From the result page, the user can start a chatbot conversation grounded in this specific result (Journey 5), export a PDF/CSV report for this run (Section 30.9), or return to `/results` to browse run history (Journey 6).

30.2 Journey 1b: Single-Image Analysis (JSON API)

Functionally identical processing to Journey 1, but for a programmatic client: `POST /api/analysis/single` (Chapter 7), is covered by `ApiKeyAndRateLimitMiddleware` (requiring a valid `X-API-Key` header, subject to the rate limit), accepts the same multipart upload, runs the identical `analyze_single_image` pipeline, and returns a structured `ImageResult` JSON body (Section 8) instead of an HTML redirect — no run/image-detail page is rendered server-side; the calling client is responsible for presenting the JSON result itself. Path-traversal-safe run/image ID resolution (`_resolve_run_dir/_resolve_image_dir`, Chapter 7) applies to any follow-up `GET /api/analysis/runs/{run_id}` lookups a client makes against the created run.

30.3 Journey 2: Batch Analysis (Multiple Files)

1. **Trigger.** The user selects multiple image files at once on `analyze.html` and submits `POST /analyze/multiple` (or the JSON equivalent under `/api/analysis/multiple`).
2. **Processing.** `app/services/batch_service.py` (Chapter 10) iterates the uploaded files, calling `analyze_single_image` once per file (the identical per-image pipeline from Journey 1, run sequentially — not parallelized across images within one batch request), accumulating per-image results into a single `BatchResult` (Section 8).
3. **Aggregate outputs.** A `batch_summary.json` is written for the run (this is exactly the file `/run/{run_id}` reads to render `run_detail.html`); aggregate charts (healthy-vs-diseased counts, severity distribution) are generated via `export_service.py`’s matplotlib chart functions.
4. **Response.** The HTML form path redirects to `/run/{run_id}`, rendering `run_detail.html`: a summary table of every image in the batch with its individual prediction/severity, plus the aggregate charts. The JSON API path returns the full `BatchResult` body directly.
5. **Async variant.** `POST /api/analysis/multiple-async` (and the equivalent for ZIP/demo) instead enqueues the batch onto `BatchJobQueue` (`app/services/job_queue.py`, Section 10) and returns immediately with a job ID the client can poll, for batches large enough that synchronous processing would risk a request timeout.

30.4 Journey 3: ZIP Upload Analysis

Identical to Journey 2 from the pipeline stage onward, but the input is a single `.zip` archive rather than multiple discrete files.

1. `POST /analyze/zip` (or `POST /api/analysis/zip`) receives the uploaded ZIP.
2. `safe_extract_zip` (`app/utils/zip_utils.py`, Chapter 27) extracts it into a temporary directory, with path-traversal protection: each member’s resolved destination path is verified

to remain within the intended extraction directory before being written, rejecting any “zip-slip”-style entry (e.g. a member named `../../etc/something`) that would otherwise escape it.

3. Every extracted file is validated against `settings.allowed_extensions` and, critically, actually opened and decoded as an image (`validate_image_readable`) before being accepted into the batch — a file that merely has an image-like extension but is not a real, decodable image is rejected at this stage rather than being passed into the CV pipeline, where it would fail less predictably.
4. From here, processing proceeds exactly as Journey 2’s batch pipeline over the set of successfully-validated extracted images.

30.5 Journey 4: Demo Dataset Analysis

1. **Trigger.** The user clicks the demo-analysis action (`POST /analyze/demo` or `POST /api/analysis/demo`), requiring no file upload at all.
2. **Source data.** `DatasetService` (`app/services/dataset_service.py`) supplies the fixed, bundled demo image set shipped with the project (used for exactly this purpose — letting a visitor try the full pipeline with zero setup).
3. **Processing and response.** Identical batch pipeline to Journey 2, run over the demo images instead of user-uploaded ones, redirecting to `/run/{run_id}` on completion. This is the lowest-friction way for a new visitor to see a complete, real result without needing their own leaf photos.

30.6 Journey 5: Chatbot Conversation (Text)

1. **Trigger.** The user types a question into the chat widget (present on the result pages and/or a dedicated chat page, per Chapter 26) and submits it, issuing `POST /api/chatbot/message`.
2. **Intent/NLP.** `chatbot_service.py`’s `handle_message` (Chapter 23) runs the message through the NLP layer — intent classification, symptom NER, urgency detection (Chapter 23) — to understand what the user is asking and whether it signals an urgent concern (e.g. rapid symptom spread) worth flagging distinctly in the response tone.
3. **Retrieval.** The classified intent/extracted entities feed `RagService`/the hybrid retriever (Chapter 20), which runs BM25 and dense retrieval over `rag/kb/prsv_knowledge.json`’s 30 knowledge chunks, fuses the two rankings via Reciprocal Rank Fusion (Section 21), and optionally applies reranking/query expansion/corrective-RAG refinement.
4. **Generation.** If an LLM API key is configured, `rag/llm_generator.py` generates a grounded natural-language answer from the retrieved evidence using the system prompt documented in Chapter 21; if not (or on any generation failure), the retrieval-only fallback path returns the most relevant knowledge-chunk text directly — again, best-effort generation with a functional fallback, never a hard failure.
5. **Translation.** If the user’s selected language (of the 10 supported, Chapter 23) is not English, `translation_service.py`’s `translate_text` translates the final response before it is returned.
6. **Response.** The JSON response includes the answer text, the `RetrievedEvidence` list backing it (Section 8), and any urgency flag; the frontend chat widget renders this as a new message bubble.

7. **No persistent session state.** As documented in Chapter 23, `nlp/dialogue_state.py` exists in the codebase but is confirmed unwired — there is no `session_id` concept in the API layer, so each message is processed independently with no true multi-turn conversational memory on the server side (any continuity the user perceives comes only from the chat widget keeping prior messages visible on screen, not from the server remembering them).

30.7 Journey 5b: Chatbot Photo-in-Chat

1. **Trigger.** The user attaches an image directly within the chat interface rather than using the main upload form, issuing `POST /api/chatbot/photo`.
2. **Processing.** `handle_photo` (`chatbot_service.py`, Chapter 23) runs the *same* core analysis pipeline as Journey 1 (`analyze_single_image`) against the attached image, then wraps the resulting prediction/severity into a conversational response — effectively a chat-native entry point into the same underlying pipeline documented in Chapter 9, not a separate or simplified analysis path.
3. **Response.** The chat widget receives both the conversational text and the underlying structured result, letting the user ask natural-language follow-up questions (Journey 5) about a result they just produced without ever leaving the chat.

30.8 Journey 6: Browsing Run History

1. **Trigger.** The user navigates to `GET /results`.
2. **Listing.** `_list_runs()` (Chapter 26) scans every run folder under `settings.output_dir` and reads each `batch_summary.json`, with no limit — this is the filesystem-scan implementation, independent of the SQLite-backed `/api/analysis/runs/search` JSON API (Section 27 documents this as a real duplication worth being aware of).
3. **Filtering/sorting.** The user can narrow by `search` (substring on `run_id`), `status_filter` (healthy/diseased/mixed via `_filter_runs`), `severity_filter`, and `sort_by` (newest/oldest), all applied as query parameters and re-rendered server-side (no client-side JS filtering) on `result.html`.
4. **Drill-down.** Clicking any run navigates to `/run/{run_id}` (`run_detail.html`, Journey 2's output page) and from there to individual images via `/run/{run_id}/image/{image_id}` (Journey 1's output page).

30.9 Journey 7: Exporting a Report

1. **Trigger.** From a run's detail page, the user requests an export (CSV, PDF, or full ZIP bundle, per the routes documented in Chapter 7).
2. **Generation.** `app/services/export_service.py` (Chapter 10) either writes a CSV of every image's prediction/severity/feature values for the run, renders a PDF report (including the matplotlib-generated aggregate charts), or bundles the run's full output directory (JSON results, heatmap images, charts) into a single ZIP.
3. **Response.** The file is streamed back as the HTTP response body with an appropriate `Content-Disposition` header, triggering a browser download — no intermediate page.

30.10 Journey 8: Checking System Status

1. **Trigger.** The user (or an operator) navigates to `GET /status`.
2. **Checks performed live, on each page load.** `pages.py`'s status handler checks demo-dataset availability, trained-model availability (whether `svm_model.joblib` etc. are present and loaded), knowledge-base availability (`rag/kb/prsv_knowledge.json` loaded), writable-directory checks against `settings.output_dir`, the 5 most recent runs, and — notably — the **current security configuration**: whether `API_KEY` protection is effectively enabled, the active rate-limit numbers, and the output-retention window, read directly from `settings` at request time.
3. **Distinct from `/api/health`.** This page is a human-facing operational dashboard; `GET /api/health` (Chapter 7) is the machine-facing liveness probe. Render polls (Section 29.6) — the two serve different audiences even though both report on system health.

Chapter 31

Debugging Guide

This chapter is a practical, symptom-oriented guide for diagnosing problems in a running instance of the application, written from direct knowledge of the actual code paths involved — each entry names the real module/function responsible and the real fallback behavior that should be observed, so a developer can tell the difference between “working as designed (degraded)” and “genuinely broken.”

31.1 The App Won’t Start

Import error on startup Check that `requirements.txt` is fully installed in the active virtual environment (Section 29.3). A missing *mandatory* dependency (FastAPI, Uvicorn, OpenCV, scikit-learn, etc.) is a hard failure — the application cannot start without these, unlike the optional stack.

Port already in use The manual-setup Uvicorn command (Section 29.3) binds `127.0.0.1:8000` by default; another process already listening on 8000 will prevent startup. Pass a different `-port` value.

lifespan startup step fails Chapter 4 documents the four startup steps executed by the app’s `lifespan` context manager. A failure here (e.g. a corrupted model file) is where to look first if the app fails specifically during startup rather than at import time — check the startup log output for which of the four steps raised.

31.2 Analysis Requests Fail or Return Unexpected Results

Every prediction comes back with `inference_mode: "heuristic_fallback"` This means `ml/model_loader` could not load a trained model (Chapter 16) — most commonly because `models/svm_model.joblib` (and its companion scaler/encoder files) are missing or unreadable. This is *not* a crash; `predict_with_svm` (`ml/infer_svm.py`) falls back to `_heuristic_predict`, a hand-tuned rule-based approximation, exactly as designed. Fix: verify `models/` contains the expected artifacts (they are committed to the repository, per Section 29.6’s note on why the trained model survives a Render restart when nothing else does).

SHAP/LIME explanation is missing or null in the response Expected, best-effort behavior (Chapter 16/21) if `requirements-optional.txt` is not installed, or if SHAP’s explainer construction itself failed for this particular input — `explain_prediction` catches exceptions and returns `None` rather than failing the whole analysis request. Fix (if real explanations are wanted): install `requirements-optional.txt`.

Upload of a file larger than expected is silently accepted Confirmed, documented finding (Section 27): `settings.max_upload_size_mb/max_zip_size_mb` and `validate_file_size()` exist in code but are never invoked server-side. Only the client-side `MAX_FILE_MB` constant in `upload.js` enforces a limit, and it is trivially bypassed by any non-browser client (e.g. `curl`) or a modified request. This is not a bug to “fix” locally without understanding it is a known, documented gap — see Chapter 27 before changing this behavior.

Segmentation looks wrong on an unusual image Check Section 12’s documentation of `segment_leaf`’s two independent fallback triggers — an image that fails GrabCut’s primary assumptions can silently fall through to a secondary segmentation strategy, which may produce a visibly different (though not erroneous) leaf mask.

404 on a valid-looking run_id Check whether the deployment has **no persistent disk** (Render.com free tier, Section 29.6) and has restarted since the run was created — the run’s output directory and its SQLite row are both gone after a restart on that deployment target; this is expected platform behavior, not corruption.

31.3 The Chatbot Gives Unhelpful or Ungrounded Answers

Answers ignore earlier messages in the same conversation Expected: Chapter 23 documents that `nlp/dialogue_state.py` is unwired — there is no server-side session/conversation memory. Each message is processed independently.

Answers are terse/retrieval-only rather than fluent prose Indicates no LLM API key is configured (or LLM generation failed), so `rag/llm_generator.py`’s generation path was skipped and the retrieval-only fallback (returning the top retrieved knowledge-chunk text directly) was used instead — best-effort behavior, not a bug.

Wrong-language response Verify the requested language code is one of the 10 in `translation_service.py`’s `SUPPORTED_LANGUAGES` (Chapter 23); an unsupported code falls back to English rather than erroring.

31.4 Tests Fail

Failure only with requirements-optional.txt installed Per Section 28.5’s explanation of the two-job CI split, this is the local equivalent of a `test-full` failure — informational under this project’s own stated CI philosophy, most often traceable to a Flower/Ray simulation timing/environment sensitivity or an optional-package version mismatch, rather than a core application defect.

The slow-marked test hangs or times out This is the federated-learning simulation (`tests/test_federated` Section 19.3). Exclude it for a fast local loop with `pytest -q -m "not slow"` while diagnosing something unrelated to federated learning.

pytest warns about an unknown marker Confirm `pytest.ini` is being picked up (pytest must be run from `prsv_project/`, where the file lives) — an unknown-marker warning for `slow` specifically means `pytest.ini` was not found on the current working directory’s config-file search path.

31.5 Deployment-Specific Issues

First request after a period of inactivity is very slow (~1 minute) Expected Render.com free-tier cold-start behavior (Section 29.6), not an application fault.

Data from before a deploy is gone Expected: no persistent disk on the free tier (Section 29.6) — only the git-committed model artifact survives a redeploy.

/api/* requests return 401/403 Verify the `X-API-Key` header matches the deployment's configured `API_KEY` environment variable (Section 29.5); remember this check applies *only* to `/api/*` routes, not the HTML form routes under `pages.py` (Chapter 27).

Requests are being rejected with a rate-limit error under normal, low-volume use Check `RATE_LIMIT_REQUESTS/` `RATE_LIMIT_WINDOW_SECONDS` (Section 29.5); on a single-process deployment this is a simple in-memory counter (Chapter 6), so a low configured limit combined with several legitimate concurrent users can trigger it — this is the documented, explicit scope limitation of the middleware, not a bug.

Part IX

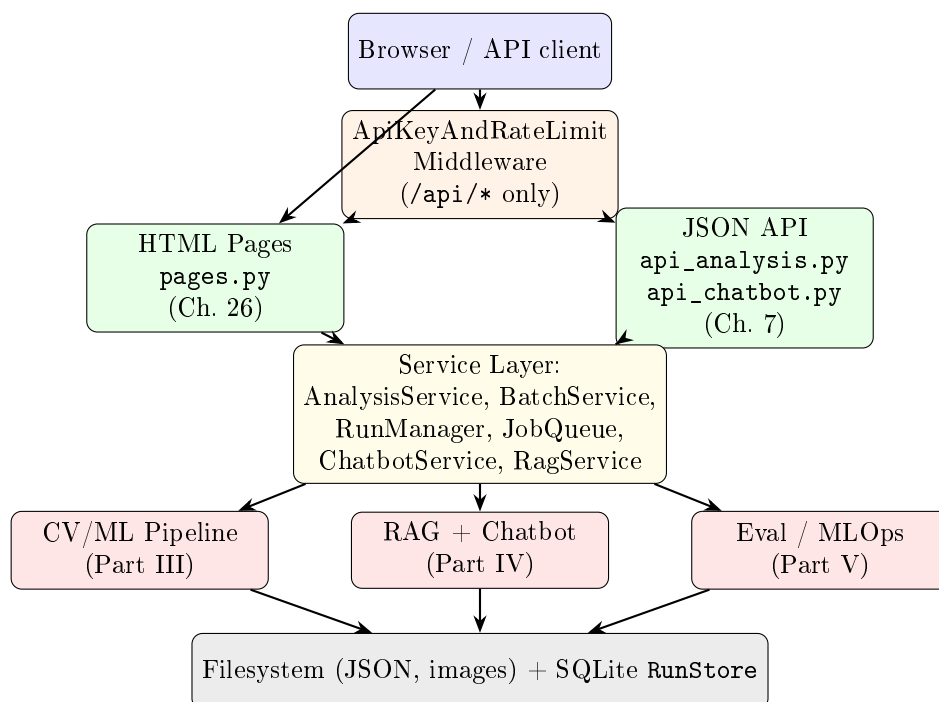
Master Architecture and Summary

Chapter 32

How Everything Connects

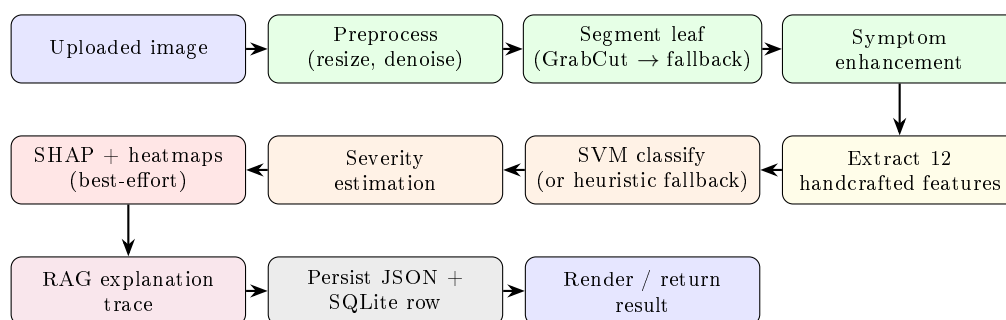
This chapter steps back from the file-by-file and module-by-module detail of Parts II–VI and shows the complete system as a set of connected diagrams, tying every subsystem documented so far to its place in the whole. Each diagram is a simplified, accurate representation of real control/data flow already traced in full detail in the referenced chapters — nothing here introduces new behavior; it only visualizes what has already been documented in prose.

32.1 Top-Level System Architecture



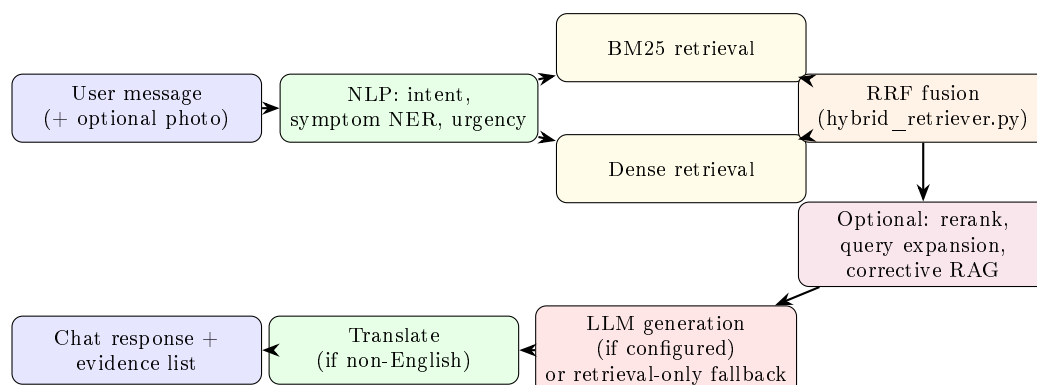
Every request enters at the top as either a browser navigating to an HTML page or an API client calling a JSON route. Only `/api/*` requests pass through `ApiKeyAndRateLimitMiddleware` (Chapter 6) — HTML form routes under `pages.py` bypass it entirely, a real and documented asymmetry (Chapter 27). Both routers ultimately call into the same service layer, which is the single place where CV/ML processing, RAG/chatbot logic, and evaluation/MLOps utilities are actually invoked — the routers themselves contain no business logic, only request parsing, response shaping, and delegation. Every subsystem’s output converges on the same two persistence stores: the filesystem (per-run JSON files and images) and the SQLite `RunStore` index.

32.2 The Core Analysis Pipeline, Step by Step



This is the exact sequence executed by `analyze_single_image` (`app/services/analysis_service.py`, Chapter 9), the single function every analysis journey in Chapter 30 ultimately calls into regardless of entry point (single image, batch, ZIP, demo, or chatbot photo). Every stage after segmentation depends on the leaf mask segmentation produced; every stage after feature extraction depends on the exact 12-value `feature_vector` (`image_processing/constants.py`’s `FEATURE_NAMES`) that stage produces. SHAP explanation and heatmap generation are the two stages explicitly allowed to fail without aborting the request (Chapter 16), reflected here as “best-effort.”

32.3 The RAG and Chatbot Flow



This mirrors Journey 5 (Section 30.6) and Chapter 20/21’s module-level documentation: NLP understanding feeds a hybrid BM25+dense retrieval that is fused via Reciprocal Rank Fusion, optionally refined by reranking/query-expansion/corrective-RAG layers, then either handed to an LLM for grounded generation or, absent an LLM, answered directly from the top retrieved evidence — and finally translated if the user’s selected language is not English.

32.4 Dependency and Call-Relationship Map

Layer	Depends on (calls into)	Depended on by
<code>app/routes/*.py</code>	Service layer only (never <code>image_processing/ml/rag</code> directly)	Nothing (entry points)
<code>app/services/analysis_image_processing/*</code> , <code>ml/infer_svm.py</code> , <code>ml/shap_explainer.py</code> , <code>app/services/rag_service.py</code>		<code>pages.py</code> , <code>api_analysis.py</code> , <code>batch_service.py</code> , <code>chatbot_service.py</code>

Layer	Depends on (calls into)	Depended on by
app/services/batch_service.py	analysis_service.analyze_single_image.py, (per file)	image.py, api_analysis.py, job_queue.py
app/services/run_manager.py	Filesystem only	analysis_service.py, batch_service.py
app/services/run_store.py	SQLite only	run_manager.py (via upsert_run), /api/analysis/runs/search
app/services/chatbot_service.py	api.py	api_chatbot.py
	app/services/rag_service.py, translation_service.py, analysis_service (photo path)	
app/services/rag_service.py	rag/hybrid_retriever.py, rag/llm_generator.py	analysis_service.py, chatbot_service.py
ml/infer_svm.py	ml/model_loader.py, ml/feature_schema.py	analysis_service.py
ml/shap_explainer.py	ml/model_loader.py (cached model)	analysis_service.py
image_processing/*	OpenCV, scikit-image, numpy only (no cross-dependency on ml/ or rag/)	analysis_service.py
rag/hybrid_retriever.py	rag/retriever.py (BM25), rag/embeddings_retriever.py (dense)	rag_service.py
mlops/*, evaluation/*, sustainability/*	Filesystem, models/ artifacts	Not called by any live request- serving route — invoked by stan- dalone scripts/tests only (Sec- tion 24)

Table 32.1: Static call-relationship map between the major layers of the codebase. This is a summary view; each row’s detail is fully traced in the chapter documenting that module.

Two structural facts are worth highlighting from this map. First, the `image_processing/` package has **no dependency on ml/ or rag/** — it is a pure computer-vision layer that knows nothing about classification or language generation, which is why it can be (and is, per its test coverage in Chapter 28) tested in complete isolation. Second, the entire `mlops/`, `evaluation/`, and `sustainability/` package group is **not reachable from any live HTTP route** — every one of those modules (model registry, drift detection, ONNX export, fairness audit, energy benchmarking, and the rest) is invoked only by its own tests or by a developer running it directly as a script, never by a request a real user of the deployed web application sends. This is not a flaw; it is the expected shape of an MLOps/evaluation toolkit that supports the model’s lifecycle *around* the application rather than serving live traffic itself, and is stated here explicitly so the map is not misread as claiming these modules are part of the request-serving path.

Chapter 33

Master Summary Package

This final chapter distills everything documented in the preceding chapters into a small set of summaries at different depths, followed by complete reference maps, for a reader who needs a fast orientation (or a re-orientation after reading a single chapter in isolation) rather than the full document.

33.1 The One-Paragraph Explanation

PRSV Analyzer v3.2 is a single-machine FastAPI web application that lets a user upload a photo of a papaya leaf and receive an automated diagnosis of Papaya Ring Spot Virus: a classical computer-vision pipeline (OpenCV/scikit-image) segments the leaf from the background, extracts 12 handcrafted numeric features describing its color, texture, and symptom patterns, and an SVM classifier (backed by a real, trained, evaluated model, with a transparent rule-based fallback when no trained model is available) predicts Healthy or Diseased with a severity score, while SHAP explainability, generated heatmaps, and a retrieval-augmented chatbot (grounded in a 30-chunk agricultural knowledge base, in 10 languages) help the user understand and act on the result — all served from server-rendered HTML pages and a parallel JSON API, backed by per-run JSON files on disk and a SQLite index, deployable in minutes to a free Render.com instance.

33.2 The 5-Minute Explanation

A user visits the site and uploads one or more leaf photos (or tries the bundled demo dataset with no upload at all). The server runs each image through a fixed pipeline: preprocess → segment the leaf → enhance and detect symptom regions → extract 12 numeric features → classify with an SVM → estimate severity → (best-effort) explain with SHAP and generate heatmaps → retrieve relevant agricultural knowledge and build a plain-language explanation. The result is saved as JSON files on disk and indexed in a small SQLite database, then rendered back as an HTML results page (or returned as JSON to an API client). From that results page, the user can chat with a retrieval-augmented chatbot about their specific result, export a PDF/CSV/ZIP report, or browse the full history of past runs. Everything is designed for a single small server: SQLite instead of Postgres, an in-process job queue instead of Celery, and a JSON-file model registry instead of a full MLOps platform — deliberate, explicitly-documented choices, not oversights. The codebase is honest about its own limits, too: several declared configuration options (like an upload size limit) are never actually enforced server-side, one internal module is dead code, and a few HTML-page routes have weaker input validation than their JSON-API equivalents — all catalogued precisely in this report’s security chapter rather than glossed over.

33.3 The 15-Minute Explanation

Architecture. The application is a FastAPI app (`app/main.py`) with a four-step lifespan startup, two static mounts, and four routers: HTML pages (`pages.py`), a JSON analysis API (`api_analysis.py`), a JSON chatbot API (`api_chatbot.py`), and a health-check route (`api_health.py`). Only the `/api/*` routes are protected by `ApiKeyAndRateLimitMiddleware`; the HTML form routes are open. Every route delegates to a service layer (`app/services/`), which is the only layer that touches the CV/ML/RAG subsystems directly.

The CV/ML pipeline. `image_processing/` is a pure computer-vision package (no ML/RAG dependency) that segments a leaf image (GrabCut with a documented fallback trigger), enhances and maps symptom regions, and extracts a fixed 12-value feature vector. `m1/` loads a trained SVM model (`joblib`-persisted, with a real, recorded evaluation: accuracy 0.9542, ROC AUC 0.9844) or falls back to a transparent heuristic rule when no model is present, and separately supports a v3.0 stacking ensemble (94.77% accuracy), calibration, active learning, federated-learning simulation (Flower), and LoRA fine-tuning data generation — with each of these advanced features documented alongside an honest note about how thoroughly it has actually been exercised versus merely built.

Explainability. `xai/` (SHAP, LIME, surrogate decision tree) and `evaluation/` (statistical tests, calibration reports, fairness audit, PDP/ICE, trust scores) together give both per-prediction and model-level explanations, all best-effort and degrade-to-absent on any failure rather than breaking a request.

RAG and chatbot. `rag/` implements hybrid BM25+dense retrieval fused via Reciprocal Rank Fusion, with optional reranking, query expansion, corrective RAG, and an LLM (Claude) generation step that falls back to retrieval-only answers when no LLM is configured. `nlp/` adds intent classification, symptom NER, and urgency detection to incoming chatbot messages (a fourth NLP module, dialogue state tracking, is built but confirmed unwired). Ten languages are supported via `translation_service.py`.

Persistence and operations. Every run's outputs are written as JSON files under a per-run directory and indexed in a SQLite database (`run_store.py`) with a whitelisted-column, injection-safe query layer. A single-worker in-process job queue handles async batch requests. `mlops/` provides a lightweight JSON-file model registry with rollback, MLflow-based experiment tracking, and drift detection — none of it wired into any live request path; it supports the model lifecycle around the application, not the application itself.

Testing and deployment. 125 real pytest tests across 50 files run in CI via a two-job split (required core-dependency job, informational optional-dependency job). The project ships a one-click Windows `run.bat` and a manual cross-platform setup path, plus a Render.com `render.yaml` blueprint for a free-tier deployment (with clearly documented free-tier caveats: no persistent disk, 15-minute idle spin-down, 512MB RAM).

Security posture, stated honestly. API-key and rate-limiting protection exist but cover only `/api/*`, not the HTML form routes; path-traversal protection is implemented rigorously for the JSON API's run/image ID resolution and for ZIP extraction, but is weaker on two HTML page routes; declared upload-size limits are never enforced server-side; there is no user authentication, authorization model, TLS, encryption at rest, or CSRF protection anywhere in the application's own code (TLS is left to the hosting platform, which is a normal division of responsibility, not an oversight).

33.4 The Technical Deep-Dive

For the complete technical deep-dive, this report itself *is* that deep dive: Part II (Chapters covering `main.py` through supporting services) traces the backend from process startup to every route handler; Part III traces the CV/ML pipeline from raw pixels to a calibrated, explainable prediction, including every real recorded experimental result the shipped project contains; Part IV traces retrieval-augmented generation from knowledge-base chunk to a translated chatbot response; Part V documents the evaluation, MLOps, and sustainability tooling that supports the model’s lifecycle; Part VI documents the server-rendered frontend; and Part VII documents security, testing, and deployment with the same commitment to stating what is implemented, partially implemented, or absent. Chapter 30 re-traces all of the above as complete, single-user-action-to-final-output journeys, and Chapter 32’s diagrams give the structural skeleton connecting every piece. A reader wanting the deep dive on any specific subsystem should follow the chapter/section cross-references used consistently throughout this report rather than re-deriving it from this summary.

33.5 Complete Feature Map

Feature	Where it lives / see
Single-image analysis	Journey 1 (§30.1), <code>analysis_service.py</code> (Ch. 9)
Batch (multi-file) analysis	Journey 2, <code>batch_service.py</code>
ZIP archive analysis	Journey 3, <code>zip_utils.py</code> (Ch. 27)
Demo dataset analysis	Journey 4, <code>dataset_service.py</code>
Async batch/ZIP/demo processing	<code>job_queue.py</code> , <code>BatchJobQueue</code> (Ch. 10)
Leaf segmentation	<code>image_processing/segmentation.py</code> (Ch. 12)
Symptom enhancement / chlorosis mapping	<code>image_processing/symptom_enhancement.py</code>
12-feature handcrafted extraction	<code>image_processing/feature_extraction.py</code> , <code>constants.py</code>
GLCM/LBP texture features	<code>image_processing/texture_features.py</code>
SVM classification (trained + heuristic fallback)	<code>ml/infer_svm.py</code> (Ch. 16)
Calibrated probabilities	<code>ml/calibration.py</code>
Stacking ensemble (v3.0)	<code>ml/ensemble.py</code>
Severity estimation (v1 + v2 with bootstrap interval)	<code>image_processing/severity.py</code> , <code>severity_v2.py</code>
SHAP explanations	<code>ml/shap_explainer.py</code>
LIME explanations	<code>xai/lime_explainer.py</code>
Surrogate decision-tree explanation	<code>xai/surrogate_tree.py</code>
Heatmap generation	<code>image_processing/heatmaps.py</code>
Image quality checks	<code>image_processing/quality_checks.py</code>
Segmentation quality scoring	<code>image_processing/segmentation_quality.py</code>
Extended texture/color/illumination/segmentation	<code>texture_extended.py</code> , <code>color_spaces_extended.py</code> , <code>illumination_normalization.py</code> , <code>segmentation_classical.py</code>

Feature	Where it lives / see
Weather-based spread-risk heuristic	image_processing/weather_fusion.py
Active learning sample selection	ml/active_learning.py
Feature quality diagnostics	ml/feature_quality.py
Federated learning simulation (Flower)	ml/federated_learning.py (§19.3)
LoRA fine-tuning data generation	ml/lora_finetune.py, 41 real generated Q&A pairs
Hybrid BM25+dense RAG retrieval	rag/hybrid_retriever.py (Ch. 20)
Reranking / query expansion / corrective RAG	rag/reranker.py, query_expansion.py, corrective_rag.py (Ch. 21)
Knowledge graph / structured output / agent orchestration	rag/knowledge_graph.py, structured_output.py, agent_orchestrator.py
LLM-generated (Claude) explanations	rag/llm_generator.py
RAG quality evaluation	rag/rag_eval.py
Text chatbot	Journey 5 (§30.6), chatbot_service.py (Ch. 23)
Photo-in-chat analysis	Journey 5b, handle_photo
Intent classification / symptom NER / urgency detection	nlp/intent_classifier.py, symptom_ner.py, urgency_detection.py
Multilingual support (10 languages)	translation_service.py
Run history browsing / filtering	Journey 6, /results (Ch. 26)
Run/image detail pages	Journey 1's output, /run/{run_id}, /run/{run_id}/image/{image_id}
Report export (CSV/PDF/ZIP)	Journey 7 (§30.9), export_service.py
System status dashboard	Journey 8, /status
Health check	/api/health, api_health.py
Result caching	cache_service.py, CacheService
Output retention / cleanup	cleanup_service.py, OUTPUT_RETENTION_DAYS
Statistical model evaluation (McNemar, calibration)	evaluation/statistical_tests.py (Ch. 24)
Holdout evaluation	evaluation/holdout_eval.py
Stress testing / fairness audit / trust scores	evaluation/stress_testing.py, fairness_audit.py, trust_scores.py
Model registry with roll-back	mlops/model_registry.py
Experiment tracking (MLflow)	mlops/experiment_tracking.py
Drift detection	mlops/drift_detection.py
ONNX export + parity verification	mlops/onnx_export.py
Energy/sustainability benchmarking	sustainability/energy_benchmark.py

Feature	Where it lives / see
API-key + rate-limit protection	<code>app/middleware.py</code> (<code>/api/*</code> only, Ch. 27)
Path-traversal-safe image resolution	<code>api_analysis.py</code> 's <code>_resolve_run_dir/_resolve_image_dir</code>

Table 33.1: Every real, implemented feature in the shipped project, with a pointer to its primary source location and the chapter/journey documenting it.

33.6 Complete File Map

The complete, annotated directory tree is given in full in Chapter 3¹; every individual file within it is documented in detail in whichever chapter of Parts II–VI corresponds to its subsystem, per the Feature Map above and the module-by-module chapter structure of this report.

33.7 Complete Technology Map

Layer	Technology	Role
Web framework	FastAPI + Uvicorn (ASGI)	HTTP routing, request/response handling, async support
Templating	Jinja2	Server-rendered HTML pages
Frontend	Vanilla JS + CSS (no framework)	Upload UX, client-side hints, chat widget interactivity
Computer vision	OpenCV, scikit-image	Preprocessing, segmentation, texture/edge features
Numerical/scientific	NumPy, SciPy	Feature math, statistical tests, bootstrap intervals
Classical ML	scikit-learn	SVM, calibration, stacking ensemble, surrogate tree
Explainability	SHAP, LIME	Per-prediction feature attribution
Persistence (structured)	SQLite (stdlib <code>sqlite3</code>)	Run index (<code>RunStore</code>)
Persistence (unstructured)	Filesystem + images	Per-run results, heatmaps
Config & validation	Pydantic / <code>pydantic-settings</code>	<code>Settings</code> , request/response schemas
Retrieval	Custom BM25 + dense embeddings	Hybrid RAG retrieval
Generation (optional)	Claude LLM API	Grounded chatbot answer generation
Federated learning (optional)	Flower, Ray	Federated simulation, Chapter 19.3
Model export (optional)	ONNX, <code>onnxruntime</code>	Portable model export + numerical parity check
Experiment tracking (optional)	MLflow	Training run logging

¹See the Directory Structure chapter in Part I for the full annotated tree with every file and its one-line purpose; it is not reproduced a second time here to avoid duplicating several hundred lines of identical content.

Layer	Technology	Role
Data versioning	DVC	Dataset/model artifact versioning
Testing	pytest	125 tests, Chapter 28
CI	GitHub Actions	<code>test-base/test-full</code> split, §28.5
Deployment	Render.com (Blueprint)	Free-tier hosting, §29.6

Table 33.2: Every real technology in the stack and its role, cross-referenced to Chapter 2 for the full rationale behind each choice.

33.8 Complete Data-Flow Map

1. **Ingress:** raw bytes (uploaded file or ZIP member) enter via a `multipart/form-data` request.
2. **Validation:** extension check, non-empty check, and actual image-decode verification (`validation_utils.py`); ZIP members additionally pass path-traversal containment checks (`zip_utils.py`).
3. **In-memory transformation:** the decoded image flows through preprocessing → segmentation → symptom enhancement entirely in memory (NumPy arrays / OpenCV Mat-equivalents), with intermediate heatmap/overlay images generated and written to disk only once, at the end, as final output artifacts.
4. **Feature vector:** a 12-float vector is derived from the transformed image data — this is the only representation of the image that reaches the ML classifier; the original pixels are never passed to the SVM.
5. **Prediction + explanation:** the feature vector produces a class label, confidence, severity score, and (best-effort) a SHAP attribution dict, entirely as in-memory Python values.
6. **Egress to storage:** every result value above is serialized to JSON and written under the run’s output directory; a summary row is upserted into the SQLite `RunStore`.
7. **Egress to client:** either the JSON is returned directly (API path) or read back and rendered into an HTML template (page path) — both egress paths read from the exact same persisted JSON, never from stale in-memory state.
8. **Chatbot data flow (independent path):** a chat message never touches the image pipeline unless it is a photo-in-chat message (Journey 5b), in which case it briefly re-enters the flow above at step 3; otherwise it flows message → NLP → retrieval → (optional) generation → (optional) translation → response, with no interaction with the filesystem/SQLite persistence layer at all (chatbot exchanges are not persisted anywhere).

33.9 Complete API Map

The full route tables are given in Chapter 7 (JSON API) and Chapter 26 (HTML pages); consolidated here by resource:

- **Analysis (JSON):** POST `/api/analysis/single`, `/multiple`, `/zip`, `/demo`, each with an `-async` variant; GET `/api/analysis/runs/search`, GET `/api/analysis/runs/{run_id}`.
- **Analysis (HTML forms):** GET `/analyze`, POST `/analyze/single`, `/multiple`, `/zip`, `/demo`.

- **Chatbot (JSON):** POST `/api/chatbot/message`, POST `/api/chatbot/photo`, plus one further route documented in Chapter 23.
- **Results browsing (HTML):** GET `/results`, GET `/batch-results`, GET `/run/{run_id}`, GET `/run/{run_id}/image/{image_id}`.
- **Operational:** GET `/`, GET `/status`, GET `/methodology`, GET `/api/health`.

33.10 Complete Database Map

The SQLite database managed by `run_store.py` (Section 10.3) has exactly two tables:

runs One row per analysis run: `run_id`, creation timestamp, status counts (healthy/diseased/mixed), and the other summary columns queried by GET `/api/analysis/runs/search` via a whitelisted-column filter/sort mechanism (Chapter 10) that prevents SQL injection through the sort/filter query parameters.

jobs One row per async batch job (`job_queue.py`), tracking job status for polling clients.

This is the entirety of the application’s structured persistence layer — every other piece of application state (individual image results, feature vectors, SHAP values, heatmap file paths) lives as JSON on the filesystem under each run’s output directory, not in SQLite; the database exists specifically to make run *listing/searching* fast without scanning every JSON file on every request (the alternative approach `/results` itself uses, per Chapter 26’s documented filesystem-scan-vs-SQLite duplication).

33.11 Closing Note

Every claim in this report traces back to a real file, function, or recorded output in the shipped PRSV Analyzer v3.2 project, read and verified directly rather than inferred from names or assumptions. Where the project’s own behavior is incomplete, unenforced, or unwired, this report says so explicitly rather than describing an idealized system that does not exist. This is intended as the complete, accurate, single reference for understanding the system from its smallest function up through its full architecture and every real user-facing feature it provides.